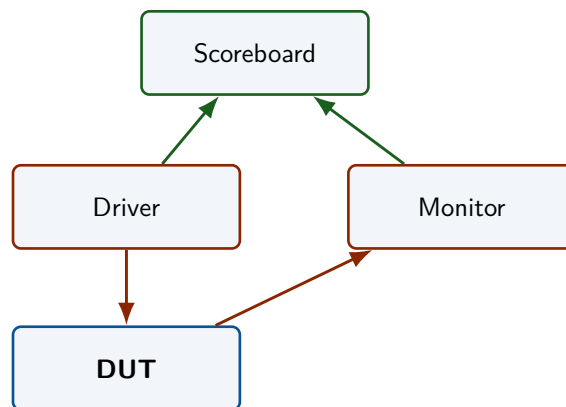


AN INTRODUCTION TO SYSTEMVERILOG

SystemVerilog for VHDL Designers

The Verification Cookbook

Classes · Constrained Random · Coverage · UVM



Luigi Giuffrida, Alessandra Dolmeta

Department of Electronics and Telecommunications
Politecnico di Torino

Draft — September 17, 2026

This booklet is one of four companions: Design, Verification (this one), Simulation, and a complete worked Example. It covers the verification-only half of SystemVerilog: classes, dynamic types, constrained randomisation, functional coverage, assertions, and UVM, none of which has a VHDL counterpart.

Preface

Verification is where SystemVerilog stops being VHDL with different punctuation. The design half of the language, covered in the *Design Cookbook*, is a translation exercise: a process is an `always` block, a generic is a parameter. The verification half is not, because VHDL has no equivalent of most of it. Classes, mailboxes, dynamic threads, constrained randomisation and functional coverage do not map onto anything you have written before.

This booklet is the **Verification Cookbook**, one of four companions covering SystemVerilog for VHDL Designers in full. It assumes you already know the design-side language (the *Design Cookbook*), and it spends its time on the part of SystemVerilog that exists purely to write better testbenches.

What this booklet is

The vocabulary, then a complete testbench, then the standard library built on top of it. Chapter 1 introduces classes, dynamic types, constrained randomisation, functional coverage, assertions and dynamic threads: none of it is synthesisable, and none of it has a VHDL counterpart. Chapter 2 assembles that vocabulary into a complete layered testbench, from nothing, using no library at all. Chapter 3 then shows that UVM is the same testbench with a standard base class in front of each part.

As throughout the series, a *trap* marks a mistake you are statistically likely to make, a *recipe* gives a pattern to copy, and a plain note supplies background you can skip on a first reading.

What this booklet is not

It is not a UVM reference manual; the Accellera UVM 1.2 user guide and the IEEE 1800.2 LRM remain the authoritative sources for anything not covered here. It does not discuss Verilator or Python testbenches in any depth: that material, including how much of this chapter's testbench survives a change of simulator, is in the *Simulation Cookbook*. The worked CORDIC example that this booklet's testbenches are built against is developed in full in the *Example Cookbook*.

Acknowledgement

The testbenches built in this booklet are exercised, throughout, against the CORDIC rotator of the *Example Cookbook*: small enough that the whole testbench fits in your head, and arithmetic enough that a golden model earns its keep.

How to Read This Booklet

Typographic conventions

- Monospaced text is code: an identifier, a keyword, a file name or a command.
- Coloured monospaced text marks a language keyword under discussion.
- Green monospaced text is a file in the accompanying source tree.
- Sans-serif names are tools: QuestaSim, Verilator, cocotb, GTKWave.

The three kinds of box

△ Trap: What a trap looks like

A trap marks a mistake that VHDL experience makes you *more* likely to commit, not less.

Recipe: What a recipe looks like

A recipe is a pattern to copy. When the text says “write it like this”, this is the box it is in.

What a note looks like

A note supplies background: where a construct came from, what the standard actually says, why a tool behaves the way it does. Skip these on a first reading and come back when something surprises you.

A rule of thumb is one line you should be able to recite from memory.

If you have been told to write a testbench by Friday

Chapter 2 is self-contained. Read it, take the layered structure, and ignore Chapter 3 entirely until somebody asks you for reuse across projects. If your simulator is Verilator or you have no licence at all, or your golden model is already written in Python, see the *Simulation Cookbook* once the structure here makes sense.

If you already know the design-side language

Start at Chapter 1: it is the vocabulary (classes, dynamic types, constrained randomisation, functional coverage, assertions, dynamic threads) that the rest of this booklet assumes. None

of it is synthesisable, and none of it has a VHDL counterpart, so there is nothing to translate here, only to learn.

The companion booklets

This is the **Verification Cookbook**. the *Design Cookbook* covers the design-side language this booklet assumes you already know. the *Simulation Cookbook* covers Verilator, QuestaSim and Python/cocotb testbenches. the *Example Cookbook* works one CORDIC design through all of it, including the testbenches built here.

Contents

Preface	v
How to Read This Booklet	vii
1 The SystemVerilog Verification Toolkit	1
1.1 Dynamic data types	1
1.2 Classes and object orientation	3
1.3 Concurrency control	6
1.4 Randomisation and constraints	10
1.5 Functional coverage	12
1.6 SystemVerilog Assertions	15
1.7 <code>program</code> blocks	18
1.8 Where this goes next	19
1.9 Checklist	19
2 Basic Testbenches	21
2.1 The Five Parts of a Testbench	21
2.2 The Literal Translation	22
2.3 Why That Testbench Is Dangerous	27
2.4 Making the Testbench Decide	31
2.5 Vectors From a File	37
2.6 The Layered Testbench	38
2.7 The Mechanics That Hold It Together	49
2.8 Assertions In and Around the Testbench	53
2.9 Running It	56
2.10 The Coverage-Closure Loop	59
2.11 Checklist	60
3 UVM Basics	63
3.1 What UVM is, and where it came from	63
3.2 The same six boxes	65
3.3 <code>uvm_object</code> versus <code>uvm_component</code>	67
3.4 Phases	68

3.5	Objections: how a test knows it is finished	71
3.6	The factory	72
3.7	The configuration database	75
3.8	Sequences and sequencers	77
3.9	The agent	82
3.10	TLM and the analysis path	83
3.11	Reporting	85
3.12	A complete UVM testbench for the CORDIC	87
3.13	The register layer	97
3.14	Running it	98
3.15	When not to use UVM	100
3.16	Checklist	101
	Further Reading	103
	Index	105

The SystemVerilog Verification Toolkit

The language features that exist only for verification: dynamic data types, classes, dynamic threads, constrained randomisation, functional coverage and assertions. None of it is synthesisable and almost none of it has a VHDL counterpart. This chapter is the vocabulary, taught one construct at a time and with no testbench architecture in the way; Chapter 2 is where it gets assembled into a working testbench.

Everything in this chapter is a tool, and a tool is easier to understand once you know what it is for. So, briefly, here is the shape of the thing being built in Chapter 2.

A modern testbench does not wiggle pins. It creates *transactions*, objects that describe one complete operation on the interface. It *randomises* them inside constraints that describe what the design is allowed to be given. It hands them to a *driver* that knows the pin-level protocol, watches the pins with an independent *monitor*, checks the results in a *scoreboard* against a reference model, and measures what the run actually exercised with *functional coverage*. Around all of it, *assertions* state protocol rules that must hold on every cycle whether anyone is looking or not.

The sections follow the path a transaction takes through that machine rather than the order of the LRM: storage first, because everything else is built out of queues; then the class that a transaction is; then the threads and the mailbox that move it between components, because a driver and a monitor are useless until they can run at once and talk to each other; then the solver that fills it in, the coverage that measures what the solver produced, and the assertions that check the pins underneath all of it. `program` blocks come last, as history.

Read it once for the shape, then use it as a reference. Nothing here is worth memorising in isolation, and nothing here is worth writing until Chapter 2 gives it a job.

What this chapter deliberately does not do

It does not build a testbench. Every construct is shown on the smallest example that demonstrates it, and questions of the form "where should this component live" or "who should own this mailbox" are left to Chapter 2, which answers them all at once with a worked example. If a section feels like it stops one step short of usefulness, that is the step Chapter 2 takes.

1.1 Dynamic data types

A testbench has to remember things: what it sent and has not yet checked, which addresses it has touched, how many of each kind of transaction it has produced. The number of things to remember is not known when the simulation elaborates, because it depends on how the run goes.

That is the problem VHDL solves badly. VHDL gives you fixed arrays, access types with manual `new` and `deallocate`, and that is all; building a queue means writing a linked list, and building a sparse memory model means writing a hash table. SystemVerilog has three dynamic containers and a string type built into the language, and they are the foundation everything later in this chapter stands on.

Dynamic arrays have a size fixed at run time:

```
1 int da [];
2 da = new[16];           // allocate
3 da = new[32](da);       // grow, preserving the first 16 elements
4 $display("%0d", da.size());
5 da.delete();           // free
```

Queues are the workhorse. They are ordered, indexable, and grow at either end:

```
1 int q [$];              // unbounded queue
2 int b [$:15];           // bounded to 16 entries
3
4 q.push_back(3); q.push_front(1);
5 int head = q.pop_front();
6 q.insert(1, 99);        // insert 99 at index 1
7 q.delete(0);            // remove element 0
8 $display("%0d entries, last = %0d", q.size(), q[$]);
9 int slice [$] = q[1:3]; // sub-queue
10 q = {};                // empty it
```

A queue is what a VHDL designer would build from an access type and a lot of pointer code. Push at one end, pop at the other, and you have the expected-value list that every scoreboard is made of.

Associative arrays are sparse maps with an arbitrary index type:

```
1 logic [31:0] mem [logic [31:0]]; // sparse memory model
2 int hits [string];              // counters keyed by name
3 byte anyidx [*];               // wildcard index type
4
5 mem[32'h8000_0000] = 32'hCAFE_BABE;
6 if (mem.exists(addr)) rdata = mem[addr];
7 hits["write"]++;
8
9 logic [31:0] k;
10 if (mem.first(k)) do $display("%h : %h", k, mem[k]); while (mem.next(k));
11 $display("%0d locations touched", mem.num());
```

An associative array indexed by a 32-bit address models a 4 GB memory that costs one entry per location actually written. Doing this in VHDL means a hash table you write yourself. Note `exists()`, and use it. Reading a key that was never written is not an error: it returns the default value of the element type, so a missing entry is indistinguishable from an entry holding zero. Some tools also *create* the entry on that read (Verilator 5.020 does; `m.num()` goes up and `m.exists()` starts returning 1), so a memory model that polls addresses it has not written can quietly grow. Test with `exists()` first, always.

Strings are a real type, not an array of characters:

```
1 string s = "cortic";
2 $display("%0d %s %s", s.len(), s.toupper(), s.substr(0,2));
3 s = {s, "_rot"};           // concatenation
4 string msg = $sprintf("x=%0d y=%0d", x, y);
5 int n = s.atoi();
```

`$sformatf` is the one to remember. It formats into a `string` instead of printing, which is how every object in this chapter builds the description of itself that a failure message will quote.

1.1.1 Array manipulation methods

Every array-like object, fixed or dynamic or queue or associative, supports a family of methods that take an optional `with` clause. The `with` clause is an expression evaluated for each element, where `item` names the element. the *Design Cookbook* lists the ones you will use.

Table 1.1. Array manipulation methods. `item` is the implicit iterator name in the `with` clause.

Method	Returns	Example
<code>sum</code>	scalar	<code>q.sum()</code>
<code>product</code>	scalar	<code>q.product()</code>
<code>and, or, xor</code>	scalar	<code>q.xor()</code>
<code>min, max</code>	queue of 1	<code>q.max()[0]</code>
<code>find</code>	queue	<code>q.find(x) with (x > 10)</code>
<code>find_first, find_last</code>	queue of 0 or 1	<code>q.find_first with (item == k)</code>
<code>find_index</code>	queue of indices	<code>q.find_index with (item < 0)</code>
<code>unique</code>	queue	<code>q.unique()</code>
<code>unique_index</code>	queue of indices	<code>q.unique_index()</code>
<code>sort, rsort</code>	in place	<code>t.sort with (item.addr)</code>
<code>reverse, shuffle</code>	in place	<code>q.shuffle()</code>
<code>size, delete</code>	nothing	<code>q.delete()</code>

Two idioms are worth memorising:

```

1 // how many transactions were writes?
2 int nwr = trs.sum() with (int'(item.is_write));
3
4 // the index of the first entry matching an address, or an empty queue
5 int idx [$] = trs.find_index with (item.addr == a);
6 if (idx.size() == 0) $error("no match for %h", a);

```

`sum()` with returning a count is the standard counting idiom: the `with` expression is evaluated per element and the results are added. Note the cast to `int`, which is not decoration.

△ Trap: The reduction is defined to happen in the element's width

1800-2017 says the accumulation for `sum` is performed in the type of the array element, so for `bit b [] = '{1,1,1,1}`; the additions are one-bit-wide and `b.sum()` is defined to be 0, the parity, not 4. Nothing warns you.

Worse, tools disagree. Verilator 5.020 returns 4 for that expression, so code that is silently wrong on a commercial simulator is silently right on the open-source one, and the bug appears only when you change tool. Icarus Verilog 12.0 does not implement `sum` on a dynamic array at all and refuses to elaborate. The portable form is the only form worth writing: `b.sum() with (int'(item))`, every time you are counting rather than adding.

1.2 Classes and object orientation

A queue holds things. The next question is what those things are, and for a testbench the answer is almost always a *transaction*: one complete operation on the interface, with the stimulus that went in, the result that came back, and enough identity to find it again in a log of ten thousand.

A SystemVerilog class is what a VHDL protected type almost is, plus inheritance, plus a garbage collector. If you have used a protected type to build a shared counter or a mutex, you already have the idea: data plus methods, with the data reachable only through the methods. What VHDL does not give you is a protected type that can be *extended*, copied, stored in a queue, or randomised.

```

1 class bus_txn;
2   rand bit [31:0] addr;
3   rand bit [31:0] data;
4   rand bit      is_write;
5   local int      id;           // not visible outside
6   static int     next_id = 0;  // one copy for the whole class
7
8   function new(bit [31:0] a = 0);
9     addr = a;
10    id   = next_id++;
11  endfunction
12
13  virtual function string convert2string();
14    return $sformatf("[%0d] %s @%h = %h", id,
15                      is_write ? "WR" : "RD", addr, data);
16  endfunction
17 endclass

```

Listing 1.1. A transaction class, the basic unit of a SystemVerilog testbench

The pieces, in VHDL terms:

- `new()` is the constructor. `bus_txn t = new();` allocates an object and returns a *handle* to it. There is no `deallocate`: objects are garbage collected when the last handle to them is dropped.
- `this` refers to the current object, exactly as in C++ or Java, and is needed only when a local name shadows a member.
- `local` and `protected` restrict visibility; the default is public, unlike a protected type where everything is private.
- `static` members belong to the class rather than to any object, so `next_id` counts every transaction ever created.
- `virtual` on a method enables polymorphism.

Two properties of a class matter more than the syntax. The first is that the locals of a class method have automatic lifetime, which is what makes it safe to call the same method from several threads at once (Section 1.3.3). The second is that a class variable is a reference, and that one catches everybody.

1.2.1 Handles are references

This is the single most common source of confusion for a VHDL designer, so it gets its own box.

△ **Trap: A class variable is a handle, and assignment copies the handle**

```

1 bus_txn a = new();
2 a.addr = 32'h1000;
3 bus_txn b = a;      // b and a name the SAME object
4 b.addr = 32'h2000;
5 $display("%h", a.addr); // 2000, not 1000

```

Assigning one handle to another does not copy the object; it makes a second name for it. A VHDL record assignment copies the whole record, and that instinct is wrong here. The symptom is always the same: a container that appears to hold N different objects and actually holds N references to one object, which carries whatever values were last written into it. Put a *copy* into the container, or construct a fresh object each time round the loop, and never store the handle you are about to reuse.

Copying is explicit, and comes in two depths:

```

1 bus_txn c = new a;           // shallow copy: new object, members copied
2                               // ... but nested handles still shared
3
4 function bus_txn clone();    // deep copy: written by hand
5     clone = new();
6     clone.addr = addr; clone.data = data; clone.is_write = is_write;
7     clone.payload = payload.clone(); // recursive, for nested objects
8 endfunction

```

`new a` performs a shallow copy: scalar members are duplicated, but a member that is itself a class handle is copied as a handle, so both objects share the nested object. UVM automates this with field macros (Chapter 3); without UVM you write `clone()` yourself.

1.2.2 Inheritance and polymorphism

```

1 virtual class txn_base;      // abstract: cannot be instantiated
2     pure virtual function string convert2string();
3     virtual function void display();
4         $display("%s", convert2string()); // works for every subclass
5     endfunction
6 endclass
7
8 class mem_txn extends txn_base;
9     rand bit [31:0] addr;
10    function new(); super.new(); endfunction
11    virtual function string convert2string();
12        return $sformatf("addr=%h", addr);
13    endfunction
14 endclass
15
16 txn_base t;                  // a base handle ...
17 mem_txn d = new();
18 t = d;                       // ... holding a derived object
19 t.display();                 // calls mem_txn's convert2string

```

A **virtual** method is resolved by the type of the *object*, not the type of the handle. A **pure virtual** method has no body and forces every subclass to supply one; a class containing one must be declared **virtual class** and cannot be instantiated. This is the mechanism the whole of UVM is built on: the framework holds base-class handles and calls virtual methods, and your subclasses supply the behaviour.

Going the other way, from a base handle to a derived one, needs a checked cast:

```

1 mem_txn d2;
2 if (!$cast(d2, t)) $fatal(1, "handle is not a mem_txn");

```

`$cast` returns 0 rather than failing when the object is not of the target type, so its return value must be checked.

1.2.3 Parameterised classes and the scope operator

```

1 class fifo_model #(type T = int, int DEPTH = 8);
2   local T q [$];
3   static int instances = 0;
4   function void put(T x);
5     if (q.size() >= DEPTH) $error("overflow");
6     q.push_back(x);
7   endfunction
8 endclass
9
10 fifo_model #(bus_txn, 16) f = new();
11 $display("%0d", fifo_model#(bus_txn,16)::instances); // :: reaches statics

```

The `::` operator reaches into a class scope without an object: static members, static methods, typedefs and parameters. Note that `fifo_model#(int,8)` and `fifo_model#(bus_txn,16)` are different types with separate static variables, which is a surprise the first time a counter you believed was global turns out to have been counting two separate populations.

VHDL

```

type counter_t is protected
  procedure incr;
  impure function get return natural;
end protected;

type counter_t is protected body
  variable n : natural := 0;
  procedure incr is begin
    n := n + 1;
  end procedure;
  impure function get return natural
  is begin return n; end function;
end protected body;

```

SystemVerilog

```

class counter_t;
  local int n = 0;
  function void incr(); n++; endfunction
  function int get(); return n; endfunction
endclass

// ... plus inheritance, copying,
// randomisation, queues of them,
// and no explicit deallocation.
counter_t c = new();

```

Say it plainly: **classes are not synthesisable**. Not "poorly supported"; not synthesisable at all, by any tool, by design. They exist for testbenches, and every line of class code you write is simulation-only.

1.3 Concurrency control

Objects on their own do nothing. Two of the components in Chapter 2, the driver and the monitor, have to run at the same time, at different rates, for the whole simulation, and they have to pass transaction objects between themselves without either one waiting on the other's schedule. That needs threads that can be created after elaboration, and a way for threads to hand each other data safely.

VHDL has neither. A VHDL process is static: it exists from elaboration to the end of simulation, and the number of processes is fixed when the design is elaborated. If you want ten concurrent bus agents you write ten processes, or one generate loop that writes them for you, and you decide how many before the run starts.

SystemVerilog threads are dynamic. You can create one at run time, wait for some of them, kill the rest, and create more.

1.3.1 fork and its endings

```

1 fork // join: wait for ALL branches

```

```

2  drive_a();
3  drive_b();
4  join
5
6  fork                                // join_any: continue after the FIRST finishes
7      begin wait_for_response(); resp_ok = 1; end
8      begin #1000ns;                 resp_ok = 0; end // a timeout
9  join_any
10 disable fork;                       // kill whatever is still running
11
12 fork                                // join_none: continue immediately
13     monitor_forever();              // a background thread
14 join_none
15
16 // later:
17 wait fork;                          // block until all my child threads are done

```

Listing 1.2. The three flavours of fork

`join` is a barrier and the one ending with a VHDL-like feel. The `join_any` plus `disable fork` pair in the middle is the canonical timeout idiom, and there is no way to write it in VHDL without a shared variable and a lot of care. `join_none` starts a background thread and returns immediately, which is how a component that must run forever gets started without blocking whoever started it.

`wait fork` is the fourth ending and the one people forget. It is not a `fork` at all: it is a statement that blocks until every thread the current thread has spawned, by any of the three forms, has finished.

△ **Trap: `disable fork` is scoped to the thread, not to the fork**

This is a language trap rather than a testbench trap, and it is worth understanding precisely because the damage is silent.

`disable fork` terminates *all* active descendant threads of the thread that executes it. Not the branches of the immediately preceding `fork`: every descendant, however long ago it was started and by which call. If a task you invoked twenty lines earlier left a `join_none` thread running in the background, that thread dies too, without a message.

The fix is to give the `disable` a bounded scope by nesting the timeout inside its own thread, so that the only descendants in existence are the two branches you care about:

```

1  fork begin : timeout_scope
2      fork
3          wait_for_response();
4          #1us;
5      join_any
6      disable fork;          // sees only timeout_scope's own children
7  end join

```

The alternative is `disable` with an explicit block label, which kills exactly that named block and nothing else. Either is fine; a bare `disable fork` in a task that anybody else may call is not.

1.3.2 Passing data between threads

SystemVerilog provides two library classes and one language construct for inter-thread communication. All three are built into the language; none of them has to be imported.

A **mailbox** is a typed, blocking FIFO. It is what a VHDL designer would build from a shared

variable and a protected type, except that the blocking semantics are already correct and already race-free.

```

1 mailbox #(bus_txn) mbx = new();    // unbounded: put() never blocks
2 mailbox #(bus_txn) mbx4 = new(4);  // bounded to 4 entries
3
4 mbx.put(t);                        // blocks while the mailbox is full
5 mbx.get(t);                        // blocks while it is empty, then removes
6 mbx.peek(t);                      // like get, but leaves the item in place
7 if (mbx.try_put(t)) ...           // non-blocking: returns 0 instead of waiting
8 if (mbx.try_get(t)) ...
9 if (mbx.try_peek(t)) ...
10 $display("%0d pending", mbx.num());

```

Bounded and unbounded are genuinely different objects, not a tuning knob. An unbounded mailbox is a pure queue with a blocking read: the producer runs as fast as it likes and memory absorbs the difference. A bounded one is a rendezvous with slack n : the producer is throttled to the consumer's rate, which is usually what you want and is always what makes a stalled consumer visible.

Note that a mailbox is untyped unless you parameterise it. Plain `mailbox mbx = new();` accepts anything, and a type error then becomes a run-time failure inside `get` rather than a compile error. Write `mailbox #(T)`.

A **semaphore** is a counting mutex. It holds a number of keys; a thread that wants the resource takes keys and blocks until enough are available, and returns them when it is done.

```

1 semaphore sem = new(1);           // one key: a plain mutex
2 sem.get(1);                       // blocks until a key is free
3 // ... exclusive section ...
4 sem.put(1);                       // return it
5
6 if (sem.try_get(1)) ...           // non-blocking

```

`new(0)` creates a semaphore with no keys, which is a barrier that some other thread must open with `put`. Note that `put` does not check that you ever held a key: putting more keys back than you took quietly increases the count and destroys the mutual exclusion.

A **named event** is a synchronisation point with no payload. It is declared with `event`, triggered with `->`, and waited on in one of two ways that behave differently.

```

1 event e;
2 -> e;                             // trigger it
3 @(e);                             // wait for the NEXT trigger
4 wait (e.triggered);               // ... or for one in this time step
5
6 wait_order(a, b, c);              // events must occur in this order

```

△ **Trap: `@(e)` and `wait (e.triggered)` are not the same wait**

`@(e)` is edge-sensitive: it blocks until the *next* trigger of `e`, and a trigger that already happened is gone. `e.triggered` is a state that is true for the whole time step in which the event fired, so a thread that reaches `wait (e.triggered)` later in the same time step still sees it.

The failure this causes is total and silent: a thread waits forever on an event that fired a few picoseconds of simulated time before it got there, and the testbench simply stops making progress with no error. It is at its most likely exactly where you least want it, between a thread that was just started with `join_none` and has not yet executed its first

statement, and the thread that started it. Prefer `wait (e.triggered)` everywhere except in a deliberate "wait for the next one" loop.

1.3.3 Static and automatic lifetime

Dynamic threads bring a trap that does not exist in a language where the number of processes is fixed. It catches everybody exactly once.

In Verilog, and by inheritance in SystemVerilog, the arguments and locals of a task or function declared in a module, interface or package have **static** lifetime by default: one copy, created at elaboration, shared by every invocation. A VHDL process variable is static in the same sense, so the concept is not new. What is new is that SystemVerilog lets you call the same task from several threads at once, and they then share those variables.

```

1 task print_after(int n);           // `int n' is STATIC here
2   #(n * 10ns);
3   $display("thread %0d at %0t", n, $time);
4 endtask
5
6 initial begin
7   for (int i = 1; i <= 4; i++) begin
8     fork
9       print_after(i);             // four concurrent calls, ONE copy of n
10    join_none
11  end
12 end

```

Listing 1.3. The bug: four threads, one variable

All four calls share the single static `n`. The first sets it to 1 and suspends at the delay; the second sets it to 2 and suspends; by the time any of them wakes, `n` is 4. The output is four identical lines, all at the same time, and the testbench appears to have started one thread instead of four.

There are two fixes and you need both.

```

1 task automatic print_after(int n); // each call gets its own n
2   #(n * 10ns);
3   $display("thread %0d at %0t", n, $time);
4 endtask
5
6 initial begin
7   for (int i = 1; i <= 4; i++) begin
8     automatic int k = i;           // a fresh k per iteration
9     fork
10      print_after(k);
11    join_none
12  end
13 end

```

Listing 1.4. The fix: automatic task, automatic loop variable

`task automatic` gives every invocation its own copy of the arguments and locals. `automatic int k = i;` inside the loop body gives every *iteration* its own variable to capture the loop counter into; without it, the forked thread reads `i` after the loop has advanced and you get the same symptom for a different reason.

△ Trap: Where the default flips, and why it bites on a refactor

Everything declared inside a **class** or a **program** already has automatic lifetime. Everything declared directly in a **module**, **interface** or **package** does not. So the trap does not appear while you are writing class-based code, and appears the moment you lift a working class method out into a module-level task to share it, at which point the same source text means something different. Write **automatic** on every task and function that is not deliberately keeping state between calls, or declare the whole enclosing unit automatic (`module automatic tb;`, `program automatic test;`), which flips the default for everything inside it. VHDL procedures are always automatic, so nothing in your background prepares you for this.

1.4 Randomisation and constraints

The transaction class exists; something has to fill it in. A VHDL testbench describes stimulus *procedurally*: a loop that walks addresses, a nested loop that walks data patterns, a hand-written list of corner cases. You write the sequence of inputs, so the test contains exactly as many cases as you had patience for.

SystemVerilog lets you describe the *legal input space* declaratively and asks a solver for points inside it. This is a different way of thinking, and it is the single largest productivity difference between the two languages in verification: the test now contains one description and a number you can raise from a hundred to a hundred thousand overnight.

```

1 class bus_txn;
2   rand bit [31:0] addr;
3   rand bit [7:0] len;
4   rand bit [1:0] size;           // weighted below; see the note on randc
5   rand bit is_write;
6
7   constraint c_align { addr[1:0] == 2'b00; }
8   constraint c_range { addr inside {[32'h0000_0000 : 32'h0000_FFFF],
9                               [32'h8000_0000 : 32'h8000_0FFF]}; }
10  constraint c_len    { len inside {[1:16]};
11                      is_write -> len <= 8; }
12  constraint c_dist   { size dist {0 := 1, 1 := 2, 2 := 5, 3 := 2}; }
13  constraint c_noxing { (addr[11:0] + len*4) < 4096; } // no 4k crossing
14 endclass
15
16 bus_txn t = new();
17 if (!t.randomize()) $fatal(1, "constraints unsatisfiable");

```

Listing 1.5. A constrained transaction

Every variable marked **rand** is chosen by the solver so that all active constraints hold simultaneously. The solver considers the constraints as a system, not in order: **c_noxing** can force **len** down even though **len** appears first in the file. That is the property that makes the declarative form worth learning, and it is also why the distribution of the result is rarely the one you pictured.

randc is the other declaration you will meet. It makes a variable *cyclic*: the solver visits every value of its range once before any value repeats, which is how you sweep a small field without writing the loop. It is tempting here, because **size** has only four values. It is not allowed here: 1800-2017 forbids a **dist** constraint on a **randc** variable, and for a good reason, since a cyclic variable already has a fixed distribution. Choose one, weighted or cyclic, per variable.

△ **Trap: Ignoring the return value of `randomize()`**

`randomize()` returns 1 on success and 0 when the solver could not satisfy the constraints. It does not throw, print or stop.

Written as a bare statement, `t.randomize();` compiles, discards the result, and leaves the object holding whatever values it had before, which for a freshly constructed object is all zeros. The test then runs happily, sending the same transaction a thousand times, and reports a full pass having verified one vector. `void'(t.randomize());` is worse than the bare call, not better, because it documents in the source that you meant to ignore a failure.

The only acceptable forms are `if (!t.randomize()) $fatal(1, "...");` or a macro that expands to it. Every serious lint rule flags a discarded `randomize()` return value, and every team turns that rule on after being bitten once. When it does fail, use the tool's constraint debugger to find the contradiction rather than deleting constraints until the solver stops complaining.

Table 1.2. Constraint constructs.

Construct	Meaning
<code>a inside {[1:9], 15}</code>	set and range membership
<code>a -> b</code>	implication: if <code>a</code> holds then <code>b</code> must
<code>if (a) b; else c;</code>	the same thing, written imperatively
<code>x dist {1 := 4, 2 := 1}</code>	weight <i>per value</i> , ratio 4:1
<code>x dist {[1:4] := 8}</code>	weight 8 shared <i>across</i> the range
<code>solve a before b</code>	ordering hint; changes the distribution only
<code>foreach (arr[i]) arr[i] < 10;</code>	one constraint per element
<code>unique {a, b, c}</code>	all different
<code>soft x == 0;</code>	default, overridable by a later constraint
<code>x.size() inside {[1:8]}</code>	constrain a dynamic array's size

1.4.1 Controlling randomisation from the outside

The class states what is *legal*. A test states what it wants *this time*, and it does so without editing the class. That separation is the reason the mechanism is worth its complexity.

Inline constraints add requirements for one call only:

```
1 if (!t.randomize() with { addr inside {[32'h1000:32'h1FFF]};
2     is_write == 1'b1; })
3     $fatal(1, "cannot build a write into the low window");
```

Individual variables and constraint blocks can be switched off:

```
1 t.size.rand_mode(0);           // stop randomising size; keep its value
2 t.c_align.constraint_mode(0); // allow unaligned addresses for this test
```

And two hooks run around every call:

```
1 function void pre_randomize(); ... endfunction // set up state
2 function void post_randomize();           // fix up derived fields
3     crc = compute_crc(addr, data);
4 endfunction
```

For variables that are not class members there is `std::randomize()`:

```

1 int a, b;
2 if (!std::randomize(a, b) with { a < b; b < 100; }) $error("no solution");

```

And for cases where a distribution is all you need, without a solver:

```

1 x = $urandom();           // 32-bit unsigned
2 y = $urandom_range(15, 4); // inclusive, either argument order
3 randcase                 // weighted branch selection
4   3: send_read();
5   1: send_write();
6   1: send_idle();
7 endrandcase

```

△ Trap: The solver does not distribute the way you expect

Given `rand bit [3:0] a; rand bit b; constraint c { b -> a == 0; }` you might expect `b` to be 1 half the time. It is not. The solver picks uniformly over the *solution space*: there are 16 solutions with `b == 0` and exactly one with `b == 1`, so `b` is 1 about one time in seventeen. `solve b before a;` restores the intuitive split by telling the solver to choose `b` first. This is not a bug and it is not tool-specific; it follows from the definition. Whenever a random variable constrains the range of another, check the distribution with a `covergroup` before trusting it.

Recipe: Reproducibility

A random test that cannot be replayed is worthless. Fix the seed on the command line (`vsim -sv_seed 12345` or the equivalent), print it in the banner of every run, and record it with the failure. Do not call `$urandom` from code whose execution order depends on the number of threads, because the per-thread random stability guarantees of the LRM depend on objects being created in a deterministic order.

Constraints need a real solver, and not every tool has one

Constrained randomisation is the feature that most sharply divides simulators. QuestaSim and the other commercial simulators ship a constraint solver and implement the whole of clause 18. The open-source tools do not, and they fail in two different and equally dangerous ways. Icarus Verilog 12.0 rejects a constraint block outright (sorry: Constraint declarations not supported), which is at least honest. Verilator 5.020 compiles the class, accepts `randomize()`, and emits `%Warning-CONSTRAINTIGN: Constraint ignored (unsupported)` once per constraint block, then randomises the `rand` variables with every constraint discarded. Compiled exactly as written, the *Design Cookbook* produces five such warnings and a first draw of `len = 156` against `len inside {[1:16]}`. A warning you filtered out of the log is the only thing standing between that and a test which believes its stimulus is legal. the *Simulation Cookbook* returns to what this means for an open-source flow.

1.5 Functional coverage

Random stimulus raises an immediate question: what did it actually test? The solver was told what is legal, not what is interesting, and it has no idea which of the legal points matter. The answer is functional coverage, and the distinction from code coverage is the first thing to get

straight, because the audience for this book has usually met only the second.

Code coverage is measured by the tool from the RTL source with no help from you: which lines executed, which branches were taken both ways, which expressions took each value, which state machine states and transitions were visited. It answers "did the stimulus exercise the code I wrote?" It cannot tell you about a feature you forgot to implement, because there is no code for it.

Functional coverage is written by you, from the specification. It answers "did the stimulus exercise the situations the specification cares about?" A design that ignores byte enables can have 100% code coverage and 0% coverage of the byte-enable cross, because you wrote a coverpoint for byte enables and the tests never varied them.

You need both. Code coverage finds untested code; functional coverage finds untested intent.

```

1  covergroup bus_cg @(posedge clk);
2      option.per_instance = 1;
3      option.at_least      = 2;          // a bin counts as hit after 2 samples
4
5      cp_kind : coverpoint kind {
6          bins rd    = {READ};
7          bins wr    = {WRITE};
8          bins idle  = {IDLE};
9      }
10
11     cp_len : coverpoint len {
12         bins one    = {1};
13         bins small  = {[2:7]};
14         bins burst[4] = {[8:63]};      // auto-split into 4 bins
15         bins max    = {64};
16         illegal_bins bad = {0};
17         ignore_bins rsv = {[65:255]};
18     }
19
20     cp_addr : coverpoint addr[15:12] {
21         wildcard bins low  = {4'b00??};
22         bins      other   = default;
23     }
24
25     cp_seq : coverpoint kind {
26         bins rd_then_wr = (READ => WRITE);
27         bins back2back  = (WRITE [*3]);
28     }
29
30     x_kind_len : cross cp_kind, cp_len {
31         ignore_bins no_idle_len = binsof(cp_kind.idle);
32         bins      long_writes = binsof(cp_kind.wr) &&
33                                binsof(cp_len) intersect {[8:64]};
34     }
35 endgroup
36
37 bus_cg cg = new();          // covergroups must be constructed

```

Listing 1.6. A covergroup for the bus transaction

The vocabulary, briefly:

- A **coverpoint** is a variable or expression whose values you care about. Without explicit bins, the tool creates one automatically per value, up to `option.auto_bin_max` (64 by default).
- **bins** name value sets. `bins b[4] = {[8:63]}` splits the range into four; `bins b[] =`

{[8:63]} makes one bin per value.

- `illegal_bins` turn a hit into a runtime error; `ignore_bins` remove values from the denominator, which is how you exclude unreachable cases honestly.
- `default` catches everything not named, and does not count towards coverage. It is a debugging aid, not a way to reach 100%.
- Transition bins use `=>`: `(READ => WRITE)` covers the value pair on consecutive samples, and `[*3]` repeats.
- `cross` builds the product of two coverpoints; `binsof/intersect` filter the product down to the combinations that mean something.
- `option.per_instance = 1` keeps a separate record per covergroup instance instead of merging them, which you almost always want when the same agent is instantiated several times.

Sampling is either automatic, through the event in the `covergroup` header, or manual:

```

1 covergroup txn_cg (ref bus_txn t); // no event: sampled explicitly
2   cp_addr : coverpoint t.addr { bins lo = {[0:'hFFFF']};
3                                   bins hi = {[ 'h8000_0000:'h8000_0FFF]}; }
4 endgroup
5 ...
6 tcg.sample();
7 $display("coverage so far: %0.2f%%", tcg.get_coverage());

```

Explicit `sample()` is the better habit, because it lets you sample a complete, validated transaction rather than whatever the wires happened to hold at an arbitrary clock edge. Chapter 2 shows where in a testbench that call belongs.

△ Trap: A coverpoint with no bins can report 100% and mean nothing

`coverpoint addr`; on a 32-bit address does not create four billion bins. It creates `auto_bin_max` of them (64), each covering a huge range, and hitting one address in each range reports full coverage. Worse, a coverpoint on a single-bit signal that never changes will report 50% and look like progress. Always write explicit bins that correspond to something in the specification. Auto-bins are for a first look, never for a coverage sign-off number.

△ Trap: A covergroup is a type; nothing is sampled until you construct it

`covergroup bus_cg; ... endgroup` declares a type, exactly as `class` does. Until `cg = new()`; has run there is no instance and every `sample()` call does nothing useful. The coverage report then shows 0% and looks like a stimulus problem rather than a missing line. A covergroup declared inside a class may only be constructed in that class's constructor, which at least makes the right place to put the line obvious.

Write the covergroup before the tests, from the specification. A coverage model written after the tests measures the tests, not the specification.

Coverage numbers are produced in order to be closed, and closing them is a loop rather than a measurement: run many seeds, merge the databases, look at the holes, and decide for each one whether it needs a directed test, a constraint change or an honest `ignore_bins`. That loop is a methodology question rather than a language question, and Section 2.10 walks through it with

real numbers.

Covergroups are a commercial-simulator feature

Unlike constraints, where Verilator silently ignores what it cannot do, covergroups are rejected outright. Verilator 5.020 reports `%Error-UNSUPPORTED: covergroup`, along with a separate error for each coverpoint and each bin specification, and stops. Icarus Verilog 12.0 does not parse the construct at all: a `covergroup` block is a syntax error. Functional coverage in this form needs QuestaSim or another commercial simulator. the *Simulation Cookbook* describes what Verilator offers instead, which is line and toggle coverage, a different and much weaker thing.

1.6 SystemVerilog Assertions

Coverage says what happened. Something still has to say what was allowed to happen, on every cycle, whether or not a scoreboard was watching.

A VHDL `assert ... report ... severity` statement checks a condition at one instant. Checking something that spans time, such as "grant follows request within three cycles", means writing a process with a counter, and the process is usually longer and buggier than the design it checks.

SVA is a temporal language embedded in SystemVerilog for exactly this. It is also the one construct in this chapter that a design engineer writes as often as a verification engineer, because an assertion inside a module is a statement of the module's contract, and `bind` (the *Design Cookbook*) lets you attach one to a module you do not own.

1.6.1 Immediate and concurrent

An *immediate* assertion is the VHDL one, in procedural code:

```
1 always_comb begin
2     a_onehot: assert ($onehot(sel))
3     else $error("sel = %b is not one-hot", sel);
4 end
```

A *concurrent* assertion is a separate, clocked, always-running check:

```
1 a_gnt: assert property (@(posedge clk) disable iff (!rst_n)
2     req |-> ##[1:3] gnt)
3     else $error("no grant within 3 cycles of request");
```

1.6.2 Sampling semantics, and why they matter

A concurrent assertion does not read signals when it runs. It reads *sampled values*, taken in the Preponed region of the clock tick, before any design process has executed in that time step. This is the same idea as the clocking-block input skew of the *Design Cookbook*, and it exists for the same reason: an assertion must see the values the flip-flops saw, not the values they are producing.

That gives you a set of sampled-value functions, all of which operate on the preponed values:

```
1 $past(sig)           // the sampled value one clock tick ago
2 $past(sig, 3)        // three ticks ago
3 $rose(sig)           // sampled value went 0 -> 1 at this tick
4 $fell(sig)           // 1 -> 0
```

```

5 $stable(sig)    // unchanged since the previous tick
6 $changed(sig)   // the negation of $stable

```

△ **Trap: An assertion sees the *old* value at the edge**

```

1 always_ff @(posedge clk) count <= count + 1;
2 a_inc: assert property (@(posedge clk) count == $past(count) + 1);

```

This passes, and the reason surprises people. At the edge where `count` becomes 5, the assertion samples `count` in the Preponed region, so it reads 4, and `$past(count)` reads 3. The assertion is comparing the two *previous* values, and it never sees the value being assigned at the current edge. Every concurrent assertion is one delta behind the procedural code around it. Write `$past` expressions in terms of what the register held *before* this edge, and cross-check the first cycle after reset, where `$past` of an unknown is `X` and comparisons quietly fail.

1.6.3 Sequences and properties

```

1 module handshake_chk #(parameter int W = 16) (
2   input logic clk, rst_n,
3   input logic valid, ready,
4   input logic [W-1:0] data
5 );
6 // 1. valid must not be withdrawn before the transfer completes
7 property p_valid_stable;
8   @(posedge clk) disable iff (!rst_n)
9     valid && !ready |>= valid;
10 endproperty
11
12 // 2. the payload must not change while the transfer is pending
13 property p_data_stable;
14   @(posedge clk) disable iff (!rst_n)
15     valid && !ready |>= $stable(data);
16 endproperty
17
18 // 3. every request is eventually granted, within 32 cycles
19 property p_no_starve;
20   @(posedge clk) disable iff (!rst_n)
21     $rose(valid) |-> ##[1:32] (valid && ready);
22 endproperty
23
24 a_valid_stable: assert property (p_valid_stable)
25   else $error("valid dropped at %t", $time);
26 a_data_stable : assert property (p_data_stable);
27 a_no_starve   : assert property (p_no_starve);
28
29 // measurement, not checking:
30 c_transfer    : cover property (@(posedge clk) valid && ready);
31 c_backpressure: cover property (@(posedge clk) valid && !ready [*4]);
32 endmodule

```

Listing 1.7. A valid/ready handshake checker

the *Design Cookbook* is a complete, useful checker. It is a plain module, so it can be instantiated normally, or attached to a design you do not own with `bind` (the *Design Cookbook*); the *Example Cookbook* attaches this one to the CORDIC rotator that way. Three assertions and two covers replace perhaps eighty lines of VHDL checking process.

The operators, in order of how often you will need them, appear in the *Design Cookbook*.

Table 1.3. The assertion operators worth learning first.

Operator	Meaning
$a \mid\rightarrow b$	overlapping implication: b starts on the same tick
$a \mid\Rightarrow b$	non-overlapping: b starts on the next tick
$\#\#n$	delay of exactly n clock ticks
$\#\#[a:b]$	delay between a and b ticks
$\#\#[1:\$]$	eventually, unbounded (needs a liveness-aware tool)
$\text{disable iff } (c)$	abandon any attempt in progress while c
$a[*n]$	a holds on n consecutive ticks
$a[->n]$	a holds on the n -th occurrence, gaps allowed
$a[=n]$	n occurrences, the last need not be at the end
$a \text{ throughout } b$	a holds for the whole duration of b
$a \text{ within } b$	sequence a occurs inside sequence b
$a \text{ intersect } b$	both hold and end on the same tick
$\text{first_match}(a)$	take only the earliest match of a
$a \text{ and } b, a \text{ or } b$	both / either sequence matches
$\text{not } p$	the property must never hold

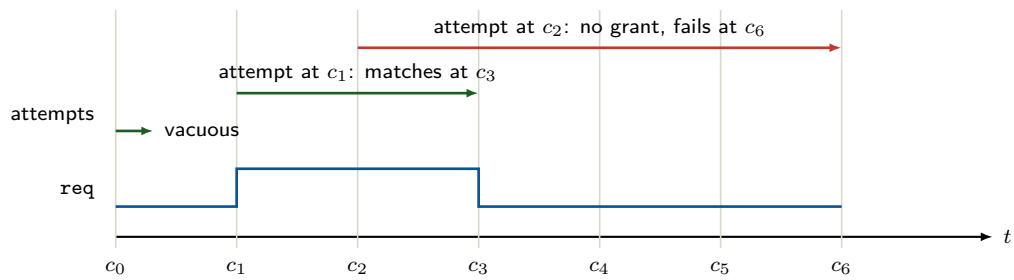


Figure 1.1. A concurrent assertion starts a new attempt at every clock tick. Ticks where the antecedent is false pass vacuously; several attempts may be in flight at once, and each one succeeds or fails independently.

the *Design Cookbook* shows the mental model that most often goes missing. A concurrent assertion is not a single check. At *every* tick of its clock, a new evaluation attempt begins. If the antecedent (the left side of $\mid\rightarrow$) is false, the attempt passes *vacuously* and costs nothing. If it is true, the attempt stays alive until the consequent succeeds or fails. Several attempts overlap in time. This is why `first_match` exists and why a badly written property can report several failures for one bug.

1.6.4 assert, assume, cover, restrict

The same property can be used four ways, and the meaning depends on the tool.

- **assert**: in simulation, check and report on failure. In formal, a proof obligation.
- **assume**: in simulation, usually checked like an assert but reported as an environment error. In formal, a *constraint* on the inputs, so the prover only considers traces where it holds. Assuming too much is the classic way to prove a false theorem.
- **cover**: never fails. It reports how many times the sequence occurred. In formal, a request to find a trace that reaches it, which is also the best way to sanity-check that your assumptions have not made the design unreachable.

- **restrict**: formal only, ignored by simulators. A constraint used to limit the proof space deliberately, for example to prove a property only for aligned addresses.

VHDL

```
-- PSL, in a VHDL comment or,
-- since VHDL-2008, directly:
default clock is rising_edge(clk);

property p_gnt is
  always (req -> next_e[1 to 3] (gnt));

assert p_gnt
  report "no grant in 3 cycles";
```

SystemVerilog

```
property p_gnt;
  @(posedge clk)
    req |-> ##[1:3] gnt;
endproperty

a_gnt: assert property (p_gnt)
  else $error("no grant in 3");
```

If you have met PSL, SVA will feel familiar: both descend from the same temporal-logic tradition, both have a boolean layer, a temporal layer and a verification layer, and the operators map closely (`next_e[1 to 3]` against `##[1:3]`, `until` against `throughout`). The practical differences are that SVA is part of the language rather than a separate standard, that its tool support is far wider, and that it can call into SystemVerilog functions and covergroups directly. PSL remains a reasonable choice for a VHDL design, and QuestaSim accepts it in both the VHDL and the Verilog flavours.

Where to put assertions

Three places, in increasing order of preference for RTL: inline in the module (fine for small, obviously-local checks such as one-hot state); in a separate checker module bound with `bind` (the default for anything non-trivial); or in the interface itself, so that every instance of the protocol is checked wherever it appears. The last is how professional verification IP ships its protocol checks.

Which half of SVA an open-source tool gives you

SVA support is not all-or-nothing, and Verilator 5.020 draws the line in a place worth knowing. With `-assert` it accepts immediate assertions, `assert property` and `cover property` with a clock and `disable iff`, the implication operators `|->` and `!>`, and the sampled-value functions `$past`, `$rose`, `$fell` and `$stable`. A property built from those runs, and a failure prints a proper message naming the assertion label. What it rejects is the temporal layer: any `##` delay, including a plain `##1`, gives `%Error-UNSUPPORTED: ##` (in sequence expression), and any `[*n]` repetition gives `%Error-UNSUPPORTED: boolean abbrev.` So `p_valid_stable` and `p_data_stable` from the *Design Cookbook* compile and run under Verilator, and `p_no_starve` and `c_backpressure` do not. Write the single-cycle properties first; they catch most protocol bugs and they are the portable ones.

1.7 program blocks

One construct remains, and it is here so that you recognise it rather than so that you use it.

A **program** block looks like a module but its code runs in the Reactive region of the scheduler, after all design activity in that time step has settled. It was introduced in SystemVerilog-3.1 to solve the testbench race problem: a testbench written in an ordinary `initial` block competes with design processes in the Active region, and whether it sees the old or the new value of a signal depends on the simulator's scheduling.

```

1 program automatic test (simple_bus.tb bus);
2   initial begin
3     bus.cb.req <= 1'b1;
4     @(bus.cb);
5     // ... implicit $finish when the program's initial blocks end
6   end
7 endprogram

```

The honest modern view is that clocking blocks solved the same problem more precisely, and better. Once every testbench signal goes through a clocking block with defined input and output skews, the region a testbench statement executes in stops mattering. UVM does not use program blocks; most current methodology guides advise against them; Verilator does not support them. Recognise one when you meet it in older code, understand why it is there, and do not write new ones. The one thing worth keeping from the design is the automatic lifetime: `program automatic` was the original way to flip the default of Section 1.3.3, and `module automatic` does the same job in code you would actually write today.

1.8 Where this goes next

You now have every construct a SystemVerilog testbench is made of, and no testbench. That is deliberate, and it is the last thing this chapter has to say.

A queue, a class, a mailbox, a constraint block, a covergroup and a property are individually unremarkable, and the reason to learn them is what they become when they are put together: a component that reconstructs transactions from pins without being told what was sent, a generator whose output you can multiply by a thousand without writing a thousand cases, and a verdict that can be trusted because it can be shown to fail. Chapter 2 assembles exactly these pieces, in this order, around the CORDIC rotator, and it also answers the questions this chapter kept postponing: which component owns which mailbox, where the covergroup is sampled, when the run is allowed to stop. Chapter 3 then replaces the hand-written scaffolding with a standard library, and changes nothing else.

Read Chapter 2 next. If a construct here did not make sense in isolation, it will make sense when you see the job it does.

1.9 Checklist

- **A class handle is a reference.** `b = a` gives you two names for one object. Copy explicitly, or construct a fresh object each time round the loop.
- **Cast before you count.** `q.sum()` accumulates in the element's width, so write `q.sum() with (int'(item))` whenever the answer is a count. Tools disagree about this; the cast is right everywhere.
- **Declare every task and function `automatic`,** or accept that two threads share one variable. Class and `program` members already are; module, interface and package members are not.
- **Contain every disable fork** inside its own `fork ... join`, or use a labelled block. A bare one kills every descendant thread you have, including a monitor somebody else started.
- **Prefer `wait (e.triggered)` to `@(e)`.** The first is race-free within the time step; the second loses a trigger that has already happened and then waits forever.

- **Always check what `randomize()` returned.** A failed solve leaves the old values in place and says nothing. `void'(t.randomize())` is worse than the bare call.
- **Constrain the legal input space**, not the test you happen to want. The test belongs in an inline with `{...}` clause.
- **Seed deliberately and record the seed.** A random test you cannot reproduce is not a test.
- **Coverage answers a different question from checking.** Passing tells you nothing about what you exercised.
- **A covergroup with no bins** auto-generates them, and auto-generated bins on a 32-bit value are meaningless. Write them, and remember that the covergroup does nothing until `new()` runs.
- **Concurrent assertions sample in the Preponed region**, so they see the value that existed *before* the edge. This is what you want, and it is not what you expect on the first day.
- **Check what your simulator implements.** Constraints, covergroups and the temporal layer of SVA are the three features an open-source tool is most likely to reject or, worse, to ignore.

Basic Testbenches

You can already write a VHDL testbench, and after Chapter 1 you know the SystemVerilog constructs a verification engineer uses. This chapter puts the two together, one layer at a time: first a literal translation, then the race conditions that translation hides, then self-checking, then file-based vectors, and finally a layered testbench with a transaction, a generator, a driver, a monitor, a scoreboard and a coverage collector. At the end you will be able to write a complete class-based testbench without importing a single line of UVM, and you will understand exactly which problem UVM was invented to solve.

Verification is where SystemVerilog stops being VHDL with different punctuation. The design half of the language, covered in the *Design Cookbook*, is a translation exercise. The verification half is not, because VHDL has no equivalent of most of it: classes, mailboxes, dynamic threads, constrained randomisation and functional coverage do not map onto anything you have written before.

Chapter 1 taught those constructs one at a time. This chapter is the other half of the job, and the division of labour between the two is worth stating once, because it governs everything that follows. Chapter 1 owns the *construct*: what a mailbox is, what `fork ... join_none` does, what a covergroup declares. This chapter owns the *use*: why the monitor puts a transaction into a mailbox rather than calling the scoreboard, why the driver runs two forked threads that do not know about each other, and why the covergroup that measures your stimulus tells you nothing about the design. If a piece of syntax here is unfamiliar, look it up there; if a piece of structure here seems arbitrary, the answer is in this chapter.

Every testbench, in every language, does the same five things, and the whole of this chapter is about *organising* those five things better. So we start by naming them.

2.1 The Five Parts of a Testbench

A testbench, whether it is fifty lines of VHDL or fifty thousand lines of SystemVerilog, consists of exactly five parts:

1. **The device under test.** One instance of the thing you are verifying, plus whatever wires it needs.
2. **Clock and reset generation.** Free-running, independent of everything else, and usually the first thing you write.
3. **Stimulus.** Something that puts values on the inputs, in a legal order, at legal times.
4. **Checking.** Something that decides whether the outputs are right.
5. **End of test.** Something that decides when to stop, and reports the verdict.

Figure 2.1 draws them. Nothing in the rest of this chapter adds a sixth box. The layered testbench of Section 2.6 looks far more complicated than a VHDL testbench, and it is, but

every one of its classes belongs to box 3 or box 4. The complexity buys separation, reuse and the ability to run ten thousand random transactions instead of the twelve you thought of by hand. It does not buy new functionality.

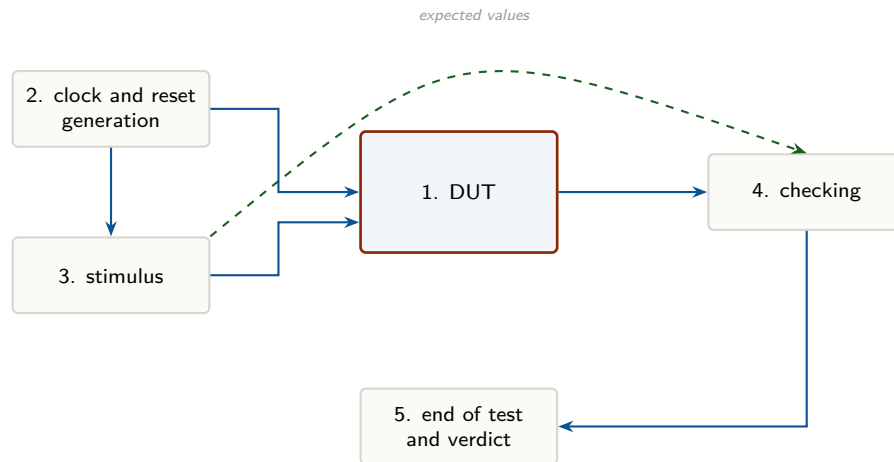


Figure 2.1. The five parts of any testbench. Every construct in this chapter belongs to part 3 or part 4.

Keep Figure 2.1 in mind. When a class-based testbench stops making sense, the fastest way back is to ask which of the five boxes the class you are staring at belongs to.

If you cannot say which of the five parts a piece of testbench code belongs to, it is in the wrong file.

2.2 The Literal Translation

Start with something small enough to hold in your head: Listing 2.1 is an eight-bit synchronous accumulator with an active-low asynchronous reset and an enable. The VHDL version is the architecture you would write without thinking about it, and it is not reproduced here.

```

1  `timescale 1ns/1ps
2
3  module accu (
4      input logic    clk,
5      input logic    rst_n,
6      input logic    en,
7      input logic [7:0] d,
8      output logic [7:0] q
9  );
10
11     logic [7:0] acc;
12
13     always_ff @(posedge clk or negedge rst_n) begin
14         if (!rst_n)    acc <= '0;
15         else if (en)   acc <= acc + d;
16     end
17
18     assign q = acc;
19
20 endmodule : accu

```

Listing 2.1. An eight-bit accumulator, the DUT for this section

Now the testbench. Listing 2.2 is a perfectly ordinary VHDL testbench for the VHDL accumulator: reset, four accumulate cycles with the values 10, 20, 30 and 40, then one check that the result is 100.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity tb_accu is
6  end entity tb_accu;
7
8  architecture behavioral of tb_accu is
9
10     constant TCLK : time := 10 ns;
11
12     signal clk      : std_logic := '0';
13     signal rst_n    : std_logic := '0';
14     signal en       : std_logic := '0';
15     signal d        : unsigned(7 downto 0) := (others => '0');
16     signal q        : unsigned(7 downto 0);
17     signal done     : boolean := false;
18
19 begin
20
21     dut : entity work.accu
22         port map (clk => clk, rst_n => rst_n, en => en,
23                 d => d, q => q);
24
25     clk <= not clk after TCLK / 2 when not done else '0';
26
27     stim : process is
28     begin
29         rst_n <= '0';
30         wait for 3 * TCLK;
31         wait until rising_edge(clk);
32         rst_n <= '1';
33
34         for i in 1 to 4 loop
35             wait until rising_edge(clk);
36             wait for 1 ns;
37             en <= '1';
38             d <= to_unsigned(i * 10, 8);
39         end loop;
40
41         wait until rising_edge(clk);
42         wait for 1 ns;
43         en <= '0';
44
45         wait until rising_edge(clk);
46         wait for 1 ns;
47         assert q = to_unsigned(100, 8)
48             report "accumulator wrong: expected 100, got "
49                 & integer'image(to_integer(q))
50             severity error;
51
52         report "test finished" severity note;
53         done <= true;
54         wait;
55     end process stim;
56
57 end architecture behavioral;

```

Listing 2.2. A conventional VHDL testbench

Listing 2.3 is the line-by-line SystemVerilog translation. Read it against the VHDL before reading the commentary.

```

1  `timescale 1ns/1ps
2
3  module tb_accu;
4
5      localparam time TCLK = 10ns;
6
7      logic      clk      = 1'b0;
8      logic      rst_n    = 1'b0;
9      logic      en       = 1'b0;
10     logic [7:0] d       = '0;
11     logic [7:0] q;
12
13     accu dut (.clk(clk), .rst_n(rst_n), .en(en),
14             .d(d), .q(q));
15
16     always #(TCLK/2) clk = ~clk;
17
18     initial begin
19         rst_n = 1'b0;
20         #(3*TCLK);
21         @(posedge clk);
22         rst_n = 1'b1;
23
24         for (int i = 1; i <= 4; i++) begin
25             @(posedge clk);
26             #1ns;
27             en = 1'b1;
28             d  = 8'(i * 10);
29         end
30
31         @(posedge clk);
32         #1ns;
33         en = 1'b0;
34
35         @(posedge clk);
36         #1ns;
37         assert (q == 8'd100)
38             else $error("accumulator wrong: expected 100, got %0d", q);
39
40         $display("[%0t] test finished", $time);
41         $finish;
42     end
43
44 endmodule : tb_accu

```

Listing 2.3. The same testbench translated literally into SystemVerilog

Six things changed. Each of them deserves a paragraph.

2.2.1 The timescale

the *Design Cookbook* explains ‘timescale, the scoped timeunit / timeprecision alternative, and the rule by which a file with no directive silently inherits whichever one the compiler read last. Read that gotcha again before you write a testbench, because a testbench is where it does its damage: the clock generator is the one piece of code whose behaviour depends entirely on what #10 means, and a half-period that has quietly become picoseconds looks exactly like a simulation that hangs. The second failure mode is precision. With 1ns/1ns the delay #0.5 is

rounded to #1 without a warning in most tools, so a 100 MHz clock silently becomes 50 MHz and every timing number in your report is wrong by a factor of two.

'timescale 1ns/1ps at the top of every file. Deviate only when you are modelling something sub-picosecond, which in digital verification you are not.

2.2.2 initial versus a process with a final wait

An initial block is a VHDL process with no sensitivity list that ends in wait; (the *Design Cookbook*). The difference that matters to a testbench writer is the default. A VHDL process loops back to the top unless you stop it, so the one-shot stimulus idiom needs an explicit final wait; an initial block runs once and its thread terminates, so the *forever* case is the one you must ask for, with always or with forever inside an initial.

VHDL

```
stim : process is
begin
    -- do something once
    a <= '1';
    wait for 10 ns;
    a <= '0';
    wait;           -- never resumes
end process stim;

tick : process is
begin
    -- runs forever
    clk <= not clk;
    wait for 5 ns;
end process tick;
```

SystemVerilog

```
initial begin
    // do something once
    a = 1'b1;
    #10ns;
    a = 1'b0;
end           // thread ends here

always begin
    // runs forever
    clk = ~clk;
    #5ns;
end
```

Several initial blocks in the same module all start at time zero and run concurrently, exactly like several VHDL processes. The order in which they start is not defined, which is the first hint of the trouble in Section 2.3.

2.2.3 Clock generation

The VHDL idiom `clk <= not clk after 5 ns;` is a concurrent signal assignment: it re-executes every time `clk` changes, so it oscillates on its own. The SystemVerilog idiom is procedural.

VHDL

```
-- concurrent assignment
clk <= not clk after TCLK/2;

-- or, in a process
process is
begin
    clk <= '0';
    wait for TCLK/2;
    clk <= '1';
    wait for TCLK/2;
end process;
```

SystemVerilog

```
// procedural, one statement
always #(TCLK/2) clk = ~clk;

// or, spelled out
always begin
    clk = 1'b0;
    #(TCLK/2);
    clk = 1'b1;
    #(TCLK/2);
end
```

Two details. First, `clk` must have an initial value: `1'bx` is `1'bx`, so an uninitialised clock stays `x` forever and nothing ever happens. The same trap exists in VHDL, where `not 'U` is `'U`, which

is why you already write `signal clk : std_logic := '0';`. Do the same in SystemVerilog: `logic clk = 1'b0;`. A variable initialiser in a module is executed once at time zero, before any initial block.

Second, use a blocking assignment for the clock. This is one of the very few places where `=` is correct in a testbench, and the reason is explained in Section 2.3: the clock must change in the Active region so that everything sensitive to the edge sees a clean, single event.

△ **Trap:** `always #5 clk <= ~clk;` is wrong

With a non-blocking assignment the clock edge happens in the NBA region, the same region where the design's flip-flops update, and every sampling guarantee you thought you had is gone. Some tools warn, some do not. Use `clk = ~clk,` always.

2.2.4 Reset sequencing

There is nothing new here except syntax, but note the shape of the idiom in Listing 2.3: hold reset asserted for a few clock periods, then release it *synchronously*, on a clock edge, so that the release has a defined relationship to the first active edge the design sees. An asynchronous release at an arbitrary time simulates fine, but every run then starts with a slightly different alignment, and in a gate-level simulation with real timing checks the recovery/removal check on the reset pin will fire.

Recipe: A reset task, not a reset paragraph

Put the reset sequence in a task from the first day, even in a flat testbench: `task automatic apply_reset(int unsigned cycles = 5);` that drives `rst_n` low, waits `cycles` clock edges and releases it non-blockingly on an edge. When the testbench grows into the layered form of Section 2.6, that task becomes `cordic_drv::reset()` unchanged. Note the default argument value, which VHDL also has, and which is the cheapest way to keep a task configurable without a second overload.

2.2.5 Ending the simulation

A VHDL testbench normally ends by *starvation*: every process has suspended on a `wait` that will never resume, no signal has a pending transaction, so the simulator has no next event and stops. That is why Listing 2.2 has the `done` signal: without it the clock generator would keep scheduling events forever.

SystemVerilog does not rely on starvation. `$finish` stops the simulator immediately and returns control to the shell; `$stop` suspends it and drops you into the interactive prompt, which is what you want under a debugger and never what you want in a regression.

VHDL

```
-- end by starvation:
-- stop the clock, then
-- suspend forever
done <= true;
wait;

-- there is no
-- "stop the simulator now"
-- statement in VHDL-2008
```

SystemVerilog

```
// end explicitly
$finish;      // exit simulator
$stop;       // break to prompt

// $finish takes a verbosity
// code: 0, 1 (default) or 2
$finish(2);
```

△ Trap: \$finish kills everything, immediately

`$finish` is not `return`: it terminates the whole simulation where it executes. A monitor halfway through reconstructing a transaction is discarded, an unflushed scoreboard queue is never checked, and an open file may lose its last lines. Never call `$finish` from inside a checking thread. Call it from one place, after an explicit drain, and put anything that must happen at the very end in a `final` block, which is the one thing `$finish` does run.

2.2.6 Printing and asserting

the *Design Cookbook* gives the full mapping from VHDL's `report ... severity` to `$display`, `$info`, `$warning`, `$error` and `$fatal`, and the C-style format specifiers that go with them. Two consequences of that mapping shape every testbench in this chapter.

The first is that SystemVerilog splits printing from counting. `$display` only prints; the severity tasks print *and* increment the simulator's error counters, which is what a batch regression reads to decide pass or fail. A mismatch reported with `$display` is invisible to the regression script. Report failures with `$error` or `$fatal`, always, and reserve `$display` for progress and configuration.

The second is `$sformatf`, which formats into a `string` instead of printing. Every `convert2string()` method in this chapter is one call to it, and that is what lets a transaction describe itself in a failure message written by someone else.

The shape of an immediate assertion

An immediate assertion has the shape of an `if`: the statement after the condition is the *pass* action, the one after `else` is the *fail* action, and both are optional. Writing a pass action is almost always a mistake, because it executes on every successful check and floods the log. Always give a label and a message naming the values involved: `a_result_ok : assert (q == 8'd100) else $error("got %0d", q);`. The label appears in the failure message, in the coverage database, and, for concurrent assertions, in the command that switches the assertion off.

At this point you have a working SystemVerilog testbench that does the same job as the VHDL one. It is also, as it stands, quietly broken.

2.3 Why That Testbench Is Dangerous

Look again at the stimulus loop in Listing 2.3. It waits for a clock edge, waits 1 ns, then drives. The 1 ns is there because you have written it in every VHDL testbench you have ever produced. Remove it and see what happens.

```

1 initial begin
2   rst_n = 1'b1;
3   for (int i = 1; i <= 4; i++) begin
4     @(posedge clk);
5     en = 1'b1;           // blocking, on the edge
6     d = 8'(i * 10);      // blocking, on the edge
7   end
8 end

```

Listing 2.4. A racy stimulus loop. Do not write this

Two processes are now sensitive to the same event. The DUT's `always_ff @(posedge clk)` wants to read `en` and `d`; the testbench's `initial` block wants to write them. Both are triggered by the same edge, and both run in the **Active** region of the same time slot.

the *Design Cookbook* explained what the LRM says about that: processes scheduled in the Active region execute in an order that is explicitly left undefined. There is no rule. The simulator may run the testbench first, in which case the DUT sees the *new* value of `en` and accumulates on this very edge; or it may run the DUT first, in which case the DUT sees the *old* value and accumulates one cycle later. Both are conforming implementations.

The observable result is a testbench that produces 100 on one simulator and 60 on another, or 100 today and 60 after an unrelated module changes the compile order and with it the tool's internal process ordering. There is no warning. In VHDL this class of bug does not exist, because the language separates the update phase from the evaluate phase and a signal assignment made during evaluation is never visible until the next delta cycle. VHDL bought determinism with the signal/variable distinction; Verilog did not.

2.3.1 Remedy (a): drive on the other edge

The first fix is the one your VHDL instincts reach for: move the driving away from the edge the design samples.

```

1 initial begin
2   for (int i = 1; i <= 4; i++) begin
3     @(negedge clk);           // half a period away from the DUT
4     en = 1'b1;
5     d  = 8'(i * 10);
6   end
7 end

```

This works. It is deterministic, it is simple, and for a small testbench with one clock it is defensible. It has two defects.

The first is that it does not model reality. No real device drives a synchronous bus on the falling edge. Run this testbench against a gate-level netlist with back-annotated timing and the stimulus arrives half a period early, so every setup check passes trivially and the simulation tells you nothing about whether the interface closes timing. Add a second clock domain, or a negative-edge-triggered sub-block, and the convention breaks silently.

The second is that it is a convention, not a declaration. It lives in your head. A colleague adding a new stimulus block six months later has no way of knowing the rule except by reading every existing block and inferring it.

2.3.2 Remedy (b): drive with non-blocking assignments

The second fix uses the scheduler rather than the waveform.

```

1 initial begin
2   for (int i = 1; i <= 4; i++) begin
3     @(posedge clk);
4     en <= 1'b1;           // non-blocking
5     d  <= 8'(i * 10);
6   end
7 end

```

Now the assignment is queued in the Active region and applied in the **NBA** region. The DUT's `always_ff`, whichever order it runs in, evaluates the right-hand side of its own non-blocking

assignments in Active, where `en` and `d` still hold their old values. The design therefore always sees the old value, on every simulator, deterministically.

This is a genuine fix and it costs nothing. It is the same reason RTL code uses `<=` inside `always_ff`: non-blocking assignment makes the order in which parallel processes execute irrelevant. Use it everywhere in a testbench that drives DUT inputs from a clocked context.

△ **Trap: You cannot read back what you just non-blocking-assigned**

```
1 en <= 1'b1;
2 $display("en is %b", en); // prints the OLD value
```

The assignment has been *queued*, not performed: `$display` runs in Active, the update happens later in NBA. You already have this reflex for VHDL signals, but the trap is sharper here, because the same variable could equally well have been assigned with `=`, in which case the print shows the new value. Use `$strobe`, which prints in the Postponed region after everything has settled, or read the value after the next clock edge.

△ **Trap: Mixing `=` and `<=` on the same testbench signal**

A signal driven with `=` in one block and `<=` in another is a race even if neither block is clocked: the blocking write lands in Active, the non-blocking write lands in NBA of the same time slot, and NBA always wins regardless of source order. The symptom is a signal that ignores the blocking assignment entirely and takes the non-blocking value, even when the blocking line is the later of the two in the source and ran last. Pick one operator per signal: clock with `=`, everything reaching the DUT with `<=` or a clocking block, local bookkeeping with `=`.

2.3.3 Remedy (c): a clocking block

Non-blocking assignment removes the race but still leaves timing to convention: the stimulus changes exactly at the edge, which is a zero propagation delay that no hardware has. A *clocking block* declares the timing instead, and it is the right answer.

the *Design Cookbook* introduced the syntax. Here is the real one, from `cordic_if.sv`, retyped:

```
1 interface cordic_if (input logic clk);
2
3     import cordic_pkg::*;
4
5     logic    rst_n, req_valid, req_ready, rsp_valid, rsp_ready;
6     data_t   req_x, req_y, rsp_x, rsp_y;
7     angle_t  req_theta;
8
9     clocking drv_cb @(posedge clk);
10         default input #1step output #1ns;
11         output req_valid, req_x, req_y, req_theta;
12         output rsp_ready;
13         input  req_ready, rsp_valid, rsp_x, rsp_y;
14     endclocking
15
16     clocking mon_cb @(posedge clk);
17         default input #1step;
18         input  req_valid, req_ready, req_x, req_y, req_theta;
19         input  rsp_valid, rsp_ready, rsp_x, rsp_y;
20     endclocking
```

```

21
22 modport drv (clocking drv_cb, output rst_n, input clk);
23 modport mon (clocking mon_cb, input rst_n, input clk);
24
25 endinterface : cordic_if

```

Listing 2.5. The clocking blocks from `cordic_if.sv`

Take the crucial line apart word by word:

```

1 default input #1step output #1ns;

```

#1step is a unit of time equal to the smallest precision in the whole simulation, and it exists for exactly one purpose. A sample taken **#1step** before the clock edge is a sample taken in the **Preponed** region, before any process in that time slot has executed. It is therefore the value the signal had *immediately before* the edge: the value a real flip-flop would capture. No design process can have changed it yet, so there is no race, by construction and not by convention.

output #1ns means that an assignment to a clocking-block output does not take effect at the edge. It is scheduled 1 ns later, in the middle of the cycle, which is what a real driver with a propagation delay does. This removes the second half of the problem: the stimulus no longer changes at the instant a flip-flop is sampling.

Figure 2.2 shows the resulting timing.

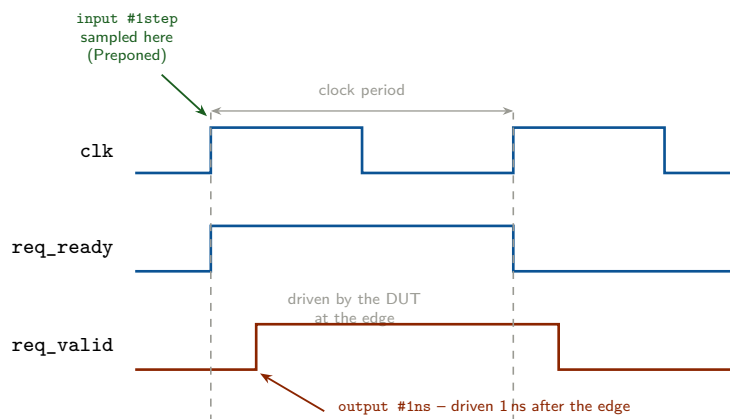


Figure 2.2. What `default input #1step output #1ns` buys you. The testbench samples strictly before the edge, so it never sees a value the DUT produced on that same edge; it drives strictly after the edge, so the DUT never samples a value in the instant it is changing.

Using it is unremarkable, and the *Design Cookbook* has the direction convention: a clocking block is written from the testbench's point of view, so `req_ready`, an output of `cordic_rot`, is an input here.

```

1 vif.drv_cb.req_valid <= 1'b1;          // driven 1 ns after the next edge
2 vif.drv_cb.req_x    <= t.x0;
3 do @(vif.drv_cb); while (vif.drv_cb.req_ready != 1'b1);
4 vif.drv_cb.req_valid <= 1'b0;        // ready was sampled before the edge

```

Those four lines are the whole valid/ready handshake, and Section 2.6 uses them unchanged as the body of the driver's `send()` task.

△ Trap: Two drivers on one signal: the clocking block always wins

A clocking-block output is driven in the Re-NBA region, after Active and after NBA. If another part of the testbench also assigns the same signal procedurally, the clocking block overwrites it in every time slot in which both happen, regardless of source order, and the symptom is a signal that ignores half your assignments. A signal listed as a clocking-block output must be driven *only* through that clocking block. This is why `cordic_if` keeps `rst_n` out of `drv_cb` and exposes it directly on the `drv` modport.

△ Trap: `vif.drv_cb.req_ready` is not `vif.req_ready`

The first is a sample taken before the most recent clock edge; the second is the live signal, right now. They differ by up to a clock cycle, and a testbench written half one way and half the other appears to see the same event at two different times. The symptom is a monitor that reports transactions one cycle later than the driver thinks it sent them, or a `wait` that hangs because it is watching a live signal that changed in a region the waiting thread cannot observe. Use the clocking-block reference for everything synchronous, and raw references only for asynchronous signals such as reset.

#1step is not #1

`1step` is a keyword, not a number with a unit. It denotes the smallest time precision used anywhere in the simulation and may appear only as a clocking-block input skew, where the LRM defines it to mean the Preponed region of that time slot. `#1` would mean one time *unit*, typically 1 ns, which is a thousand times too coarse.

Drive the DUT only through a clocking block. Sample the DUT only through a clocking block. Then the question "does my testbench race with the design?" has the answer "no", permanently, and you stop having to think about it.

2.4 Making the Testbench Decide

The testbench in Listing 2.3 contains one assertion, checking one value, at one time. That is not a testbench, it is a waveform generator with a comment. Going from "run it and look at the waves" to "the testbench decides" is the single largest quality improvement available to you, and it costs less effort than the waveform inspection it replaces.

A self-checking testbench needs four things: a model of what the correct answer is, a comparison with a defined tolerance, a record of how many checks passed and failed, and a guarantee that it terminates with a verdict even when the DUT misbehaves.

2.4.1 The reference model must describe the specification

The reference model, also called the golden model or the predictor, computes what the output *should* be. There is exactly one rule about how to write it, and it is not negotiable:

Write the reference model from the specification, never from the RTL. A model derived from the implementation reproduces the implementation's bugs and proves nothing.

This rule is easy to state and constantly violated, because copying the RTL is so much quicker. Consider the CORDIC. A tempting reference model would iterate the same micro-rotations the hardware performs:

```

1 // DO NOT DO THIS
2 function automatic void bad_predict(input cordic_txn t,
3                                   output data_t x_ref, output data_t y_ref);
4     idata_t x = idata_t'(t.x0)    <<< GL;
5     idata_t y = idata_t'(t.y0)    <<< GL;
6     iangle_t z = iangle_t'(t.theta) <<< GL;
7     for (int i = 0; i < int'(N); i++)
8         if (z >= 0) begin
9             x = x - (y >>> i); y = y + (x >>> i); z = z - ATAN_ROM[i];
10        end else begin
11            x = x + (y >>> i); y = y - (x >>> i); z = z + ATAN_ROM[i];
12        end
13    x_ref = sat_narrow(x); y_ref = sat_narrow(y);
14 endfunction

```

Listing 2.6. A worthless reference model: it is the RTL again

If the RTL uses the wrong `ATAN_ROM` entry, so does this model, and they agree. If the RTL folds the quadrant incorrectly, so does this model, and they agree. If `ATAN_ROM` itself is wrong, both are wrong together. This model detects only typographical differences between two pieces of code you wrote on the same afternoon in the same frame of mind, and it will pass a design that computes something which is not a rotation at all.

Compare the real one from `tb_cordic_pkg.sv`:

```

1 function automatic void predict(input cordic_txn t,
2                                output real x_ref, output real y_ref);
3     real th = q_to_real(t.theta, QA);
4     real xr = q_to_real(t.x0,    QF);
5     real yr = q_to_real(t.y0,    QF);
6     x_ref = K_GAIN * (xr * $cos(th) - yr * $sin(th));
7     y_ref = K_GAIN * (xr * $sin(th) + yr * $cos(th));
8 endfunction

```

Listing 2.7. The reference model from `tb_cordic_pkg.sv`: pure specification

Six lines, and they say nothing about micro-rotations, guard bits, quadrant folding, saturation or the FSM. They say what the block computes: a rotation by θ , scaled by the CORDIC processing gain K . Reimplement `cordic_rot` as a lookup table, as a pipelined unrolled array, or as a multiply by a rotation matrix, and this model still checks it. That is the test for whether a reference model is any good: could you swap the DUT for a completely different implementation of the same specification and keep the model unchanged?

Three legitimate places for a reference model

The model does not have to be written in SystemVerilog. In order of increasing independence: a function in the testbench package, as here; a C or C++ routine called through DPI-C (the *Simulation Cookbook*), which is what you do when the model already exists as part of the firmware; and a Python or MATLAB script that produced a vector file offline, which is the subject of the *Simulation Cookbook*. The third is the most independent,

because the person who wrote the model was solving a mathematical problem and had never seen your RTL.

2.4.2 Tolerance: why zero is the wrong answer

For a control block, a checker compares for exact equality and that is correct. For a fixed-point arithmetic block, exact equality is almost always the wrong check, and demanding it produces one of two bad outcomes: either you weaken the reference model until it matches the RTL bit for bit, which lands you back in Listing 2.6, or you spend a week chasing "failures" that are one least significant bit of rounding. `tb_cordic_pkg.sv` declares:

```
1 // Comparison tolerance, in LSBs of the output format. The theoretical
2 // bound for N = 14 micro-rotations plus the guard-bit truncation is a
3 // little under 5 LSB; 12 leaves margin without hiding a real bug.
4 localparam int TOL_LSB = 12;
```

The number 12 is not arbitrary and it is not a knob you turn until the test passes. It comes from an error budget, and you must be able to write the budget down:

- **Angle quantisation.** The rotation stops after $N = 14$ micro-rotations, so the residual angle is bounded by $\arctan(2^{-13}) \approx 1.2 \times 10^{-4}$ rad; times the maximum vector magnitude, that is on the order of one external LSB.
- **Datapath truncation.** Each micro-rotation shifts right and truncates. With $GL = 2$ guard bits the accumulated truncation over 14 iterations is bounded by roughly $N \cdot 2^{-GL}$ internal LSBs, a few external LSBs.
- **Output narrowing.** `sat_narrow` drops the guard bits with an arithmetic shift, which rounds towards $-\infty$: up to one more LSB.
- **The reference model's own error.** `$cos` and `$sin` are double precision, so this term is negligible here, but in a fixed-point model it would not be.

The total is a little under five LSB. `TOL_LSB` is set to 12, about two and a half times the bound. That margin absorbs the corner cases the hand analysis did not cover, without being so loose that a real functional bug slips through: a bug that inverts a sign, uses the wrong ROM entry or folds the wrong quadrant produces an error of *thousands* of LSB, not tens. There is a wide gap between the noise floor and the smallest real bug, and the tolerance is placed inside that gap.

△ **Trap: Tolerance is not a place to hide a bug**

A tolerance is a claim about the numerical accuracy of the design and it belongs in the specification. If you have to raise `TOL_LSB` to make a test pass, you have either found a bug or discovered that the accuracy claim was wrong, and someone must decide which, in writing. Raising the number until the regression turns green converts a failing test into a passing test that checks nothing. The scoreboard guards against this by tracking `worst_err` and printing it: if the worst observed error creeps from 4 to 11 LSB between two releases the test still passes, but the report says something changed.

Here is the comparison itself, from the scoreboard:

```
1 real x_ref, y_ref, ex, ey;
2 predict(t, x_ref, y_ref);
3
4 ex = (q_to_real(t.x, QF) - x_ref) * (2.0 ** QF); // error in LSB
```

```

5 ey = (q_to_real(t.y, QF) - y_ref) * (2.0 ** QF);
6 if (ex < 0.0) ex = -ex;
7 if (ey < 0.0) ey = -ey;
8 if (ex > worst_err) worst_err = ex;
9 if (ey > worst_err) worst_err = ey;
10
11 n_checked++;
12 if (ex > real'(TOL_LSB) || ey > real'(TOL_LSB)) begin
13     n_failed++;
14     $error({"MISMATCH %s\n          got  x=%0d y=%0d\n",
15           "          want x=%.1f y=%.1f  (err %.2f / %.2f LSB)",
16           t.convert2string(), t.x, t.y,
17           x_ref * (2.0 ** QF), y_ref * (2.0 ** QF), ex, ey});
18 end

```

Listing 2.8. Tolerance-based comparison, in LSBs of the output format

Note the braces around the two halves of the format string. SystemVerilog has no C-style adjacent string literal concatenation: "a" "b" is a syntax error. A long format string is split with the concatenation operator, {"a", "b"}, which is the same operator used on bit vectors.

Note also that the error is converted into LSBs of the output format before being compared. That is what makes the tolerance a number a specification can state. Comparing raw real values against $1e-4$ works too, but the number means nothing to a reader and has to be rederived whenever QF changes.

Recipe: Always print both values on a mismatch

A failure message that says `assertion failed` costs an hour. A failure message that prints the stimulus, the observed value, the expected value and the error in the natural unit costs a minute. `convert2string()` exists for precisely this, and it is the cheapest thing in the whole testbench.

2.4.3 Counting, and the final block

An error counter is a single integer, incremented on every mismatch, whose value at the end of simulation determines the verdict. Report it from a `final` block, a SystemVerilog construct with no VHDL counterpart: it executes once, at the end, whether the simulation ended by `$finish` or by running out of events.

```

1 int unsigned n_checked = 0, n_failed = 0;
2 localparam int unsigned MIN_TXN = 50;
3
4 final begin
5     $display(" transactions checked : %0d", n_checked);
6     $display(" mismatches           : %0d", n_failed);
7     if      (n_failed != 0)      $display(" *** TEST FAILED ***");
8     else if (n_checked < MIN_TXN) $display(
9         " *** TEST FAILED: only %0d of %0d transactions ***", n_checked, MIN_TXN);
10    else                          $display(" *** TEST PASSED ***");
11 end

```

Listing 2.9. A verdict that cannot be missed

△ Trap: A final block may not consume time

`final` runs in the last time slot, after the simulator has decided to stop, and it is a function-like context: no `#`, no `@`, no `wait`, no time-consuming task calls. Drain queues and wait for outstanding transactions *before* `$finish`, and leave the `final` block to print.

2.4.4 \$error or \$fatal?

`$error` prints, increments the error count, and *continues*. `$fatal` prints and terminates the simulation immediately, with an exit code taken from its first argument (`$fatal(1, "...")` is the conventional form; the argument is a `$finish`-style verbosity code, and a non-zero value is what your regression script tests).

The choice is not about severity, it is about whether continuing produces useful information. Use `$error` for a *data* mismatch: one wrong result does not prevent the next thousand transactions from being checked, and the pattern of which ones fail is the most valuable debug information you will get. A run that stops on the first mismatch tells you one thing; a run reporting "917 of 1000 failed, all with $\theta < 0$ " tells you where the bug is. Use `$fatal` when the testbench itself can no longer function: randomisation failed, a configuration file is missing, the DUT has hung. Continuing would then produce thousands of secondary failures that bury the real one. `tb_cordic_pkg.sv` follows exactly this split.

Better than choosing once and for all is a switch: read a `+MAX_ERRORS=n` plusarg (Section 2.9), default it to zero meaning "never stop", and call `$fatal` when `n_failed` reaches it. Run the regression with no limit and debug interactively with `+MAX_ERRORS=1`.

2.4.5 The watchdog

A DUT that hangs is not a passing test, but that is what a naive testbench reports: the driver blocks forever waiting for a `req_ready` that never arrives, the simulation runs out of events (or runs until the grid job is killed), and no verdict is printed. Every testbench needs an independent timeout.

```

1 localparam time TIMEOUT = 500us;
2
3 initial begin
4     #TIMEOUT;
5     $fatal(1, "WATCHDOG: no completion after %0t", TIMEOUT);
6 end

```

Listing 2.10. An absolute watchdog

That is the crude version, and it is better than nothing. The better version is an *activity* watchdog: it fires not after a fixed simulated time but after a number of clock cycles with no progress, so it scales automatically with the size of the test.

```

1 localparam int unsigned MAX_IDLE = 5000;
2 int unsigned idle = 0, last_checked = 0;
3
4 initial forever begin
5     @(posedge clk);
6     if (sb.n_checked != last_checked) begin
7         last_checked = sb.n_checked;
8         idle          = 0;
9     end else begin
10        idle++;

```

// not '++idle' inside the condition

```

11     if (idle > MAX_IDLE)
12         $fatal(1, "WATCHDOG: %0d cycles with no completed transaction",
13             MAX_IDLE);
14     end
15 end

```

Listing 2.11. An activity watchdog that resets on progress

The increment is deliberately on a line of its own. `else if (++idle > MAX_IDLE)` is legal SystemVerilog and Icarus runs it, but Verilator 5.020 aborts with an internal error on a pre-increment used inside a condition. Side effects buried in expressions are hard to read anyway, and in a testbench that must run on more than one tool they are not worth the two saved characters.

△ Trap: A watchdog is not optional and it is not a debug aid

The failure mode a watchdog catches is the one that looks most like success. A hung simulation in a nightly regression produces no error message, no failing assertion and no exit code until the batch system kills it; the log then ends mid-sentence and the summary script records "no result" rather than "failed". Teams do discover months later that a block was never verified because its testbench had been timing out silently since the third week. Add the watchdog on day one, before the first check.

2.4.6 The test that passed because it checked nothing

This is the most dangerous failure mode in verification. Suppose the monitor's clocking block is mis-declared, or the mailbox between the monitor and the scoreboard is never connected, or a constraint is unsatisfiable so every transaction is one the DUT never accepts. In each case `n_failed` is zero at the end of the run, the verdict is TEST PASSED, and nothing was verified. `n_failed == 0` is necessary for a pass; it is not sufficient. The sufficient condition is:

```

1 if (n_failed == 0 && n_checked >= MIN_TXN) ... PASSED

```

`tb_cordic_pkg.sv` implements the weak version of this in `cordic_sb::report()`:

```

1 if (n_failed == 0 && n_checked > 0) $display(" *** TEST PASSED ***");
2 else $display(" *** TEST FAILED ***");

```

and the strong version in `cordic_env::wait_drain()`, which knows how many transactions were sent and fails if fewer came back:

```

1 task automatic wait_drain(int unsigned expected, int unsigned timeout_cyc);
2     int unsigned waited = 0;
3     while (sb.n_checked < expected && waited < timeout_cyc) begin
4         @(vif.mon_cb);
5         waited++;
6     end
7     if (sb.n_checked < expected)
8         $fatal(1, "timeout: only %0d of %0d transactions completed",
9             sb.n_checked, expected);
10 endtask

```

Listing 2.12. Drain with a bound: the guard against a silent pass

Every testbench must be able to fail. Before you trust a new checker, break the DUT on purpose – invert a sign, delete a guard bit – and confirm that the test goes red. A checker that has never failed has never been tested.

Recipe: The five-minute sanity ritual

When a testbench first reports PASSED, do all five of these before believing it:

1. Comment out one term in the DUT and check that it fails.
2. Set TOL_LSB to 0 and check that it fails.
3. Check that `n_checked` is the number you expected, not zero and not one.
4. Run with a different `-sv_seed` and check that the stimulus actually changed.
5. Look at the waveform for exactly one transaction, and confirm the handshake looks like the protocol document.

2.5 Vectors From a File

Before the layered testbench, one intermediate step that is often enough on its own: keep the stimulus and the expected results in a file, and let the testbench read them. It is how you use a model written in another language without linking anything, and it is completely reproducible — the file is the test.

The *Simulation Cookbook* covers the whole technique, including how to generate the file from a Python model and how to read it from SystemVerilog, from VHDL and from C++. Two things belong here, because they are checking bugs rather than file format bugs.

```

1  int fd, n_read;
2  logic [15:0] a, b, expect_q;
3
4  fd = $fopen("vectors.txt", "r");
5  if (fd == 0) $fatal(1, "cannot open vectors.txt");
6
7  n_read = 0;
8  while (!$feof(fd)) begin
9      // $fscanf returns the number of items it matched. Ignoring it is how a
10     // truncated file turns into a test that quietly checks nothing.
11     if ($fscanf(fd, "%h %h %h\n", a, b, expect_q) == 3) begin
12         drive_and_check(a, b, expect_q);
13         n_read++;
14     end
15 end
16 $fclose(fd);
17
18 // The vector count is part of the test.
19 if (n_read != EXPECTED_VECTORS)
20     $fatal(1, "read %0d vectors, expected %0d", n_read, EXPECTED_VECTORS);

```

Listing 2.13. Reading a vector file the safe way

△ Trap: A short vector file is a silent pass

`$readmemh` on a file with fewer lines than the array has entries fills what it finds and leaves the rest at their initial value, with no error. `$fscanf` on a malformed line returns a count

you did not check. Both produce a run that reports zero failures because it performed zero comparisons. Count what you read and assert the count — this is the same failure as the one in Section 2.4.6, arriving by a different route.

2.6 The Layered Testbench

Everything so far has been one `initial` block doing everything. That works until one of four things happens: you want to reuse the pin-level protocol code in a second test; you want stimulus and checking to run concurrently rather than in lockstep; you want random stimulus, so the checker can no longer know the answer in advance; or the DUT pipelines, so that several transactions are in flight at once and "drive, wait, check" stops being a possible shape.

The answer to all four is the same: split the testbench into components with one responsibility each, and let them communicate by passing *transaction objects* rather than by sharing wires. Figure 2.3 shows the result. Study it before the code, because every class that follows is one box in that picture.

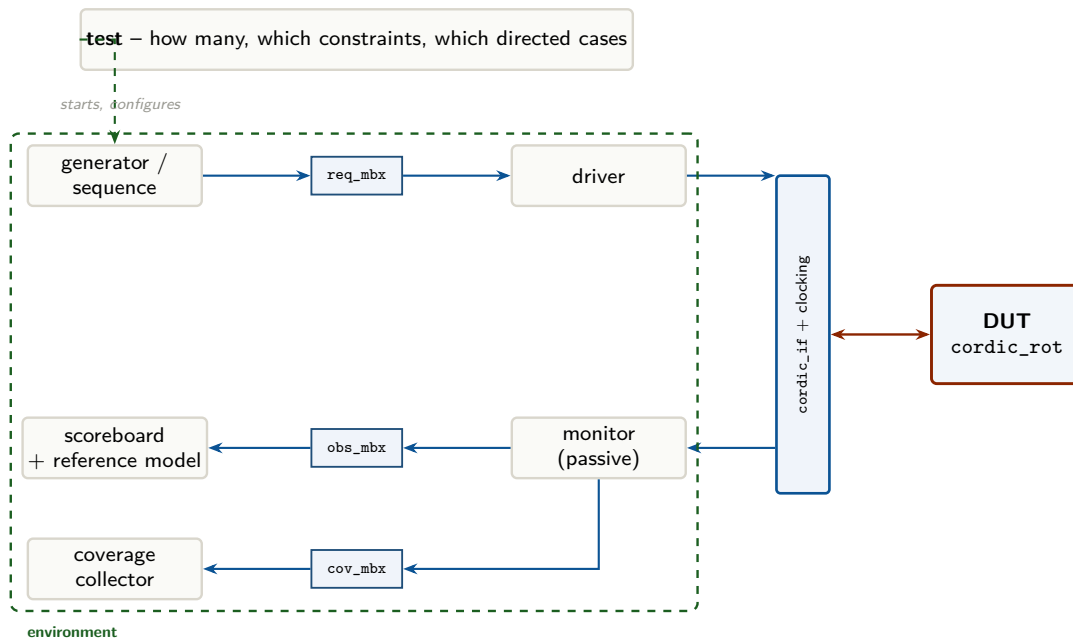


Figure 2.3. The layered testbench. Solid arrows carry transaction objects through mailboxes; the double arrow is the only pin-level connection in the whole picture. Chapter 3 replaces each box with a UVM base class and each mailbox with an analysis port, and changes nothing else.

Notice what the picture says. Only two components touch a wire: the driver and the monitor. Everything else works with objects, so everything else is independent of the pin-level protocol and survives a change to it unchanged. That is the entire point of the layering.

2.6.1 The transaction

A transaction is a class holding what goes into the DUT and what came out of it, for one unit of work. It is a *value*, not a component: created, filled in, handed on, and eventually garbage collected.

Why a class and not a set of signals? It travels through a mailbox as a single item, so producer and consumer never have to agree on timing. It carries its own randomisation constraints, so the rules about what constitutes a legal request live with the definition of a request. And it can print itself, which turns a failure message from a puzzle into a diagnosis.

```

1  class cordic_txn;
2
3  // ---- stimulus -----
4  rand data_t x0;
5  rand data_t y0;
6  rand angle_t theta;
7
8  // ---- observed result, filled in by the monitor -----
9  data_t x;
10 data_t y;
11
12 // ---- bookkeeping -----
13 int unsigned id;
14 static int unsigned next_id = 0;
15
16 constraint c_theta {
17     theta inside {[-PI_EXT : PI_EXT]};
18 }
19 constraint c_magnitude {
20     x0 inside {[-6000 : 6000]};
21     y0 inside {[-6000 : 6000]};
22 }
23 constraint c_corners {
24     theta dist {
25         0                :/ 1,
26         PI_EXT           :/ 1,
27         -PI_EXT          :/ 1,
28         PI_EXT / 2       :/ 1,
29         -PI_EXT / 2      :/ 1,
30         [-PI_EXT:PI_EXT] :/ 20
31     };
32 }
33
34 function new();
35     id = next_id++;
36 endfunction
37
38 function string convert2string();
39     return $sformatf(
40         "txn#%0d x0=%0d (%.4f) y0=%0d (%.4f) theta=%0d (%.4f rad)",
41         id, x0, q_to_real(x0, QF), y0, q_to_real(y0, QF),
42         theta, q_to_real(theta, QA));
43 endfunction
44
45 endclass : cordic_txn

```

Listing 2.14. The transaction from `tb_cordic_pkg.sv`

The `id` field and the static `next_id` counter matter more than they look. `next_id` is a **static** member (Section 1.2), so `next_id++` in the constructor gives every transaction a unique serial number. When transaction 4172 fails in a run of ten thousand, that number is how you find it in the waveform, in the log and in the coverage database.

The three constraints deserve a note each. `c_theta` states a legality rule taken straight from the specification: angles outside $\pm\pi$ are not accepted, and `cordic_rot` has an immediate assertion that fires if you send one. `c_magnitude` is a *test* decision rather than a legality rule: it keeps

$|(x, y)| \cdot K$ inside the output range so that saturation is exercised only by directed tests, where the expected behaviour is unambiguous. `c_corners` uses `dist` to bias the distribution, because a uniform draw over $[-\pi, \pi]$ hits exactly zero, exactly π and exactly $\pi/2$ with probability approximately zero, and those are the three angles where the quadrant-folding logic changes behaviour.

△ Trap: Reusing one transaction object for a whole run

This is what the handle-is-a-reference rule of Section 1.2 looks like at the place it actually bites you. `mbx.put(t)` puts the *handle* into the mailbox, not a copy of the object. If the generator then does `t.theta = ...; mbx.put(t);` again, both entries point at the same object and the consumer sees the second value twice. The fix is one line, `cordic_txn t = new();` *inside* the loop rather than above it; forgetting it produces a testbench that appears to send N transactions and actually sends the same one N times, at full speed, reporting a clean pass. The same rule applies downstream: a scoreboard that stores the handle it was given, rather than a copy, holds a value that the monitor can still change.

convert2string is not a UVM invention

The name comes from UVM, where `uvm_object` declares `convert2string()` as a virtual method you override. There is nothing magical about it: it is an ordinary function returning a `string`. Write it now, and when you move to UVM in Chapter 3 the method is already there and the base class simply starts calling it for you.

2.6.2 The generator

The generator produces transactions and puts them into a mailbox. That is all it does. It does not know how a transaction reaches the pins, how long one takes, or whether the DUT is ready.

```

1 task automatic send_random(int unsigned n);
2   repeat (n) begin
3     cordic_txn t = new();
4     if (!t.randomize()) $fatal(1, "randomization failed");
5     req_mbx.put(t);
6   end
7 endtask
8
9 task automatic send_directed(data_t x0, data_t y0, angle_t theta);
10  cordic_txn t = new();
11  t.x0 = x0; t.y0 = y0; t.theta = theta;
12  req_mbx.put(t);
13 endtask

```

Listing 2.15. Random and directed stimulus generation

△ Trap: An unchecked `randomize()` turns a random test directed

Section 1.4 explains why the return value must be tested. Here is what ignoring it costs. An over-constrained transaction keeps whatever values it had before the failed solve, which for a freshly constructed object is all zeros. The generator then pushes a thousand identical zero transactions, the driver drives them, the scoreboard checks them, and the run reports a thousand passing checks against one vector. Every number in the report looks healthy. `void'(t.randomize());` is worse than the bare call, because it documents

that you meant to ignore the failure. When a solve does fail, use the tool's constraint debugger rather than deleting constraints at random.

Randomisation changes the shape of the testbench because it decouples *what* you test from *how many* tests you run. A directed VHDL testbench contains as many checks as you had patience for. A constrained-random testbench contains one check and a number that you raise from 100 to 100 000 when the regression machine is idle overnight. Section 2.10 explains how you find out whether the extra runs buy anything.

The constraint language itself is Section 1.4's subject. What is new here is where the constraints live: on the transaction class, not in the generator. That is deliberate. A rule about what constitutes a legal request belongs with the definition of a request, so that every test inherits it and no test can forget it, while a rule about what *this* test wants goes in an inline with {...} clause at the call site. Put a test's preference into a class constraint and you have made it everybody's preference.

2.6.3 The driver

The driver takes transactions out of the mailbox and turns them into pin wiggles. It knows the protocol: the handshake, the setup order, the reset sequence, and nothing else. It does not know what the values mean, does not check anything, and never looks at the response.

One declaration in Listing 2.16 needs a word before you read the rest of it. `virtual cordic_if vif` is a *handle* to an interface instance, not an instance: a class object lives on the heap and has no position in the module hierarchy, so it cannot name `tb_cordic_top.intf` the way one module names another. The handle is `null` until the top-level module hands the real instance to the constructor, which is the whole of the argument `new(intf)` in Listing 2.23. Until Section 2.7 takes the mechanism apart, read `vif.drv_cb.req_valid` as “the `req_valid` signal of whichever interface instance this driver was given, driven through the `drv_cb` clocking block”.

```

1  class cordic_drv;
2
3  virtual cordic_if vif;
4  mailbox #(cordic_txn) req_mbx;      // from the generator
5  int unsigned stall_pct = 25;        // random back-pressure
6
7  function new(virtual cordic_if vif, mailbox #(cordic_txn) req_mbx);
8      this.vif      = vif;
9      this.req_mbx = req_mbx;
10 endfunction
11
12 task automatic reset();
13     vif.rst_n      <= 1'b0;
14     vif.drv_cb.req_valid <= 1'b0;
15     vif.drv_cb.req_x   <= '0;
16     vif.drv_cb.req_y   <= '0;
17     vif.drv_cb.req_theta <= '0;
18     vif.drv_cb.rsp_ready <= 1'b0;
19     repeat (5) @(vif.drv_cb);
20     vif.rst_n <= 1'b1;
21     @(vif.drv_cb);
22 endtask
23
24 task automatic send(cordic_txn t);
25     vif.drv_cb.req_valid <= 1'b1;
26     vif.drv_cb.req_x     <= t.x0;

```

```

27     vif.drv_cb.req_y      <= t.y0;
28     vif.drv_cb.req_theta <= t.theta;
29     do @(vif.drv_cb); while (vif.drv_cb.req_ready !== 1'b1);
30     vif.drv_cb.req_valid <= 1'b0;
31     endtask
32
33 endclass : cordic_drv

```

Listing 2.16. The driver from `tb_cordic_pkg.sv`

Read `send()` with the protocol in mind. The payload and `req_valid` are driven together through the clocking block, so they appear 1 ns after a clock edge. The loop then advances one clock at a time and inspects the *sampled* `req_ready`. Because the sample is taken in Preponed, the value tested at edge n is the value `req_ready` held during cycle $n - 1$, so the loop exits on the first edge that follows a cycle in which both valid and ready were high. That is the valid/ready contract exactly. Note also what `send()` does *not* do: it never changes the payload while `req_valid` is high, which is required by the concurrent assertion `a_req_stable` inside `cordic_rot`.

The second thread is back-pressure. A response channel that is always ready is never stalled, so the DUT's `ST_DONE` state lasts exactly one cycle in every transaction and any bug in holding the result is invisible. The driver therefore runs a second, independent process that randomly de-asserts `rsp_ready`:

```

1 task automatic run();
2   fork
3     forever begin
4       cordic_txn t;
5       req_mbx.get(t);
6       send(t);
7     end
8     forever begin
9       if ($urandom_range(99) < stall_pct) begin
10        vif.drv_cb.rsp_ready <= 1'b0;
11        repeat ($urandom_range(3) + 1) @(vif.drv_cb);
12      end
13      vif.drv_cb.rsp_ready <= 1'b1;
14      @(vif.drv_cb);
15    end
16  join_none
17 endtask

```

Listing 2.17. Two independent driver threads

`fork ... join_none` (Section 1.3) starts both branches and returns immediately, so `run()` does not block its caller. That is the property being used here, and it is the reason every component in this testbench exposes a `run()` that starts threads instead of a task that does work: the environment can start five components in five statements and still have control afterwards. The two branches are genuinely concurrent and independent, neither knowing the other exists. There is no way to express this in VHDL, where the number of processes is fixed at elaboration and a subprogram cannot create one.

Random back-pressure on every ready signal, from the first test. A protocol bug that only appears under stall is the most common bug in a valid/ready design and the least likely to be found by hand-written stimulus.

Recipe: Make the stall percentage a knob

`stall_pct` is a plain public variable, so the test can set `env.drv.stall_pct = 0` for a throughput measurement, `= 90` for a stress run, and leave it at the default for regressions. Anything a test might want to vary should be a variable on the component, not a literal buried in a task.

2.6.4 The monitor

The monitor watches the pins and reconstructs transactions. It is *strictly passive*: it never drives anything, and its clocking block `mon_cb` lists every signal as an input precisely so that it cannot.

The temptation, always, is to have the driver tell the monitor what it sent. That saves code and destroys the testbench: the checking is then against what the testbench *intended* to send, not against what appeared on the wires, so a driver bug becomes invisible, because the same wrong information reaches the checker from both sides. Valid asserted for two cycles, a payload changed mid-handshake, a transaction dropped when `req_ready` deasserts: none of those is detectable by a checker that was told the answer. There are three further, practical reasons: a passive monitor can be attached to a gate-level netlist where no driver exists, to the same block instantiated inside a larger system and driven by real neighbouring logic, and to an emulation or formal flow, all unchanged.

```

1  class cordic_mon;
2
3  virtual cordic_if vif;
4  mailbox #(cordic_txn) obs_mbx;      // to the scoreboard
5  mailbox #(cordic_txn) cov_mbx;      // to the coverage collector
6
7  // Requests accepted but whose result has not come back.
8  protected cordic_txn pending[$];
9
10 task automatic run();
11     forever begin
12         @(vif.mon_cb);
13         if (vif.rst_n !== 1'b1) continue;
14
15         // Request accepted -> remember what went in.
16         if (vif.mon_cb.req_valid === 1'b1 &&
17             vif.mon_cb.req_ready === 1'b1) begin
18             cordic_txn t = new();
19             t.x0 = vif.mon_cb.req_x;
20             t.y0 = vif.mon_cb.req_y;
21             t.theta = vif.mon_cb.req_theta;
22             pending.push_back(t);
23         end
24
25         // Response accepted -> complete the oldest outstanding request.
26         if (vif.mon_cb.rsp_valid === 1'b1 &&
27             vif.mon_cb.rsp_ready === 1'b1) begin
28             cordic_txn t;
29             if (pending.size() == 0) begin
30                 $error("monitor: response with no outstanding request");
31             end else begin
32                 t = pending.pop_front();
33                 t.x = vif.mon_cb.rsp_x;
34                 t.y = vif.mon_cb.rsp_y;
35                 obs_mbx.put(t);

```

```

36         cov_mbx.put(t);
37     end
38 end
39 end
40 endtask
41
42 endclass : cordic_mon

```

Listing 2.18. The monitor from `tb_cordic_pkg.sv`

The pending queue is the part worth studying. `cordic_rot` takes $N + 2$ cycles per transaction, and with back-pressure the response can be held arbitrarily long, so the request going in and the response coming out are separated in time. The monitor therefore cannot form a complete transaction in one place: it records the request when the request handshake completes, holds it in a queue, and completes it when the response handshake completes. For a deeply pipelined DUT with several transactions in flight this is not an optimisation, it is the only way the monitor can work at all.

`cordic_txn pending[$]` is a queue (Section 1.1), and the choice of container is a statement about the protocol. A queue with `push_back/pop_front` models *in-order* completion, which is what `cordic_rot` guarantees. A design that can reorder its responses needs an associative array indexed by the transaction tag, `cordic_txn pending[int];`, and nothing else in the monitor changes. If you pick the wrong one, the monitor will happily pair every response with the wrong request and the scoreboard will report a design that computes nonsense.

△ **Trap: === and !==, not == and !=**

With `==`, a comparison involving `x` or `z` returns `1'bx`, and `if (1'bx)` takes the false branch. Where that bites is the negative test: `if (sig != 1'b1)` is *false* when `sig` is `x`, so an unknown signal passes a check meant to catch it. `===` and `!==` compare `x` as itself and always give a clean 0 or 1. In VHDL the trap does not exist, because `'X' = '1'` is ordinary equality on an enumeration and is simply *false*. Use `===` and `!==` everywhere in a monitor.

△ **Trap: A monitor that skips reset must skip it correctly**

`if (vif.rst_n !== 1'b1) continue;` discards everything seen while reset is asserted. That is right, but it also discards the cycle in which reset releases, and it does *not* clear `pending`. If the DUT can be reset mid-transaction, the queue still holds the orphaned request afterwards and it is matched against the first response of the next test, producing a mismatch that looks like an arithmetic bug and is a testbench bug. Clear the queue on the reset edge.

2.6.5 The scoreboard

The scoreboard receives observed transactions, computes what they should have been, compares, and keeps the statistics. Its reference model and its tolerance were the subject of Section 2.4; what remains is the shape of the component.

```

1 class cordic_sb;
2
3     mailbox #(cordic_txn) obs_mbx;
4     int unsigned n_checked = 0;
5     int unsigned n_failed  = 0;
6     real        worst_err = 0.0;
7

```

```

8  function new(mailbox #(cordic_txn) obs_mbx);
9      this.obs_mbx = obs_mbx;
10 endfunction
11
12 task automatic run();
13     forever begin
14         cordic_txn t;
15         real x_ref, y_ref, ex, ey;
16         obs_mbx.get(t);
17         predict(t, x_ref, y_ref);
18         // ... tolerance comparison, exactly as shown earlier ...
19     end
20 endtask
21
22 function void report();
23     $display(" scoreboard: %0d transactions checked, %0d failed",
24             n_checked, n_failed);
25     $display(" worst error: %.2f LSB (tolerance %0d LSB)",
26             worst_err, TOL_LSB);
27     if (n_failed == 0 && n_checked > 0) $display(" *** TEST PASSED ***");
28     else                                $display(" *** TEST FAILED ***");
29 endfunction
30
31 endclass : cordic_sb

```

Listing 2.19. The scoreboard from `tb_cordic_pkg.sv`

Two structural observations. First, `run()` is an infinite loop that blocks on `obs_mbx.get(t)`. It never terminates and it never polls; the mailbox suspends the thread until something arrives. This is the standard shape for every consumer component in a layered testbench. Second, `report()` is a function, not a task, so it consumes no time and can be called from a `final` block.

For a DUT whose outputs are exact rather than approximate, a scoreboard is usually a queue of expected transactions, a queue of observed ones, and a comparison of the two fronts. Here the expected value is instead computed on demand from the observed stimulus, which works because the model is pure and immediate. A reference model with state – a memory model, a protocol with modes – must be driven by the same transaction stream in the same order, and the scoreboard is then also the place where that ordering is enforced.

2.6.6 The coverage collector

Section 1.5 explains what a covergroup declares and why functional coverage answers a different question from code coverage. The component here answers a third question: where in a testbench does the sampling happen. The scoreboard reports "was the answer right?"; the coverage collector reports "what did I actually try?", and a regression can be green for weeks while exercising a third of the design.

```

1  class cordic_cov;
2
3      mailbox #(cordic_txn) cov_mbx;
4
5      angle_t sampled_theta;
6      data_t  sampled_x0;
7
8      covergroup cg_stimulus;
9          option.per_instance = 1;
10
11      cp_quadrant : coverpoint sampled_theta {
12          bins q3_neg = {[-PI_EXT      : -PI_EXT/2 - 1]};

```

```

13     bins q4_neg = {[ -PI_EXT/2 : -1]};
14     bins zero = {0};
15     bins q1_pos = {[1 : PI_EXT/2 - 1]};
16     bins q2_pos = {[PI_EXT/2 : PI_EXT]};
17 }
18 cp_sign_x : coverpoint sampled_x0 {
19     bins neg = {[ -32768 : -1]};
20     bins zero = {0};
21     bins pos = {[1 : 32767]};
22 }
23 x_quadrant_sign : cross cp_quadrant, cp_sign_x;
24 endgroup
25
26 function new(mailbox #(cordic_txn) cov_mbx);
27     this.cov_mbx = cov_mbx;
28     cg_stimulus = new();
29 endfunction
30
31 task automatic run();
32     forever begin
33         cordic_txn t;
34         cov_mbx.get(t);
35         sampled_theta = t.theta;
36         sampled_x0 = t.x0;
37         cg_stimulus.sample();
38     end
39 endtask
40
41 endclass : cordic_cov

```

Listing 2.20. The coverage collector from `tb_cordic_pkg.sv`

△ Trap: A covergroup inside a class is built in the constructor, or not at all

Section 1.5 notes that a covergroup must be constructed. Inside a class there is a further restriction that catches people: the instance may be created *only* in that class's function `new()`. Forget `cg_stimulus = new();` and every `sample()` call does nothing, the coverage report shows 0%, and you spend an afternoon convinced the stimulus is broken.

Now the distinction the heading promised. `cg_stimulus` covers *stimulus*: it records what the testbench sent, which tells you whether the constraints produce the variety you assumed. It says nothing about what the DUT did internally. A design can receive every quadrant and still never have its saturation logic activate, never stall in `ST_DONE`, and never see two requests back to back.

Covering DUT state means sampling signals inside the design, which needs a second covergroup in a module, usually clocked and usually bound to the DUT (Section 2.8):

```

1 covergroup cg_dut_state @(posedge clk_i);
2     cp_state : coverpoint dut.state_q {
3         bins idle = {dut.ST_IDLE};
4         bins rotate = {dut.ST_ROTATE};
5         bins done = {dut.ST_DONE};
6         bins stalled_in_done = (dut.ST_DONE => dut.ST_DONE);
7     }
8     cp_sat : coverpoint (dut.x_q[IW-1:GL] > MAX_DATA ||
9         dut.x_q[IW-1:GL] < MIN_DATA) {
10         bins saturating = {1'b1};
11         bins normal = {1'b0};
12     }

```

```

13 x_state_sat : cross cp_state, cp_sat;
14 endgroup

```

Listing 2.21. Covering DUT state rather than stimulus

The (ST_DONE => ST_DONE) bin is a *transition bin*: it is hit only when the FSM stays in ST_DONE for two consecutive samples, which happens only when `rsp_ready` is low. If that bin is empty at the end of a regression, the back-pressure thread never actually stalled anything and you have not verified the stall path, no matter how green the scoreboard was.

Stimulus coverage tells you your constraints work. DUT-state coverage tells you your verification works. Write both, and never sign off on stimulus coverage alone.

2.6.7 The environment

The environment owns the components, owns the mailboxes, connects them and starts them. It contains no stimulus and no protocol knowledge, which is what makes it reusable across every test.

```

1 class cordic_env;
2
3   virtual cordic_if vif;
4
5   mailbox #(cordic_txn) req_mbx;
6   mailbox #(cordic_txn) obs_mbx;
7   mailbox #(cordic_txn) cov_mbx;
8
9   cordic_drv drv;
10  cordic_mon mon;
11  cordic_sb sb;
12  cordic_cov cov;
13
14  function new(virtual cordic_if vif);
15      this.vif = vif;
16      // "build"
17      req_mbx = new();
18      obs_mbx = new();
19      cov_mbx = new();
20      // "connect": the mailboxes are passed to the constructors
21      drv      = new(vif, req_mbx);
22      mon      = new(vif, obs_mbx, cov_mbx);
23      sb       = new(obs_mbx);
24      cov      = new(cov_mbx);
25  endfunction
26
27  task automatic run();
28      fork
29          mon.run();
30          sb.run();
31          cov.run();
32      join_none
33          drv.run();
34  endtask
35
36  function void report();
37      sb.report();
38      cov.report();
39  endfunction
40

```

```
41 | endclass : cordic_env
```

Listing 2.22. The environment: build, connect, run

Three steps happen here, in a strict order. **Build**: every object is constructed, nothing is connected and nothing runs. **Connect**: the mailboxes are handed to the components that use them, which here is done by passing them to the constructors, so build and connect are textually merged; in a larger testbench with optional components you separate them, because you cannot connect two components until both exist. **Run**: the consumer threads are started with `join_none` so they run in the background, and the driver is started last.

Those three steps, done by hand here, are exactly what UVM automates. UVM's `build_phase` constructs components top-down, its `connect_phase` wires analysis ports bottom-up, its `run_phase` starts the threads. The mechanism is different, being a factory, a configuration database and a phasing engine, but the structure is the one you are reading. When Chapter 3 introduces those three method names, you will already know what belongs in each.

Why the phases have to be separate

Two components cannot be connected until both have been constructed, and a component cannot be constructed after another component has already started running and begun looking for it. A single-phase design therefore only works when the component graph happens to be a tree that you can build in dependency order by hand. As soon as there is a cycle – a scoreboard that needs a handle to the coverage collector and vice versa – you need construction to be complete before connection begins. That, and nothing more mysterious, is why UVM has phases.

2.6.8 The test

The test is the top of the pyramid: it decides what varies from run to run. Number of transactions, which constraints are active, which directed corner cases to add, how much back-pressure, which seed.

```
1 | `timescale 1ns/1ps
2 |
3 | module tb_cordic_top;
4 |
5 |     import cordic_pkg::*;
6 |     import tb_cordic_pkg::*;
7 |
8 |     logic clk = 1'b0;
9 |     always #5ns clk = ~clk;
10 |
11 |     cordic_if intf (.clk(clk));
12 |
13 |     cordic_rot dut (
14 |         .clk_i (clk),                .rst_ni      (intf.rst_n),
15 |         .req_valid_i (intf.req_valid), .req_ready_o (intf.req_ready),
16 |         .req_x_i (intf.req_x),        .req_y_i      (intf.req_y),
17 |         .req_theta_i (intf.req_theta),
18 |         .rsp_valid_o (intf.rsp_valid), .rsp_ready_i (intf.rsp_ready),
19 |         .rsp_x_o (intf.rsp_x),        .rsp_y_o      (intf.rsp_y)
20 |     );
21 |
22 |     cordic_env env;
23 |     int unsigned n_txn = 200;
24 |
```

```

25 initial begin
26     if (!$value$plusargs("NTXN=%d", n_txn)) n_txn = 200;
27
28     env = new(intf);
29     env.drv.reset();
30     env.run();
31
32     // directed corners first: they are the ones you can reason about
33     env.send_directed(X0_UNIT, 0, angle_t'(0));
34     env.send_directed(X0_UNIT, 0, angle_t'(PI_EXT));
35     env.send_directed(X0_UNIT, 0, angle_t'(-PI_EXT));
36     env.send_directed(X0_UNIT, 0, angle_t'(PI_EXT/2));
37     env.send_directed(0, 0, angle_t'(1234));
38
39     // then the random bulk
40     env.send_random(n_txn);
41
42     env.wait_drain(n_txn + 5, 200 * (n_txn + 5));
43     env.report();
44     $finish;
45 end
46
47 // independent watchdog
48 initial begin
49     #5ms;
50     $fatal(1, "WATCHDOG: test did not finish");
51 end
52
53 endmodule : tb_cordic_top

```

Listing 2.23. The top-level module and one test

The stimulus is in the test, not in the environment, and that separation is not stylistic. The environment is the part you reuse: the same driver, monitor, scoreboard and coverage collector serve every test of this block, and serve it again when the block is instantiated inside a larger system. The stimulus is different in every test, by definition. Put a `send_random(1000)` call inside the environment and you have welded one particular test into the reusable part; the next test then has to accept it or copy the whole environment.

Note the directed cases coming first. Random stimulus finds bugs you did not think of; directed stimulus proves the cases you can reason about by hand. Start with a handful of vectors whose correct answer you can compute on paper, so that when the random bulk fails you already know the basic path works.

2.7 The Mechanics That Hold It Together

Chapter 1 gave you the constructs the layered testbench is built from: handles, mailboxes, semaphores, events, dynamic threads and the lifetime rules that govern them. This section is about the four decisions the testbench makes with them, and about the one mechanism Chapter 1 could not cover, because it belongs to neither the class world nor the module world but to the join between them.

2.7.1 The virtual interface

Start with that join, because it is a genuine problem. `cordic_if` is an *interface instance*: elaborated, present in the module hierarchy, with a static name `tb_cordic_top.intf`. `cordic_drv`

is a *class*: created at run time, living on the heap, with no position in the hierarchy at all. A class cannot hold a hierarchical reference to something that might not exist, and there might be two instances of the interface, or none.

A virtual interface is the bridge. It is a *handle* to an interface instance, of type `virtual cordic_if`, and it can be stored in a class member, passed to a function, and assigned at run time.

```

1 // in the top-level module: a static instance, elaborated at time 0
2 cordic_if intf (.clk(clk));
3
4 // in the class: a handle, null until someone assigns it
5 class cordic_drv;
6     virtual cordic_if vif;
7     function new(virtual cordic_if vif, mailbox #(cordic_txn) req_mbx);
8         this.vif = vif;           // the handle is copied in here
9     endfunction
10 endclass
11
12 // back in the module: the assignment
13 initial env = new(intf);         // `intf' converts to `virtual cordic_if'

```

Listing 2.24. How the interface handle reaches the classes

The whole mechanism is that one argument. `new(intf)` passes the static interface instance where a virtual interface handle is expected, and the tool performs the conversion. The environment stores it and passes it on to the driver and the monitor. Nothing else is needed for a testbench of this size.

△ Trap: A null virtual interface fails at run time, not compile time

`virtual cordic_if vif;` initialises to null. If you forget to assign it, the first `vif.drv_cb.req_valid <= 1'b1;` is a null-handle dereference: a fatal run-time error with a message like "Illegal reference through a null virtual interface", pointing at a line that looks perfectly correct. This is one of the two most common first-day errors in a class-based testbench (the other is forgetting `cg = new()`). If a driver appears to do nothing at all, check the handle first.

Virtual interfaces, modports and the configuration database

Two refinements you will meet later. A virtual interface can be typed to a modport: `virtual cordic_if.drv vif;` restricts the handle to the driver's view, so an accidental write to a monitor-only signal is a compile error; tool support is good but not universal, which is why `tb_cordic_pkg.sv` uses the plain form. And UVM replaces the constructor argument with a lookup in a configuration database (Chapter 3), so that a deeply nested component can find its interface without every component above it passing the handle down. For four components, the constructor argument is clearer.

2.7.2 Why the components talk through mailboxes

The mailbox itself is Section 1.3's business. What belongs here is why there is one at all, because the obvious alternative looks simpler: the monitor could call `sb.check(t)` directly and save an object, a thread and a blocking call.

The reason is that a direct call is a *dependency*. A monitor that calls `sb.check()` needs a handle to the scoreboard, needs the scoreboard to exist before the monitor starts, and cannot be

attached to a testbench that has no scoreboard, or two of them, or a scoreboard that wants to think for a while before answering. A monitor that calls `obs_mbx.put(t)` needs none of those things. It needs a mailbox handle, which it was given at construction, and it does not care who is on the other end, whether anyone is, or how long they take. That is the whole of what makes the monitor reusable: it is the *connection* that is configurable, not the monitor.

Look back at Listing 2.22 with that in mind: three mailboxes created and passed to four constructors are the entire wiring diagram of the testbench. When Chapter 3 replaces the mailbox with an analysis port it takes the same idea one step further, letting the monitor broadcast once to any number of subscribers without naming one of them.

Bounded or unbounded is a real decision, and it is the one mailbox question this chapter answers rather than Chapter 1. An unbounded mailbox never blocks the producer, so `send_random(10000)` in Listing 2.23 returns almost immediately with ten thousand objects queued in memory. A bounded one applies natural back-pressure to the generator, which then produces transactions at the rate the DUT consumes them. For a testbench of this size either is fine; for a long random run with large transactions, bound it.

△ **Trap: An unbounded mailbox turns a deadlock into a memory leak**

If the driver thread dies, from a null handle, a fatal error or a `disable` that caught more than you meant, a bounded mailbox blocks the generator almost immediately and you notice. An unbounded one lets the generator run to completion, so the test proceeds to `wait_drain`, sits there until the timeout, and reports a drain failure that gives no hint that the driver is dead. Bounding the request mailbox to four or eight entries makes a dead consumer visible within a few transactions.

The other two primitives of Section 1.3 have narrower jobs here. A `semaphore` belongs where two stimulus threads want the same physical resource, a named `event` where one thread announces a one-off milestone such as reset completion. Neither appears in `tb_cordic_pkg.sv`, because one driver owns the interface and the reset is sequential with the stimulus. If you reach for a semaphore in a block-level testbench, check first whether the real answer is a second mailbox.

2.7.3 Threads, and where `disable` bites

The layered testbench uses two of the fork endings from Section 1.3 and no others. `join_none` is what every component's `run()` uses: start a background thread and give control back to the caller, which is what lets `cordic_env::run()` start three consumers and then the driver without ever blocking. `join_any` with `disable fork` is the timeout idiom, and it is the one that needs a warning at the point of use.

△ **Trap: `disable fork` kills every descendant, not just the last fork**

`disable fork` terminates all active descendants of the *current* thread, which includes threads started by a `join_none` several calls ago. The classic damage: a task uses the timeout idiom, and the `disable fork` silently kills the monitor that an earlier `env.run()` left running. The scoreboard then receives nothing, the test finishes with `n_checked == 0`, and the naive verdict logic reports a pass. Contain the damage by wrapping the timeout in its own `fork ... join`, so the `disable` only sees that block's children:

```
1 fork begin : timeout_scope
2   fork
3     wait_for_response();
```

```

4     #1us;
5     join_any
6     disable fork;           // only kills children of timeout_scope
7     end join

```

2.7.4 Static versus automatic, at the fork

Section 1.3.3 states the rule: a task declared in a module, interface or package has *static* lifetime unless you write `automatic`, so one copy of its locals is shared by every concurrent call. The rule is easy to read and easy to forget. Here is what it does the first time you fork in a loop, which is where a testbench meets it.

```

1 task print_after(int n);           // `int n' is STATIC here
2     #(n * 10ns);
3     $display("thread %0d at %0t", n, $time);
4 endtask
5
6 initial begin
7     for (int i = 1; i <= 4; i++) begin
8         fork
9             print_after(i);         // four concurrent calls, ONE copy of n
10        join_none
11    end
12 end

```

Listing 2.25. The bug: all four threads print 4

All four calls share the single static `n`. The first sets it to 1 and suspends at the delay, the second sets it to 2 and suspends, and by the time any of them wakes up `n` is 4. The output is four identical lines, all saying `thread 4`, all at the same time, and the testbench appears to have started one thread instead of four. There are two fixes and you want both.

```

1 task automatic print_after(int n); // each call gets its own n
2     #(n * 10ns);
3     $display("thread %0d at %0t", n, $time);
4 endtask
5
6 initial begin
7     for (int i = 1; i <= 4; i++) begin
8         automatic int k = i;       // a fresh k per iteration
9         fork
10            print_after(k);
11        join_none
12    end
13 end

```

Listing 2.26. The fix: automatic task, automatic loop variable

`automatic` on the task gives every invocation its own arguments and locals; `automatic int k = i`; inside the loop body gives every *iteration* its own variable to capture the counter into. Both are needed, because without the second the forked thread reads `i` after the loop has advanced and you get the same symptom for a different reason.

△ Trap: Forking in a loop without automatic

The symptom is always the same and always misleading: N threads that all behave as if they had received the last value. If you ever see "my four drivers are all sending transaction 4", this is it, and it is worth recognising on sight because nothing in the log points at the loop. Note that everything declared inside a `class` is already automatic, which is why the layered testbench is far less exposed to this than the flat one you started with.

2.7.5 Ending a test that contains forever loops

The monitor, the scoreboard and the coverage collector all run `forever`. None of them will ever finish. So what decides that the test is over?

The answer, in a hand-built testbench, is a two-part protocol: a *drain* condition and a *timeout*. The drain condition is a statement about the testbench's own bookkeeping, "every transaction I sent has been checked", and the timeout is the guarantee that the wait terminates even when the condition never becomes true. That is precisely `wait_drain()` in Listing 2.12: it polls `sb.n_checked` against the number sent, on every clock, up to a cycle limit, and calls `$fatal` if the limit is reached.

Once `wait_drain()` returns, the test calls `report()` and `$finish`, which terminates every remaining thread including the three `forever` loops. The `final` block, if there is one, still runs.

Be careful with the limit. `wait_drain(n, 200*n)` allows 200 cycles per transaction, which is generous for a CORDIC that takes 16. Add a test that stalls `rsp_ready` for a thousand cycles, or raise `ITER` to 32, and the same limit becomes marginal: the test starts failing intermittently with a drain timeout that reads exactly like a DUT bug. Derive the limit from the design's real latency, or use the activity watchdog of Listing 2.11.

This is the problem UVM's objections solve

Every component in a UVM testbench can `raise_objection` when it starts something that must not be interrupted and `drop_objection` when it is done; the phase ends when the count reaches zero. That is a distributed version of `wait_drain`: instead of one place in the test knowing how many transactions are outstanding, each component states its own condition and the framework computes the conjunction. `set_drain_time()` adds a settling period, and a global timeout makes a stuck objection fail instead of hang. Same two-part protocol, bookkeeping moved into the library.

2.8 Assertions In and Around the Testbench

Section 1.6 covered the syntax of SystemVerilog Assertions, the sampling semantics and the operator set. This section answers the question that follows: given a working testbench and a set of properties, where does each one belong. The answer is not "in the RTL", and it is not "all in one checker file" either.

2.8.1 Immediate assertions: checking code

An immediate assertion belongs in checking code, and its most valuable use is one nobody teaches: checking the testbench.

```

1 // inside a scoreboard or a monitor
2 a_no_orphan : assert (pending.size() > 0)
3   else $error("response with no outstanding request");
4
5 a_range : assert (t.theta >= -PI_EXT && t.theta <= PI_EXT)
6   else $fatal(1, "generator produced an illegal angle: %0d", t.theta);

```

The second one is worth a comment. It checks the *testbench*, not the DUT: if a constraint is wrong and the generator produces an out-of-range angle, the DUT's own assertion will fire and you will spend an hour suspecting the design. An assertion on your own stimulus, at the point where the stimulus is produced, turns that hour into a line number.

2.8.2 Concurrent assertions: protocol properties

A concurrent assertion belongs wherever a rule about the *protocol* is stated, which is usually not in the testbench at all. `cordic_rot` carries one, because the rule is part of its published contract:

```

1 property p_req_stable;
2   @(posedge clk_i) disable iff (!rst_ni)
3     (req_valid_i && !req_ready_o) | =>
4       $stable(req_x_i) && $stable(req_y_i) && $stable(req_theta_i);
5 endproperty
6
7 a_req_stable : assert property (p_req_stable)
8   else $error("cordic_rot: request payload changed while stalled");

```

Listing 2.27. The request-stability property from `cordic_rot.sv`

Read it as a sentence: on every rising edge of `clk_i`, ignoring anything that happens while reset is asserted, if `req_valid_i` is high and `req_ready_o` is low in this cycle, then in the *next* cycle ($|=>$, the non-overlapping implication) the three payload signals must be unchanged. That is the valid/ready stability rule, stated once, checked on every cycle of every simulation.

Checking the same thing procedurally needs a process, a set of shadow registers, a condition and the discipline to keep all of it in step with the protocol:

VHDL

```

-- VHDL-2008: a process, by hand
stab : process (clk) is
  variable px : unsigned(15 downto 0);
begin
  if rising_edge(clk) then
    if req_valid = '1' and
       req_ready = '0' then
      assert req_x = px
        report "payload changed"
        severity error;
    end if;
    px := req_x;
  end if;
end process stab;

```

SystemVerilog

```

// SVA: one property
property p_stable;
  @(posedge clk)
    disable iff (!rst_n)
      (req_valid && !req_ready)
      | => $stable(req_x);
endproperty
a_stable : assert property
  (p_stable)
  else $error("payload changed");

```

2.8.3 bind: checkers without touching the RTL

Assertions inside the DUT are right when they are part of the design contract, as `a_req_stable` is. They are wrong when they belong to verification: they clutter the source, they need an `'ifndef SYNTHESIS` guard, and a designer will eventually delete them. the *Design Cookbook* introduced `bind`, which attaches a module instance to another module from the outside without editing the target. This is its main use.

```

1 // ---- a standalone checker module, in its own file -----
2 module cordic_checker
3     import cordic_pkg::*;
4 (
5     input logic    clk_i, rst_ni,
6     input logic    req_valid_i, req_ready_o,
7     input logic    rsp_valid_o, rsp_ready_i,
8     input angle_t req_theta_i
9 );
10
11 // valid must never drop before ready
12 a_valid_held : assert property (@(posedge clk_i) disable iff (!rst_ni)
13     (req_valid_i && !req_ready_o) | => req_valid_i)
14     else $error("req_valid dropped before req_ready");
15
16 // a response must eventually arrive after an accepted request
17 a_eventual_rsp : assert property (@(posedge clk_i) disable iff (!rst_ni)
18     (req_valid_i && req_ready_o) | -> ##[1:100] rsp_valid_o)
19     else $error("no response within 100 cycles");
20
21 // prove that the interesting corner really happened
22 c_theta_pi : cover property (@(posedge clk_i) disable iff (!rst_ni)
23     (req_valid_i && req_ready_o && req_theta_i == angle_t'(PI_EXT)));
24
25 endmodule : cordic_checker
26
27 // ---- attach it, from the testbench file, without editing the DUT -----
28 bind cordic_rot cordic_checker u_chk (
29     .clk_i      (clk_i),
30     .rst_ni     (rst_ni),
31     .req_valid_i (req_valid_i),
32     .req_ready_o (req_ready_o),
33     .rsp_valid_o (rsp_valid_o),
34     .rsp_ready_i (rsp_ready_i),
35     .req_theta_i (req_theta_i)
36 );

```

Listing 2.28. A protocol checker bound to the DUT

The `bind` statement names a target module *type*, so every instance of `cordic_rot` anywhere in the design gets a checker, including instances buried inside a system-level testbench you have not written yet; you can also bind to a single instance by naming it. The port connections are written in the scope of the target, so `clk_i` refers to the target's own `clk_i`.

Because the port expressions are elaborated in the target's scope, a mismatched width is silently truncated or extended exactly as an ordinary port connection would be. Write every connection by name and check the widths.

2.8.4 cover property: proving a corner happened

The `c_theta_pi` statement in Listing 2.28 is a `cover property`, and it exists to answer a question the assertions cannot: did the corner they guard ever happen? If the count is zero, no assertion about that corner ever got the chance to fail, and a green run means nothing. This is the cheapest form of coverage in the language and it is criminally underused:

```

1 // did back-pressure ever actually stall a response?
2 c_rsp_stalled : cover property (
3   @(posedge clk_i) disable iff (!rst_ni)
4     rsp_valid_o && !rsp_ready_i);
5
6 // did two requests ever arrive back to back?
7 c_back_to_back : cover property (
8   @(posedge clk_i) disable iff (!rst_ni)
9     (req_valid_i && req_ready_o) ##1 (req_valid_i && req_ready_o));

```

For every assert property, ask whether its antecedent ever becomes true. If you cannot prove it from the log, write the matching cover property.

2.9 Running It

A testbench you cannot run reproducibly is not finished. This section gives the commands for QuestaSim, which is what the department licenses, and one line each for the two other commercial simulators. the *Simulation Cookbook* covers the open-source route, which has different constraints: no constrained random, no functional coverage, and a compile-then-run model rather than an interpreted one.

2.9.1 The four commands

```

# 1. create and map a working library (once per project)
vlib work
vmap work work

# 2. compile. Order matters: packages before the things that import them.
vlog -sv -work work \
    +incdir+../rtl \
    ../rtl/cordic_pkg.sv \
    ../rtl/cordic_if.sv \
    ../rtl/cordic_rot.sv \
    ../tb/tb_cordic_pkg.sv \
    ../tb/tb_cordic_top.sv

# 3. optimise, keeping visibility for debug
vopt -o tb_opt +acc=rnb work.tb_cordic_top

# 4. run, batch mode, fixed seed
vsim -c -sv_seed 12345 +NTXN=500 -do run.do work.tb_opt

```

Four things in that sequence are not obvious if your only QuestaSim experience is `vcom` and `vsim` on VHDL. `-sv` tells `vlog` to compile as SystemVerilog rather than Verilog-2001; without it, logic, `always_ff` and every class is a syntax error. `vopt` with `+acc=rnb` keeps registers, nets and blocks visible: QuestaSim optimises aggressively by default and removes signals that no longer need to exist, which makes them invisible in the waveform and unreachable from a `bind` or a

hierarchical reference. It costs simulation speed, so a regression that does not need waveforms should drop it. `-c` runs without the GUI, and `-sv_seed` sets the seed for the constraint solver and for `$urandom`; `-sv_seed random` picks a different one each run and prints it, which is what a nightly regression wants. Finally, `+NTXN=500` is a *plusarg*: a command-line argument the simulation itself can read.

2.9.2 Plusargs

A plusarg is how you change a test's behaviour without recompiling. SystemVerilog provides two system functions.

```

1  int unsigned n_txn      = 200;
2  int unsigned stall     = 25;
3  string       vector_file = "";
4  bit          verbose    = 0;
5
6  initial begin
7      void'($value$plusargs("NTXN=%d", n_txn));
8      void'($value$plusargs("STALL=%d", stall));
9      void'($value$plusargs("VEC=%s", vector_file));
10     verbose = $test$plusargs("VERBOSE");
11     $display("config: NTXN=%0d STALL=%0d VEC=%s VERBOSE=%0b",
12             n_txn, stall, vector_file, verbose);
13 end

```

Listing 2.29. Reading the command line from inside the simulation

`$test$plusargs` returns 1 if the named argument appeared on the command line at all. `$value$plusargs` looks for an argument starting `+NTXN=`, parses the rest with the given format, writes it into `n_txn` and returns 1; if the argument is absent it returns 0 and leaves the variable alone, which is why initialising the variable at its declaration is essential.

△ Trap: `$value$plusargs` leaves the variable untouched on failure

If the plusarg is missing, or present but unparseable, the target variable keeps whatever it had. Declare it with a sensible default on the same line, and print the resulting configuration at the start of every run. A run whose log does not state its own configuration cannot be reproduced, and a regression result you cannot reproduce is worth very little.

UVM's plusargs are the same mechanism

`+UVM_TESTNAME=my_test`, `+UVM_VERBOSITY=UVM_HIGH` and `+uvm_set_config_int=...` are ordinary plusargs read with `$value$plusargs` inside the UVM library, exactly as in Listing 2.29. There is no simulator support for UVM specifically; it is a package that reads the same command line you can read.

2.9.3 A minimal `.do` file

```

# run.do -- batch execution with a non-zero exit code on failure
onerror {quit -code 3}

# log everything recursively so a failure can be debugged from the WLF
log -r /*

run -all

```

```
# TESTSTATUS: 0 = no errors, 1 = warning, 2 = error, 3 = fatal
quit -code [expr {[coverage attribute -name TESTSTATUS -concise] > 1 ? 1 : 0}]
```

The last line matters more than it looks. A simulation that printed a hundred `$error` messages still exits with status 0 unless you make it do otherwise, and a regression script that only checks the exit code records it as a pass. Either translate the simulator's error count into an exit code as above, or have your script grep the log for your own `*** TEST FAILED ***` marker. Do both.

2.9.4 Waveforms: the dump file is not the viewer

Two separate things are involved in looking at a waveform, and confusing them is the reason students ask why their `.vcd` will not open. One is the *dump file*, written by the simulator, which records who changed and when. The other is the *viewer*, a separate program that draws it.

QuestaSim writes its own format, a `.wlf` file, and shows it in its own wave window. `log -r /*` in the `do` file above is what fills it; `add wave -r /*` is what puts signals on the screen. Run interactively with `vsim -voptargs="+acc=rnb"` and the whole loop stays inside one tool, which is the fastest way to debug and the reason the department licenses it.

The portable route is a `$dumpfile/$dumpvars` pair in the testbench itself. It is plain IEEE 1800, so it works in every simulator, QuestaSim included, and it writes *VCD*: the IEEE 1364 text format that every tool on earth can read. Gate it behind a plusarg, because a VCD is enormous:

```
1 initial if ($test$plusargs("DUMP")) begin
2     $dumpfile("tb_cordic.vcd");
3     $dumpvars(1, tb_cordic_top.dut); // this scope only, not everything
4 end
```

The first argument to `$dumpvars` is a depth: 1 means the named scope only, 2 adds its children, and 0 means everything below it, recursively. `$dumpvars(0, tb_cordic_top)` on a toy accumulator costs nothing. The same line on a real design, run for a few million cycles, produces a file measured in gigabytes, and you will spend longer waiting for the viewer to index it than you spent on the simulation. Name the scope you care about.

A VCD opens in GTKWave, the long-standing open-source viewer, and in Surfer, which is newer, has a modern interface and runs in a browser as well as natively. QuestaSim will also read a VCD if somebody hands you one. Where the flow allows it, prefer *FST*, a compressed binary format that is far smaller than VCD and far faster to open; Verilator writes it with `--trace-fst` (the *Simulation Cookbook*) and both viewers read it. For anything longer than a few thousand cycles, FST is what you want.

Dump selectively and dump late. A plusarg-gated `$dumpvars` on one scope, turned on only for the seed that failed, beats a nightly regression that writes a gigabyte per run and is never looked at.

2.9.5 Coverage, and the other simulators

```
# collect coverage: same flags on vlog, vopt and vsim
vlog -sv +cover=bcefsx -work work ../rtl/*.sv ../tb/*.sv
vopt -o tb_opt +cover=bcefsx +acc=rnb work.tb_cordic_top
vsim -c -coverage -sv_seed random work.tb_opt \
    -do "run -all; coverage save run1.ucdb; quit"
vcover merge merged.ucdb run*.ucdb
vcover report -details -html -htmldir covhtml merged.ucdb
```

`+cover=bcefsx` asks for branch, condition, expression, FSM, statement and extended toggle

coverage. Those are the *code* coverage metrics of Section 1.5, computed from the RTL without any help from you. The functional coverage from Listing 2.20 lands in the same UCDB, and the merge report shows both. Read them as answers to different questions.

The other two commercial simulators use a single compile-and-run command. Cadence Xcelium: `xrun -sv -access +rwc -seed 12345 +NTXN=500 *.sv`. Synopsys VCS: `vcs -sverilog -full64 -debug_access+all -o simv *.sv` followed by `./simv +ntb_random_seed=12345 +NTXN=500`. The `plusarg` spelling for the seed is the only genuinely tool-specific thing in this chapter.

Recipe: Put all of this in a Makefile on day one

Nobody remembers `+acc=rnb` at eleven at night. A four-target Makefile (`comp`, `sim`, `wave`, `cov`) that hides the flags is twenty minutes of work and it is the difference between a testbench that gets run and one that gets run once. the *Simulation Cookbook* gives a complete one.

2.10 The Coverage-Closure Loop

Chapter 1 names this loop in a sentence. It is worth a section, because it is the daily working rhythm of the whole activity and because every step of it has a way of going wrong. A constrained-random testbench is not a test, it is a *machine for producing tests*. Running it once proves almost nothing. Running it correctly is a loop:

1. **Run with N different seeds.** Ten seeds of a thousand transactions is a better use of an hour than one seed of ten thousand, because a bug that depends on a rare ordering is more likely to appear in ten independent sequences.
2. **Merge the coverage databases** from all the runs into one.
3. **Look at the holes.** Which bins are empty? Which crosses have zero hits? Which `cover` property never fired?
4. **Classify each hole.** Reachable but never produced by the constraints: adjust the constraints or add a directed test. Unreachable by construction: exclude it with `ignore_bins` and a comment saying why. Reachable and *should not be*: you have found a bug in the specification or in your understanding of it, which is the most valuable outcome available.
5. **Repeat** until the remaining holes are all excluded with reasons.

What this looks like in practice, honestly, is less tidy. The first run typically reaches 60 to 80% functional coverage almost immediately, because the constraints were written by someone thinking about the interesting cases. The next 15% takes a day of adjusting distributions. The last few percent are individually painful: each is a corner the constraints cannot reach without being rewritten, and the usual resolution is a small directed test that forces it, added to the test list with a comment.

Two failure modes are worth naming, because both feel like progress. The first is chasing coverage instead of bugs: it is entirely possible to reach 100% functional coverage on a design with a serious bug, if the covergroups describe the stimulus you sent rather than the behaviour you needed. Coverage measures what you tried; the scoreboard measures whether it worked, and a green coverage report with a weak scoreboard is worse than a red one because it stops people looking. The second is excluding holes to close the report. `ignore_bins` exists for genuinely unreachable states, and every use of it should carry a comment a reviewer can check. An exclusion added the week before tape-out, with no comment, is how a bug ships.

Coverage tells you where you have not looked. It never tells you that the design is correct. Only the checker does that, and only if you have proved the checker can fail.

Recipe: A regression script in five lines

```
for seed in $(seq 1 20); do
  vsim -c -sv_seed $seed +NTXN=1000 \
    -do "run -all; coverage save cov_$seed.ucdb; quit" work.tb_opt \
    | tee log_$seed.txt
done
grep -l "TEST FAILED" log_*.txt
vccover merge merged.ucdb cov_*.ucdb && vccover report -details merged.ucdb
```

That is the whole methodology. Everything a commercial regression manager adds is bookkeeping on top of these three commands.

Where this chapter stops and Chapter 3 starts

Nothing in `tb_cordic_pkg.sv` needs UVM. It is about 470 lines and does everything described in this chapter. What UVM adds is standardisation: the same six components with agreed names and base classes, a factory so a test can substitute one component without editing the environment, a configuration database so a handle can reach a deeply nested component, phasing, and objections. Every one of those solves a problem you have now met by hand. That is why this chapter comes first.

2.11 Checklist

Translating a VHDL testbench

- ‘timescale 1ns/1ps at the top of every file, or timeunit in every module and package. Never inherit it.
- A VHDL process ending in `wait;` becomes an initial block. A process that loops forever becomes `always` or `forever`.
- `clk <= not clk after 5 ns;` becomes `always #5ns clk = ~clk;`. Blocking assignment, and initialise `clk` at declaration.
- `report ... severity note` becomes `$display;` `severity error` becomes `$error;` `severity failure` becomes `$fatal(1, ...)`.
- `assert c report m severity error` becomes `a_name: assert (c) else $error(m)`. Always label it, always give a message with values in it.
- End with `$finish`, not by starving the event queue.

Never do this

- Never drive a DUT input with `=` from a block triggered by the same clock edge the DUT samples on. Use `<=`, or better, a clocking block.
- Never mix `=` and `<=` on the same signal.
- Never call `randomize()` without testing its return value.
- Never let the driver hand transactions to the monitor or the scoreboard. The monitor reconstructs from the pins or the testbench proves nothing.

- Never write a reference model by copying the RTL.
- Never raise a tolerance to make a test pass without recording why.
- Never declare a testbench task without `automatic`, and never fork in a loop without copying the loop variable into an `automatic` local.
- Never trust a `TEST PASSED` that does not also state how many transactions were checked.

Always do this

- Drive and sample through a clocking block with `default input #1step output #1ns`.
- Use `===` and `!==` in a monitor.
- Give every transaction a unique `id` and a `convert2string()`.
- Add a watchdog before the first check, and prefer one that times out on lack of progress.
- Assert a minimum transaction count as part of the pass condition.
- Apply random back-pressure to every ready signal from the first test.
- Write both stimulus coverage and DUT-state coverage, and a `cover property` for every assertion whose antecedent might never occur.
- Bind protocol checkers rather than editing the RTL.
- Break the design on purpose once, and confirm the testbench goes red.
- Print the run's full configuration, including the seed, in the first lines of the log.
- Gate `$dumpfile/$dumpvars` behind a plusarg and name the scope. `$dumpvars(0, top)` on a real design writes gigabytes.

Structure

- Transaction: data plus constraints plus `convert2string()`. No behaviour.
- Generator: makes transactions, puts them in a mailbox. Knows nothing about pins.
- Driver: mailbox to pins. Knows the protocol only.
- Monitor: pins to mailbox. Passive, and independent of the driver.
- Scoreboard: reference model plus comparison plus statistics.
- Coverage: what did we actually exercise?
- Environment: build, connect, run. No stimulus.
- Test: stimulus, constraints, counts, seed. The only thing that changes between runs.

UVM Basics

The Universal Verification Methodology is a class library written in SystemVerilog. It is not an extension of the language, not a tool, and not a new file format. In Chapter 2 you built a transaction, a generator, a driver, a monitor, a scoreboard, a coverage collector and an environment by hand. This chapter puts a UVM base class in front of each of those six boxes, shows what the library adds on top, and then builds a complete, runnable UVM testbench for the CORDIC rotator. At the end you will be able to read an industrial UVM testbench, write a small one, and judge honestly whether a given project needs one at all.

3.1 What UVM is, and where it came from

UVM is a library of about 350 SystemVerilog classes, plus a few dozen macros, plus a handful of package-scope functions. You compile it once, you `import` it like any other package, and you extend its classes like any other classes. Everything it does could be done by hand, and in Chapter 2 you did do it by hand. Nothing in UVM required a change to IEEE 1800: every construct the library is built from, classes and virtual methods, parameterised types, queues, mailboxes and dynamic threads, is in Chapter 1. That is the single most important sentence in this chapter, and if you keep it in mind the rest of UVM stops being mysterious.

The point matters because of how UVM is usually taught. It arrives with its own vocabulary (agents, sequencers, phases, objections, the factory), its own naming convention, its own report format and its own command-line switches, and the effect on a newcomer is that of meeting a new language. It is not. ``uvm_info` is a macro that expands to an `if` and a method call. `uvm_component` is a class with a `string` name and a parent pointer. `run_phase` is a virtual task that somebody else calls.

Why the UVM listings are a different colour

Throughout this chapter, listings that contain UVM identifiers use the `uvmcode` environment. It typesets UVM class names, macros and phase methods in rust, while the SystemVerilog keywords stay blue. Every rust identifier you see is something defined in `uvm_pkg.sv` or `uvm_macros.svh`. Nothing rust is part of IEEE 1800. Look at any listing in this chapter and mentally delete the rust: what is left is the language you already learned in Chapter 1.

3.1.1 A short history

Constrained-random, coverage-driven verification became mainstream around 2002, and each of the three big EDA vendors shipped a methodology library with its simulator. Synopsys had VMM (Verification Methodology Manual), written for VCS. Cadence had eRM, ported from the *e* language into SystemVerilog as the Universal Reuse Methodology, URM. Mentor had AVM (Advanced Verification Methodology), which was the first of the three to be released under an

open licence and the first to be built around TLM communication rather than around signals. In 2008 Mentor and Cadence merged AVM and URM into the Open Verification Methodology, OVM. OVM was genuinely open: Apache-licensed source, downloadable, and usable on any simulator. It was widely adopted, which left Synopsys with a methodology nobody outside VCS used. In 2009 Accellera took OVM as the starting point for a standard methodology, added the register layer from VMM and a few ideas from both camps, and in February 2011 released UVM 1.0. The version that most production code was written against is UVM 1.1d (2013); UVM 1.2 (2014) changed a few interfaces and was, for a while, unpopular precisely because it broke code. In 2017 the IEEE standardised the class reference as IEEE 1800.2-2017, revised in 2020. Note what was standardised: the *API*, not the implementation. Accellera continues to publish a reference implementation (the current one identifies itself as UVM-1.2 / IEEE 1800.2-2020 depending on the build), and every commercial simulator ships a copy, usually with vendor patches for performance. You almost never download UVM yourself: you point the compiler at the copy in the tool installation.

3.1.2 What it costs

Be honest with yourself about the price, because most of the UVM literature is not.

- **The learning curve is long.** A competent SystemVerilog programmer needs a week to write a first working UVM testbench and a month before the factory and the configuration database stop producing surprises. Compare that with Chapter 2, where the entire infrastructure was about 470 readable lines.
- **The code is verbose.** The minimal CORDIC testbench in Section 3.12 is about 330 lines of UVM against about 470 lines of hand-written testbench, and the UVM version does less, because the hand-written one included the reference model and the reporting that UVM provides. Real testbenches are verbose in a different way: boiler-plate constructors, boiler-plate registrations, boiler-plate `build_phase` bodies.
- **It is slow at run time.** The factory, the string-based configuration database, the field-automation macros and the objection mechanism all cost simulator time. On a small block the UVM overhead can dominate the RTL simulation entirely. The field macros in particular are notorious: see Section 3.8.
- **No open-source simulator runs it.** Be precise about why, because the usual summary is wrong. Verilator compiles classes but not with the completeness UVM needs; it accepts a `constraint` block and then ignores it, warning `CONSTRAINTIGN` and randomising as if the block were not there; and it rejects `covergroup` outright as unsupported. the *Simulation Cookbook* explains what it does instead. Icarus Verilog is often said not to support classes at all, and that is simply untrue: version 12 with `-g2012` compiles and runs a class with a constructor, member functions, inheritance and `super.new()`. What it lacks is everything built on top. `randomize()` does not exist (`randomize` is not a method of `class`) and `constraint` blocks are rejected outright; `covergroup` is a syntax error; concurrent assertions are rejected; and a queue of class handles is refused with Sorry: `Queue of type ... is not yet supported`, which alone rules out most of `uvm_pkg.sv`. Even the class support that exists is fragile: two properties declared on one line, `int a, b;`, compile and then crash the runtime. Icarus is a useful Verilog-2005 simulator and a poor SystemVerilog one, and it will not run UVM.

In practice, then, UVM means a commercial licence: QuestaSim, Xcelium or VCS, each of which ships a pre-compiled copy of the library. For a student that means the university lab.

UVM is a reuse technology. If nothing will be reused, not across blocks, not across projects and not across people, you are paying the cost and collecting none of the benefit.

3.2 The same six boxes

Open `tb_cordic_pkg.sv` from Chapter 2 next to this page. It contains six classes: `cordic_txn`, `cordic_drv`, `cordic_mon`, `cordic_sb`, `cordic_cov` and `cordic_env`. UVM has a base class for every one of them. Table 3.1 is the whole translation.

Read the right-hand column again. Half the rows say “almost nothing”. That is not a criticism of UVM; it is the point. UVM’s value is not in any single base class. It is in the fact that ten thousand engineers agree on the same six boxes, the same names for them, the same connection mechanism and the same start-up sequence, so that a verification component written by one team drops into another team’s testbench.

3.2.1 The standard hierarchy

Figure 3.1 is the picture every UVM testbench draws. Learn it once and you will recognise the structure of any testbench you are handed.

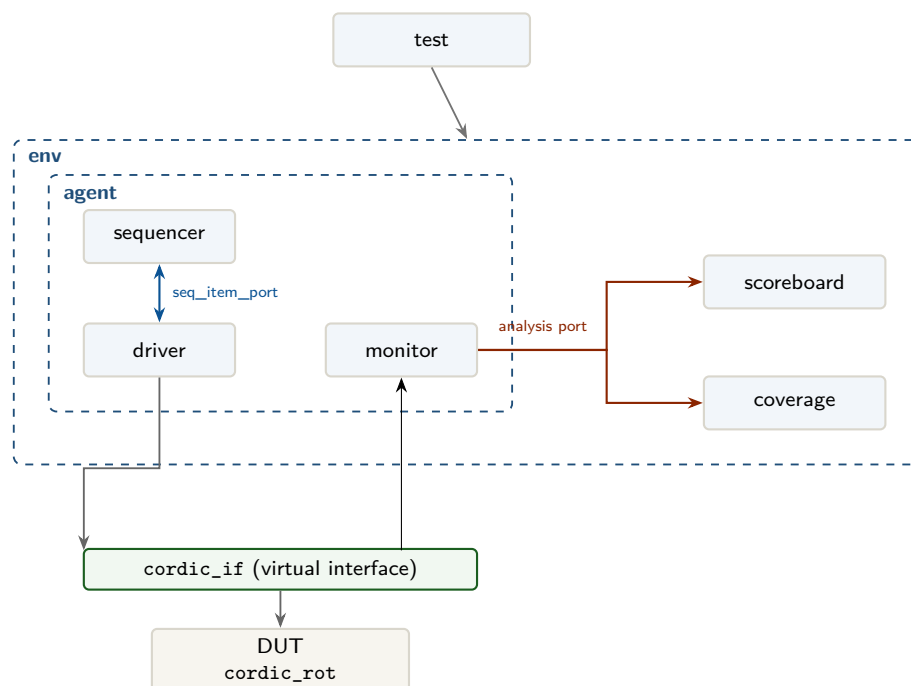


Figure 3.1. The standard UVM component hierarchy. Blue double arrows carry sequence items; rust arrows are analysis ports and are one-to-many. The driver and the monitor reach the pins through the same virtual interface, but only the driver ever writes to it.

Four things about that picture are worth stating explicitly, because they are the ones students get wrong.

1. **The test is at the top, and there is exactly one.** It is created by `run_test()` from the class name given on the command line. Everything else is created, directly or indirectly, by the test’s `build_phase`.

Table 3.1. Every class you wrote by hand in Chapter 2, its UVM base class, and what the base class actually adds.

Chapter 2	UVM base class	What UVM adds
<code>cordic_txn</code> (Chapter 1)	<code>uvm_sequence_item</code>	A name, a factory registration, automatic copy/compare/pack/print, and a handle back to the sequence that created it.
<code>env.send_random()</code>	<code>uvm_sequence</code>	Stimulus becomes an object with its own class hierarchy: it can be randomised, parameterised, extended, nested inside another sequence, and selected at run time.
<code>cordic_drv</code>	<code>uvm_driver #(T)</code>	A standard blocking port to the sequencer (<code>seq_item_port</code>) instead of a bare mailbox, plus phasing.
<code>cordic_mon</code>	<code>uvm_monitor</code>	Nothing, structurally. What you actually gain is the analysis port: one <code>write()</code> call feeds any number of subscribers, added without touching the monitor.
<code>cordic_sb</code>	<code>uvm_scoreboard</code>	Again almost nothing by itself. The gain is <code>uvm_tlm_analysis_fifo</code> , which decouples the monitor from the checking thread, and the report server, which counts your errors and decides the verdict.
<code>cordic_cov</code>	<code>uvm_subscriber #(T)</code>	A <code>write()</code> callback instead of a <code>forever</code> loop on a mailbox. One fewer process to start and stop.
<code>cordic_env</code>	<code>uvm_env</code>	A named component tree, automatic phase execution, the configuration database and the factory.
(the top-level module)	<code>uvm_test</code>	The test becomes a class, chosen on the command line with <code>+UVM_TESTNAME</code> . One compiled image, many tests.
(none)	<code>uvm_agent</code>	A packaging unit: sequencer + driver + monitor for one protocol, with an active/passive switch. This is the unit that gets reused.
66 <code>mailbox #(T)</code>	<code>uvm_analysis_port</code> , <code>uvm_tlm_analysis_fifo</code>	One-to-many broadcast, a non-blocking producer, and connection made in <code>connect_phase</code> rather

2. **The agent owns one protocol.** If the CORDIC had a configuration register bus as well as the data channel, it would get a second agent, with its own sequencer, driver and monitor.
3. **The scoreboard and the coverage collector are outside the agent.** They are specific to the DUT, not to the protocol. The agent is the reusable part; the scoreboard is not.
4. **The arrows from the monitor are broadcast.** Adding a third subscriber, say a protocol checker, requires no change at all to the monitor. This is the concrete benefit over the two mailboxes you wired by hand in Chapter 2, where the monitor had to know that there were exactly two consumers.

3.3 uvm_object versus uvm_component

Before any of the mechanisms make sense, internalise this distinction. Every class in UVM derives from `uvm_void`, and immediately below it the tree splits in two.

`uvm_object` is *data*. It has a name but no place. It is created, passed around, copied, compared and thrown away, thousands of times per simulation. Sequence items and sequences are objects. So are configuration objects and register-model fields.

`uvm_component` is *structure*. It has a name *and* a *parent*, so components form a tree. Components are created during `build_phase`, they exist for the whole simulation, and they are never destroyed. Drivers, monitors, sequencers, agents, environments, scoreboards and tests are components.

The mental model that works is the VHDL one. A `uvm_component` is an *instance in an elaborated design hierarchy*: it corresponds to a component instantiation inside an architecture, it has a path, and once elaboration is over it is fixed. A `uvm_object` is a *value of some type*: it corresponds to a variable of a record type inside a process. VHDL keeps those two worlds strictly apart, and so does UVM.

The consequence you will use every day is the hierarchical name. Because a component knows its parent, `get_full_name()` returns a dotted path:

```
uvm_test_top.env.agt.drv
uvm_test_top.env.agt.mon
uvm_test_top.env.sb
```

`uvm_test_top` is the name `run_test()` gives to the test it creates; everything below is what you named the handles in `build_phase`. Those strings are not cosmetic. They are the keys used by the configuration database (Section 3.7), by instance-specific factory overrides (Section 3.6), and by verbosity control (Section 3.11). Every message printed by UVM carries the full name of the component that produced it, which is how you find out which of your four identical agents is the one misbehaving.

Recipe: Name your handles after the instance, not the type

Write `drv = cordic_driver::type_id::create("drv", this);`, not `create("cordic_driver", this)`. The string becomes the path element. Short, consistent instance names such as `drv`, `mon`, `sqr`, `agt`, `env`, `sb` and `cov` make configuration paths like `"env.agt.*"` readable. A common convention is to use the same string as the handle name, so that the source and the printed topology look the same.

A `uvm_object` has no parent and therefore no path. Its `get_full_name()` returns just its own name, unless it is a sequence running on a sequencer, in which case UVM splices in the sequencer's path. This is why you cannot look up configuration “for a transaction”: there is nothing to look it up against.

3.4 Phases

In Chapter 2 the environment's constructor built the sub-objects, then you called `env.run()`, then you called `env.report()`. Construction, connection, execution, reporting, in that order, by hand. UVM does the same four things, calls them phases, and calls them for you.

A phase is a virtual method with a fixed name that UVM invokes on every component in the tree. You override the ones you need and ignore the rest.

Table 3.2. The common UVM phases, in execution order.

Phase	Kind	Order	What goes in it
<code>build_phase</code>	function	top-down	Create sub-components with the factory; fetch configuration.
<code>connect_phase</code>	function	bottom-up	Connect TLM ports, exports and analysis ports.
<code>end_of_elaboration_phase</code>	function	bottom-up	Last chance to inspect the assembled tree; print the topology.
<code>start_of_simulation_phase</code>	function	bottom-up	Print a banner, display the resolved configuration.
<code>run_phase</code>	<i>task</i>	all in parallel	The only phase that consumes simulation time.
<code>extract_phase</code>	function	bottom-up	Pull results out of components into a common place.
<code>check_phase</code>	function	bottom-up	Check for leftovers: unmatched transactions, empty FIFOs.
<code>report_phase</code>	function	bottom-up	Print your own summary.
<code>final_phase</code>	function	top-down	Close files, flush databases.

3.4.1 Why build is top-down and connect is bottom-up

This is not an arbitrary choice, and understanding it removes most build-order confusion.

`build_phase` must be top-down because a parent *creates* its children. The test's `build_phase` creates the environment; only then does the environment exist, so only then can UVM call the environment's `build_phase`, which creates the agent, and so on. The tree grows downwards as the phase executes. A useful side effect: because the parent runs first, the parent can write configuration into the database that the child will read a moment later in its own `build_phase`. That is exactly how the active/passive setting and the virtual interface reach the driver.

`connect_phase` must be bottom-up because a connection needs both ends to exist. When the agent connects `drv.seq_item_port` to `sqr.seq_item_export`, both the driver and the sequencer must already have been constructed *and* must already have constructed their own ports. Running the deepest components first guarantees it.

The remaining function phases are bottom-up for the same reason in reverse: by the time the environment reports, its children have already reported, so the environment can summarise them. `final_phase` is top-down only for symmetry with `build_phase`.

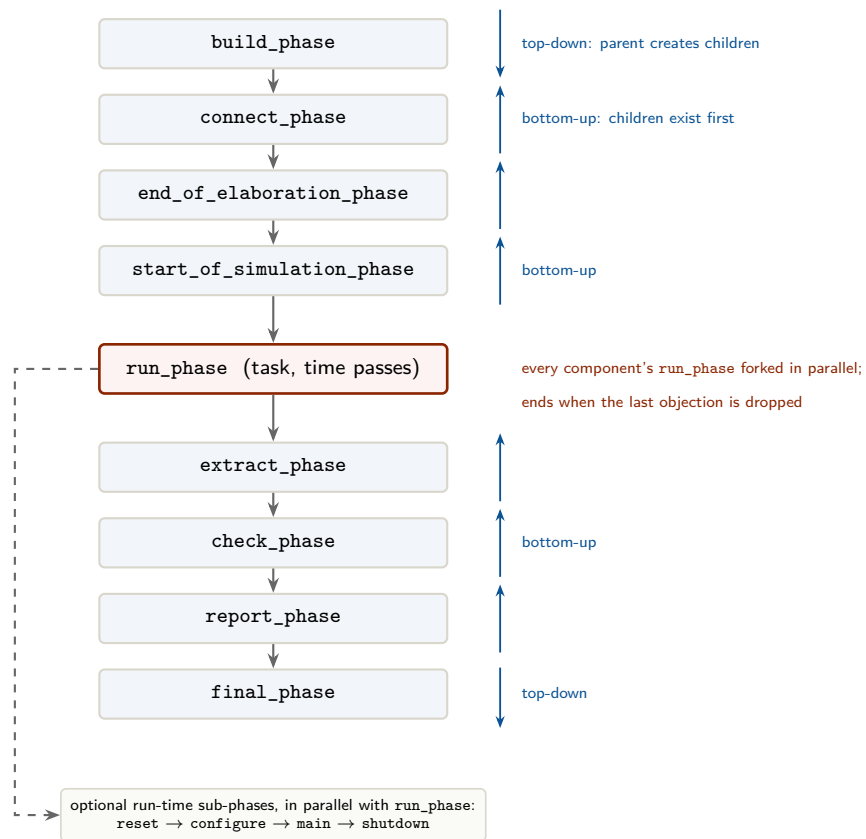


Figure 3.2. UVM phase order. Only **run_phase** consumes simulation time; everything above and below it happens at a single time step.

3.4.2 `super.build_phase(phase)`

Every phase method you override should begin by calling the base-class version:

```

1 class cordic_agent extends uvm_agent;
2   `uvm_component_utils(cordic_agent)
3
4   function new(string name, uvm_component parent);
5     super.new(name, parent);
6   endfunction
7
8   function void build_phase(uvm_phase phase);
9     super.build_phase(phase);    // never omit this line
10    // ... create children here ...
11  endfunction
12 endclass

```

Listing 3.1. The shape of every component

△ Trap: Omitting `super.build_phase(phase)`

This is the single most common UVM bug, and its symptom is baffling: things silently do not get configured.

`uvm_component::build_phase` is not empty. It calls `apply_config_settings()`, which walks the configuration database and automatically assigns any registered field of this component whose name matches a setting. `uvm_agent::build_phase` additionally reads the `is_active` setting. If you skip `super.build_phase(phase)`, none of that happens: your agent stays `UVM_ACTIVE` regardless of what the test asked for, and any field you expected the automation to fill stays at its default.

The failure produces no error and no warning. It produces an agent that drives pins when it was supposed to be passive.

In VHDL terms: you overrode a procedure in a derived type and forgot that the parent's version was doing real work. VHDL has no inheritance, so nothing in your previous experience warns you about this.

3.4.3 The run-time sub-phases

UVM also defines a second schedule of run-time sub-phases that executes in parallel with `run_phase`. There are four of them, namely `reset`, `configure`, `main` and `shutdown`, and each has a `pre_` and a `post_` variant, twelve in all. Each is a task, each is objected to independently, and the intent is that a testbench can, for example, apply reset in `reset_phase` and stimulus in `main_phase` without the two getting tangled.

Nobody uses the sub-phases

That is an exaggeration, but only a small one. Many teams – probably most – put everything in `run_phase` and manage ordering with ordinary `fork/join` and events. The sub-phases add a second synchronisation mechanism on top of objections, they interact badly with sequences that span phases, and *jumping* between them (`phase.jump()`) is a source of hard-to-debug behaviour. If you are learning UVM, use `run_phase` only. If you join a team that uses the sub-phases, follow the local convention.

3.5 Objections: how a test knows it is finished

In Chapter 2 the test finished because you wrote `wait_drain()`: a loop that polled the scoreboard's transaction count against an expected number, with a cycle-count timeout that called `$fatal` if the count never arrived. That works, but it puts the knowledge of “am I done” in one place, the test, and that place has to know how many transactions everyone else is still holding. UVM inverts it. Any component that has outstanding work *objects* to the phase ending. The phase ends when the objection count reaches zero and stays zero for the drain time. Nobody needs to know the global count.

```

1 task run_phase(uvm_phase phase);
2   cordic_random_seq seq;
3
4   phase.raise_objection(this, "stimulus running");
5   phase.phase_done.set_drain_time(this, 500ns);
6
7   seq = cordic_random_seq::type_id::create("seq");
8   seq.start(env.agt.sqr);
9
10  phase.drop_objection(this, "stimulus done");
11 endtask

```

Listing 3.2. An objection around the stimulus

The drain time is the direct equivalent of the margin you built into `wait_drain`'s timeout, but it means the opposite thing. It is not a limit: it is a grace period. When the last objection drops, UVM waits 500 ns of simulation time before killing the run phase, which gives the pipeline in the DUT time to push out the last results, the monitor time to see them, and the scoreboard time to check them. Without it, the test would end at the instant the last request was accepted and the last few responses would never be checked. Five hundred nanoseconds at a 10 ns clock is 50 cycles, which comfortably covers the CORDIC's $N + 2$ cycle latency.

The second argument to `raise_objection` and `drop_objection` is a description string. It costs nothing and it is invaluable: `+UVM OBJECTION TRACE` on the command line prints every raise and drop with its description, which is how you find out who is refusing to let your test finish.

3.5.1 Where to put them

Raise and drop objections in the test, around the sequence start. Everywhere else, prefer not to.

You *can* raise an objection in a driver, a monitor or a sequence, and sometimes you must, since a monitor still holding a half-observed transaction has a legitimate claim that the test is not over. But every raise/drop pair is a place where the test can hang, and objections raised deep in the tree are hard to trace. Start with one pair, in the test.

△ Trap: A test that ends at time zero

You run your brand-new testbench and the transcript says:

```

UVM_INFO ... [UVM/REPORT/SERVER]
--- UVM Report Summary ---
** Report counts by severity
UVM_INFO :      9

```

```

UVM_WARNING : 0
UVM_ERROR   : 0
UVM_FATAL   : 0
$finish called from file "uvm_root.svh"

```

at simulation time 0, with no transactions and no waveform. Nobody raised an objection, so the run phase had nothing to wait for and ended immediately.

The *opposite* bug – a test that never ends – has the same cause inverted: an objection raised and never dropped, usually because the `drop_objection` sits after a line that blocks forever. Run with `+UVM OBJECTION_TRACE` and read the last raise.

Note that a component whose `run_phase` is a plain `forever` loop (the driver, the monitor) does *not* keep the phase alive. UVM kills those threads when the phase ends. Only objections keep a phase alive.

Recipe: Always set a global timeout

An objection that is never dropped hangs the simulation. Bound it:

```

1 function void build_phase(uvm_phase phase);
2   super.build_phase(phase);
3   uvm_top.set_timeout(1ms, 1); // hard stop, overridable
4   env = cordic_env::type_id::create("env", this);
5 endfunction

```

or pass `+UVM_TIMEOUT=1ms,YES` on the command line. A regression that hangs costs a licence for hours; a regression that fails takes a minute.

3.6 The factory

The factory is the mechanism that most repays understanding, because it is the reason UVM testbenches are reusable at all.

3.6.1 The problem it solves

Suppose you have a working environment, and you now want a test in which the driver occasionally corrupts a transaction, so that you can check that the scoreboard notices. In Chapter 2 you would have to edit `cordic_env::new()`, because that is where `drv = new(...)` is written. Editing a working, shared environment in order to add one test is exactly what you must not do.

The factory removes the type name from the point of instantiation. Instead of naming the class you want, you ask a registry for “whatever class is currently registered under this type”, and a test can change the answer without touching the environment.

```

1 class cordic_driver extends uvm_driver #(cordic_item);
2   `uvm_component_utils(cordic_driver) // registers the type
3   // ...
4 endclass
5
6 class cordic_item extends uvm_sequence_item;
7   `uvm_object_utils(cordic_item) // registers the type
8   // ...
9 endclass
10

```

```

11 // inside cordic_agent::build_phase
12 drv = cordic_driver::type_id::create("drv", this); // ask the factory
13 // NOT: drv = new("drv", this);

```

Listing 3.3. Registration and creation

``uvm_component_utils`` and ``uvm_object_utils`` expand to a typedef of a parameterised registry class (`type_id`), a static `get_type()` function, a `create()` function and a `get_type_name()` override, and they run a static registration that inserts the type into the global factory. `type_id::create()` then asks the factory for an instance.

3.6.2 Overriding a type

With registration in place, a new test can substitute a derived class for the original everywhere it is created:

```

1 // ----- the new driver: it exists only in the test's own file -----
2 class cordic_bad_driver extends cordic_driver;
3   `uvm_component_utils(cordic_bad_driver)
4
5   int unsigned corrupt_one_in = 20;
6
7   function new(string name, uvm_component parent);
8     super.new(name, parent);
9   endfunction
10
11 // Same signature as the parent's. The parent declares drive() as
12 // `virtual', so the parent's run_phase dispatches to this one.
13 protected virtual task drive(cordic_item it);
14   if ($urandom_range(corrupt_one_in - 1) == 0) begin
15     `uvm_info(get_type_name(),
16               $sformatf("injecting error into %s", it.convert2string()),
17               UVM_LOW)
18     it.theta = it.theta ^ 16'h0040; // flip one bit
19   end
20   super.drive(it);
21 endtask
22 endclass
23
24 // ----- the test: no change at all to the env or the agent -----
25 class cordic_inject_test extends cordic_base_test;
26   `uvm_component_utils(cordic_inject_test)
27
28   function new(string name, uvm_component parent);
29     super.new(name, parent);
30   endfunction
31
32   function void build_phase(uvm_phase phase);
33     // The override MUST be set before the component is created, so it goes
34     // before super.build_phase(), which is what creates the env.
35     set_type_override_by_type(cordic_driver::get_type(),
36                               cordic_bad_driver::get_type());
37     super.build_phase(phase);
38   endfunction
39
40   task run_phase(uvm_phase phase);
41     cordic_random_seq seq;
42     phase.raise_objection(this);
43     seq = cordic_random_seq::type_id::create("seq");
44     void'(seq.randomize() with { n_items == 200; });
45     seq.start(env.agt.sqr);

```

```

46     phase.drop_objection(this);
47 endtask
48 endclass

```

Listing 3.4. An error-injecting driver, substituted by a test

Run this test and every `cordic_driver` in the tree becomes a `cordic_bad_driver`. The environment source file is untouched. The agent source file is untouched. Nothing was recompiled that did not have to be.

If there are several agents and you want only one of them corrupted, use an instance override, which takes a path string:

```

1 set_inst_override_by_type("env.agt_a.drv",
2                           cordic_driver::get_type(),
3                           cordic_bad_driver::get_type());

```

Instance overrides take priority over type overrides, and the path is matched against `get_full_name()` relative to the component that sets it. There are also `_by_name` variants of both calls, which take a type name as a string instead of a type handle; prefer the `_by_type` forms, because a typo in a type name is a silent no-op while a typo in a type handle is a compile error.

3.6.3 This is VHDL's `configuration`, done with classes

If you have used VHDL's configuration declarations, the factory will feel familiar, because they solve the same problem: choose which implementation fills a hole in the hierarchy, without editing the hierarchy.

VHDL

```

-- one architecture is written
-- against a component declaration
architecture struct of top is
    component drv_c is
        port (...);
    end component;
begin
    u_drv : drv_c port map (...);
end architecture;

-- a separate configuration unit
-- decides what fills the hole
configuration cfg_bad of top is
    for struct
        for u_drv : drv_c
            use entity work.drv_bad(rtl);
        end for;
    end for;
end configuration;

```

SystemVerilog

```

// the agent is written against
// the base type
drv = cordic_driver::type_id
      ::create("drv", this);

// a separate test decides what
// fills the hole
class cordic_inject_test
    extends cordic_base_test;
    function void build_phase(
        uvm_phase phase);
        set_type_override_by_type(
            cordic_driver::get_type(),
            cordic_bad_driver::get_type());
        super.build_phase(phase);
    endfunction
endclass

```

The differences are instructive. The VHDL configuration is elaborated once, statically, and selects an entity/architecture pair. The UVM override is executed at run time by ordinary code, so it can be conditional, can be driven by a plusarg, and can be undone. On the other hand the VHDL configuration is checked by the analyser, while a UVM override of an unrelated type fails only when the factory tries the `$cast` and reports an error at run time.

△ Trap: new instead of create

```

1 drv = new("drv", this); // legal, and wrong
2 drv = cordic_driver::type_id::create("drv", this); // right

```

The first line compiles, runs, and produces a perfectly functional driver. It simply bypasses the factory, so every override of that type silently does nothing. There is no warning. Your error-injection test will run happily and inject nothing.

The rule has no exceptions worth learning as a beginner: inside a testbench, components and sequence items are always built with `type_id::create()`. The only `new` calls in a UVM testbench are in constructors (`super.new(name, parent)`) and for objects that are genuinely not factory-registered, such as TLM ports and analysis ports.

Recipe: Check what the factory actually did

Call `uvm_factory::get().print()` in `end_of_elaboration_phase`, or pass `+UVM_CONFIG_DB_TRACE` and `-uvmtrace` depending on your simulator. The factory print lists every registered type and every override in force. If your override is not in that list, it was never applied.

3.7 The configuration database

The factory decides *what type* is built. The configuration database decides *what values* it is built with. It is a global, string-keyed, type-parameterised store:

```

1 uvm_config_db #(T)::set(uvm_component cntxt, string inst_name,
2                        string field_name, T value);
3
4 bit ok = uvm_config_db #(T)::get(uvm_component cntxt, string inst_name,
5                                string field_name, ref T value);

```

The lookup key is the concatenation of `cntxt.get_full_name()` and `inst_name`, so the two arguments together name a scope. Passing `null` as the context means “the root”, and then `inst_name` is an absolute path from the top. The path may contain `*` and `?` wildcards, and when several settings match, the one whose scope is *higher* in the tree wins, with later settings beating earlier ones at the same level. That precedence rule is what lets a test override what an environment set.

3.7.1 Getting the virtual interface into the driver

This is the one use of the configuration database that every testbench needs, and it is worth writing out in full because the two halves live in different files.

The DUT and the physical interface are in a module. Classes cannot be instantiated inside a module hierarchy, and a class cannot reach into a module by name in any portable way, so the handle has to be passed. The top-level module puts it into the database before `run_test()` returns control to the phasing:

```

1 module tb_cordic_uvm;
2   import uvm_pkg::*;
3   `include "uvm_macros.svh"
4   import cordic_pkg::*;
5   import cordic_uvm_pkg::*;

```

```

6
7  logic clk = 1'b0;
8  always #5ns clk = ~clk;
9
10 cordic_if vif (.clk(clk));
11
12 cordic_rot #(ITER(N)) dut ( /* ports: see the full listing below */ );
13
14 initial begin
15     uvm_config_db #(virtual cordic_if)::set(null, "*", "vif", vif);
16     run_test();
17 end
18 endmodule

```

Listing 3.5. Top-level module: publish the interface handle

and every component that needs pins fetches it in its `build_phase`:

```

1 function void build_phase(uvm_phase phase);
2     super.build_phase(phase);
3     if (!uvm_config_db #(virtual cordic_if)::get(this, "", "vif", vif))
4         `uvm_fatal("NOVIF",
5                 {"no virtual interface set for ", get_full_name(), ".vif"})
6 endfunction

```

Listing 3.6. Driver and monitor: fetch the interface handle

Read the arguments carefully.

- In the `set`, the context is `null` and the instance name is `"*"`: available to every component in the tree. In a testbench with two interfaces you would write `*.agt_a.*` and `*.agt_b.*` instead of `"*"`.
- In the `get`, the context is `this` and the instance name is the empty string: look up the key exactly for this component. Almost every `get` you will ever write has this form.
- The field name `"vif"` is a string and must match on both sides. It is conventional, not required, that it equals the name of the class member it is assigned to.
- The type parameter must match exactly. `virtual cordic_if` and `virtual cordic_if.drv` (a modport-qualified virtual interface) are *different* types and will not find each other.

△ Trap: The null virtual interface

Forget the `set`, misspell the field name, or use a wildcard that does not match, and the `get` returns 0. If you did not test the return value, `vif` stays `null`, and the first clocking-block access in `run_phase` produces something like

```

** Fatal: (vsim-3363) Null virtual interface reference.
Time: 0 ns Iteration: 0 Process:
/tb_cordic_uvm/#INITIAL#33/uvm_pkg::run_test

```

or, on other simulators,

```

Error-[NOA] Null object access
The object is being used before it was constructed/allocated.

```

Neither message mentions the configuration database, the field name, or the component. This is why the ``uvm_fatal` guard in Listing 3.6 is not optional style: it converts an unreadable simulator error at an arbitrary time into a message at time zero that names the component and the missing key.

The most common cause among students: the `set` was written *after* `run_test()` in the

same `initial` block. `run_test()` does not return until the simulation is over.

uvm_config_db versus uvm_resource_db

`uvm_config_db` is a thin, hierarchy-aware layer over `uvm_resource_db`. The resource database has no notion of scope precedence; the configuration database adds the “higher in the tree wins” rule. Use `uvm_config_db` unless you have a specific reason not to. Both are string-keyed and both fail silently on a typo, so use `+UVM_CONFIG_DB_TRACE` when a value does not arrive.

Recipe: Pass a configuration object, not fifteen fields

Once a component needs more than two or three settings, stop putting individual fields in the database. Define a class:

```
1 class cordic_cfg extends uvm_object;
2   `uvm_object_utils(cordic_cfg)
3   int unsigned      stall_pct = 25;
4   int unsigned      tol_lsb   = 12;
5   uvm_active_passive_enum is_active = UVM_ACTIVE;
6   function new(string name = "cordic_cfg"); super.new(name); endfunction
7 endclass
```

and put one handle in the database: `uvm_config_db #(cordic_cfg)::set(this, "env.agt*", "cfg", cfg);`. One key, one lookup, and the settings are type-checked by the compiler instead of by a string comparison.

3.8 Sequences and sequencers

In Chapter 2 the stimulus was a task on the environment: `send_random(n)` looped n times, created a transaction, randomised it and pushed it into a mailbox. That is adequate and it is easy to read, but it has a structural weakness: the stimulus is a *task*, and a task cannot be extended, cannot be randomised as a whole, and cannot be selected by name at run time. UVM makes stimulus an *object* instead, and that single change is what makes test-writing scale.

3.8.1 The sequence item

A sequence item is a `uvm_object` that carries one transaction. It is the UVM version of `cordic_txn`.

```
1 class cordic_item extends uvm_sequence_item;
2
3   rand data_t x0;
4   rand data_t y0;
5   rand angle_t theta;
6   data_t      x, y;           // filled in by the monitor
7
8   `uvm_object_utils_begin(cordic_item)
9     `uvm_field_int(x0,      UVM_DEFAULT | UVM_DEC)
10    `uvm_field_int(y0,      UVM_DEFAULT | UVM_DEC)
11    `uvm_field_int(theta,   UVM_DEFAULT | UVM_DEC)
12    `uvm_field_int(x,       UVM_DEFAULT | UVM_DEC)
13    `uvm_field_int(y,       UVM_DEFAULT | UVM_DEC)
```

```

14  `uvm_object_utils_end
15
16  function new(string name = "cordic_item");
17      super.new(name);
18  endfunction
19
20  endclass

```

Listing 3.7. A sequence item with field automation

The block between ``uvm_object_utils_begin` and ``uvm_object_utils_end` is the *field automation*. Each ``uvm_field_int` adds the named field to a table that the base class walks whenever somebody calls `copy()`, `compare()`, `print()`, `pack()`, `unpack()`, `record()` or `sprint()`. You get seven methods for five lines. The flags select which of them the field takes part in: `UVM_DEFAULT` means all of them, `UVM_DEC` is a formatting hint, and you can subtract behaviours with, for example, `UVM_DEFAULT | UVM_NOCOMPARE` for a field the scoreboard should ignore.

Field macros are convenient and slow

The automation is implemented by a generic recursive walk over a table of field descriptors, driven by string comparisons and `$cast`. It is one to two orders of magnitude slower than the hand-written equivalent, and it is executed on every single transaction. On a testbench that moves millions of items, the field macros can become a visible fraction of total run time.

Many experienced teams therefore do not use them at all. They register the type with the plain ``uvm_object_utils` and write the three methods they actually need:

```

1  `uvm_object_utils(cordic_item)
2
3  function void do_copy(uvm_object rhs);
4      cordic_item that;
5      super.do_copy(rhs);
6      if (!$cast(that, rhs))
7          `uvm_fatal("CAST", "do_copy: type mismatch")
8      x0 = that.x0; y0 = that.y0; theta = that.theta;
9      x = that.x; y = that.y;
10 endfunction
11
12 function bit do_compare(uvm_object rhs, uvm_comparer comparer);
13     cordic_item that;
14     if (!$cast(that, rhs)) return 0;
15     return super.do_compare(rhs, comparer)
16         && (x == that.x) && (y == that.y);
17 endfunction
18
19 function string convert2string();
20     return $sformatf("x0=%0d y0=%0d theta=%0d -> x=%0d y=%0d",
21                     x0, y0, theta, x, y);
22 endfunction

```

That is more lines, but it is explicit, fast, and it is what you will see in most industrial code. Use the macros while you are learning; be ready to delete them when a profiler tells you to.

△ Trap: `uvm\_field\_int` on a *real*

The field macros come in typed flavours and you must pick the right one: ``uvm_field_int` for integral types, ``uvm_field_real` for *real* and *shortreal*, ``uvm_field_enum`, ``uvm_field_string`, ``uvm_field_object`, ``uvm_field_array_int`, ``uvm_field_queue_object`, and so on, roughly thirty of them.

Writing ``uvm_field_int(gain, UVM_DEFAULT)` where *gain* is a *real* does not always fail to compile. The macro body assigns through a `uvm_bitstream_t` (a 4096-bit vector), so a *real* may be implicitly converted to an integer, and the fractional part is silently lost. Your comparison then passes on values that differ, or your print shows 1 where the value is 1.6468.

There is a deeper reason not to put a *real* in a transaction at all: a transaction should hold what crossed the pins, and pins carry bits. Keep the fixed-point value; convert to *real* inside the scoreboard, where the reference model lives.

3.8.2 The sequence

A sequence is a `uvm_object` whose `body()` task generates items. It is the direct replacement for `send_random()`.

```

1 class cordic_random_seq extends uvm_sequence #(cordic_item);
2   `uvm_object_utils(cordic_random_seq)
3
4   rand int unsigned n_items = 200;
5   constraint c_n { n_items inside {[1:5000]}; }
6
7   function new(string name = "cordic_random_seq");
8     super.new(name);
9   endfunction
10
11  task body();
12    cordic_item it;
13    repeat (n_items) begin
14      it = cordic_item::type_id::create("it");
15      start_item(it);
16      if (!it.randomize())
17        `uvm_fatal(get_type_name(), "item randomization failed")
18      finish_item(it);
19    end
20  endtask
21
22 endclass

```

Listing 3.8. A random sequence

Three details deserve attention.

The parameter. `uvm_sequence #(cordic_item)` is really `uvm_sequence #(REQ, RSP)` with `RSP` defaulting to `REQ`. Unless you use the response path (Section 3.8.6), one parameter is enough.

The randomisation is late. The item is randomised *between* `start_item` and `finish_item`, not before `start_item`. That is deliberate. `start_item` blocks until the sequencer grants this sequence the right to send, and only then does the item become the “current item”; randomising after the grant means the constraints see the state of the world at the moment the item will actually be driven, and it allows a sequencer-level constraint layer to influence the result. Randomising before `start_item` works too, and is common, but it is the less general pattern.

The sequence itself is randomisable. `n_items` is *rand*, so the test can write `seq.randomize()`

with { n_items inside {[100:200]}; }. This is the capability a task simply does not have.

The ``uvm_do` family compresses the create/start/randomise/finish sequence into one line:

```
1 task body();
2     cordic_item it;
3     repeat (n_items)
4         `uvm_do_with(it, { theta inside {[-PI_EXT : PI_EXT]}; })
5 endtask
```

``uvm_do`, ``uvm_do_with`, ``uvm_create`, ``uvm_send` and their `_on` variants (which take an explicit sequencer) are still widely used. They are also widely disliked: they hide the handshake you are trying to learn, they make the constraint block hard to read, and their error messages point inside the macro. Write the long form until the handshake is second nature.

3.8.3 Starting a sequence

A sequence does not run by itself. Somebody calls `start()` on it and tells it which sequencer to run on:

```
1 cordic_random_seq seq;
2 seq = cordic_random_seq::type_id::create("seq");
3 void'(seq.randomize() with { n_items == 500; });
4 seq.start(env.agt.sqr); // blocks until body() returns
```

`start()` is a task and it blocks, which is exactly what you want inside an objection. The full signature is `start(sequencer, parent_seq, priority, call_pre_post)`; the extra arguments matter only for nested sequences.

Inside `body()`, the handle `m_sequencer` refers to the sequencer the sequence is running on, typed as `uvm_sequencer_base`. If your sequencer is a derived class with extra members, say a pointer to a configuration object or a handle to another agent's sequencer, you need a typed handle. The macro ``uvm_declare_p_sequencer(type)` declares `p_sequencer` and casts `m_sequencer` into it in `pre_body`:

```
1 class cordic_cfg_seq extends uvm_sequence #(cordic_item);
2     `uvm_object_utils(cordic_cfg_seq)
3     `uvm_declare_p_sequencer(cordic_sequencer)
4     // ...
5     task body();
6         // p_sequencer is a cordic_sequencer, not a uvm_sequencer_base
7         if (p_sequencer.cfg.stall_pct > 50) ...
8     endtask
9 endclass
```

3.8.4 Sequence libraries and virtual sequences

Two constructions appear in every large testbench and are worth recognising even though you will not write one this week.

A *sequence library* (`uvm_sequence_library #(REQ)`) is a sequence that picks other sequences and runs them, in random order, in a random selection, or round-robin. It is how a long soak test is built out of a dozen small directed sequences.

A *virtual sequence* is a sequence that runs on no protocol at all. It coordinates several real sequences on several sequencers, which is how you say “start the register configuration on the APB agent, wait for it to finish, then start data traffic on both the input and the output agents

at once”. Its sequencer, a *virtual sequencer*, holds handles to the real sequencers and drives nothing itself. Virtual sequences are where system-level tests actually live.

3.8.5 The sequencer–driver handshake

This is the part students get wrong most often, so read Figure 3.3 slowly.

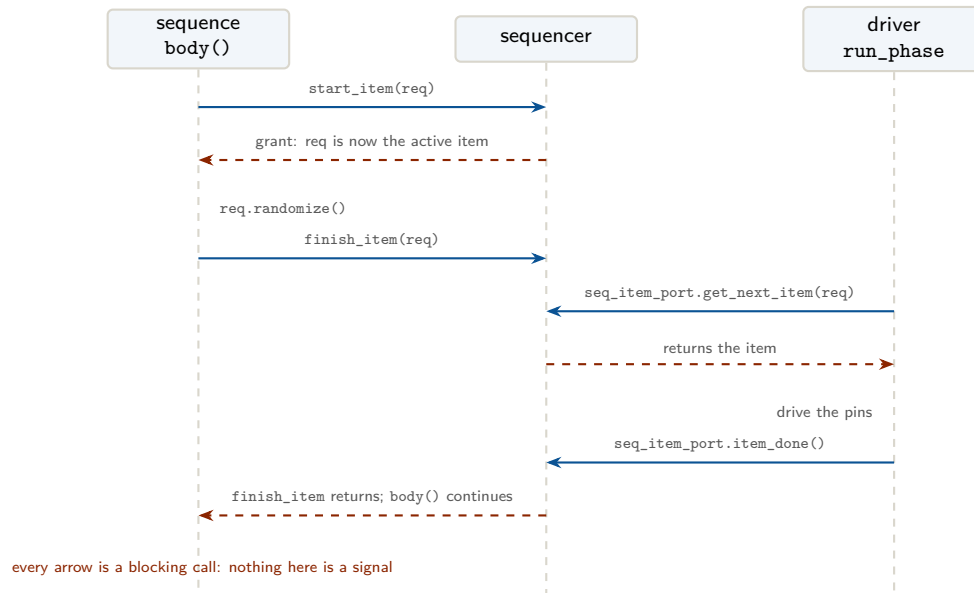


Figure 3.3. One round of the sequencer–driver handshake. The sequence blocks in `finish_item` from the moment it hands the item over until the driver calls `item_done`, which is what makes the driver’s back-pressure propagate all the way back to the stimulus.

On the driver side the loop is always the same three lines:

```

1 task run_phase(uvm_phase phase);
2   forever begin
3     seq_item_port.get_next_item(req); // blocks until an item arrives
4     drive(req);                       // your pin wiggling, takes time
5     seq_item_port.item_done();        // releases the sequence
6   end
7 endtask

```

Listing 3.9. The driver’s side of the handshake

`req` is not a variable you declare: it is inherited from `uvm_driver #(REQ, RSP)`, together with `rsp`. The port `seq_item_port` is inherited too. All you provide is `drive()`.

Notice what this buys you compared with the mailbox in Chapter 2. There, `req_mbx.get(t)` removed the transaction from the mailbox and the generator carried on immediately: the generator ran ahead of the driver, bounded only by the mailbox depth. Here, the sequence is blocked inside `finish_item` until `item_done()`, so the stimulus generation rate is exactly the rate at which the DUT accepts requests. Back-pressure propagates from the pins to the sequence for free.

△ Trap: Forgetting `item_done()`

```

1 forever begin
2   seq_item_port.get_next_item(req);

```

```

3   drive(req);
4   // item_done() missing
5 end

```

The first item is driven correctly. The second `get_next_item` then blocks forever, because the sequencer still considers the first item outstanding. Your simulation drives exactly one transaction and then hangs until the global timeout, with no error message and a waveform that shows one perfect handshake followed by silence.

The mirror-image bug is calling `item_done()` twice, or calling it without a matching `get_next_item`, which produces a `SEQREQZMB` error from the sequencer (“Item done() called with no outstanding requests”).

If your driver uses `try_next_item` instead, remember that `item_done()` must be called only when `try_next_item` actually returned a non-null handle.

3.8.6 The pipelined alternative

`get_next_item` is blocking, which is wrong for a driver that must keep the bus busy every cycle whether or not stimulus is available. The non-blocking form returns `null` instead of waiting:

```

1 task run_phase(uvm_phase phase);
2   forever begin
3     seq_item_port.try_next_item(req);
4     if (req == null) begin
5       drive_idle();           // hold valid low for one cycle
6       @(vif.drv_cb);
7     end else begin
8       drive(req);
9       seq_item_port.item_done();
10    end
11  end
12 endtask

```

Listing 3.10. A driver that never stalls the bus

For a genuinely pipelined protocol, where the response to a request arrives many cycles later and the driver must not block in the meantime, the pattern is two threads: one calls `get_next_item/item_done()` as fast as the request channel allows, the other collects responses and returns them with `seq_item_port.put_response(rsp)` or the combined `item_done(rsp)`. The sequence then reads them with `get_response(rsp)`. The CORDIC does not need this, because its monitor observes the responses independently, but you will meet it on any bus agent.

If the sequence never needs to see the result, do not use the response path. Let the monitor observe the result and send it to the scoreboard. Responses are for sequences that must make a decision based on what came back.

3.9 The agent

An agent packages everything that knows about one protocol: a sequencer, a driver and a monitor, plus whatever configuration they share. It is the unit of reuse, and it is the one box in Figure 3.1 that has no counterpart in Chapter 2.

`uvm_agent` adds exactly one thing to `uvm_component`: a field called `is_active`, of an enumerated type whose two values are `UVM_ACTIVE` and `UVM_PASSIVE`. The base class reads it from the

configuration database in its own `build_phase`, and `get_is_active()` returns it. An active agent builds all three sub-components and drives the pins. A passive agent builds only the monitor.

```

1 function void build_phase(uvm_phase phase);
2   super.build_phase(phase);           // reads is_active from the config db
3   ap = new("ap", this);
4   mon = cordic_monitor::type_id::create("mon", this);
5   if (get_is_active() == UVM_ACTIVE) begin
6     sqr = uvm_sequencer #(cordic_item)::type_id::create("sqr", this);
7     drv = cordic_driver::type_id::create("drv", this);
8   end
9 endfunction
10
11 function void connect_phase(uvm_phase phase);
12   mon.ap.connect(ap);
13   if (get_is_active() == UVM_ACTIVE)
14     drv.seq_item_port.connect(sqr.seq_item_export);
15 endfunction

```

Listing 3.11. The active/passive switch

Why does this matter enough to be built into the standard? Because of what happens when the block is integrated. At block level, your CORDIC agent is active: it generates the requests. At subsystem level, the CORDIC is fed by another block, and stimulus now arrives from the design, not from you. You still want the monitor: you still want the scoreboard to check every rotation, and you still want the coverage to be collected. So you instantiate *the same agent*, set it passive, and everything downstream of the monitor works unchanged.

```

1 // in the subsystem test's build_phase, before super.build_phase()
2 uvm_config_db #(uvm_active_passive_enum)::set(
3   this, "env.agt", "is_active", UVM_PASSIVE);

```

An agent written so that it can be dropped into somebody else's testbench, self-contained, configured only through the database, with no reference to a particular DUT, is what the industry calls a *VIP*, a verification IP. Commercial VIP for AXI, PCIe, USB or DDR is exactly this: a package of UVM classes with a documented configuration object, a documented sequence API, and a monitor whose analysis port you connect to your own scoreboard. Buying VIP instead of writing it is the main commercial reason UVM exists.

Recipe: Put the analysis port on the agent, not on the monitor

In Listing 3.11 the agent has its own `ap`, and the monitor's port is connected to it. The environment then connects `agt.ap`, never `agt.mon.ap`. This costs one line and it means the agent's internal structure is private: you can later split the monitor in two, or add a filter between the monitor and the port, without changing any environment that uses the agent.

3.10 TLM and the analysis path

TLM, transaction-level modelling, is the mechanism by which UVM components exchange objects. You have already met the two forms it takes: the blocking `seq_item_port` between sequencer and driver, and the broadcast analysis port out of the monitor. It is the analysis path that replaces the mailboxes of Chapter 2, and it is worth being precise about the difference.

A mailbox `#(T)` is a point-to-point queue. It has one producer and one consumer, the producer may block if it is bounded, and the consumer blocks until something arrives. In Chapter 2 the

monitor had to hold two mailboxes, `obs_mbx` and `cov_mbx`, and put each transaction into both. To add a third consumer you had to edit the monitor.

A `uvm_analysis_port #(T)` is a broadcast. The producer calls `write(t)`, which is a *function*: it never blocks, never consumes time, and returns immediately. The port forwards the call to every subscriber connected to it, in an unspecified order. Zero subscribers is legal and does nothing.

```

1 class cordic_monitor extends uvm_monitor;
2   uvm_analysis_port #(cordic_item) ap;
3
4   function new(string name, uvm_component parent);
5     super.new(name, parent);
6     ap = new("ap", this);    // ports are NOT factory-created
7   endfunction
8   // ... in run_phase, when a transaction is complete:
9   //   ap.write(it);
10 endclass

```

Listing 3.12. Producing on an analysis port

3.10.1 Three ways to consume

`uvm_subscriber #(T)` is the simplest. It is a component with a built-in `analysis_export` and one pure virtual method, `write(T t)`, which you implement. There is no thread and no queue: your `write` runs inside the monitor's call stack. That makes it ideal for coverage sampling, which is a function anyway, and unsuitable for anything that needs to consume time.

`uvm_analysis_imp #(T, PARENT)` is the general form: it forwards `write()` to a method named `write` in the parent component. A component can have only one such method, so a scoreboard that must listen to two monitors, one on the input and one on the output, cannot use two plain imps. The macro ``uvm_analysis_imp_decl` solves it by generating a suffixed variant:

```

1 // At package scope, outside any class:
2 `uvm_analysis_imp_decl(_req)
3 `uvm_analysis_imp_decl(_rsp)
4
5 class two_port_sb extends uvm_scoreboard;
6   `uvm_component_utils(two_port_sb)
7
8   uvm_analysis_imp_req #(cordic_item, two_port_sb) req_imp;
9   uvm_analysis_imp_rsp #(cordic_item, two_port_sb) rsp_imp;
10
11   function new(string name, uvm_component parent);
12     super.new(name, parent);
13     req_imp = new("req_imp", this);
14     rsp_imp = new("rsp_imp", this);
15   endfunction
16
17   function void write_req(cordic_item t); /* ... */ endfunction
18   function void write_rsp(cordic_item t); /* ... */ endfunction
19 endclass

```

Listing 3.13. Two analysis inputs on one scoreboard

The macro invents the class `uvm_analysis_imp_req` and makes it call `write_req` instead of `write`. It must appear at package or compilation-unit scope, exactly once per suffix, or you get a duplicate definition error when two files declare the same suffix.

`uvm_tlm_analysis_fifo #(T)` is what a scoreboard usually wants. It is an unbounded FIFO with

an `analysis_export` on the input side and a blocking `get()` on the output side. The monitor's `write()` still returns immediately, because the FIFO's `write` only pushes, but the checking code becomes an ordinary `run_phase` loop:

```

1 uvm_tlm_analysis_fifo #(cordic_item) fifo;
2 // ...
3 task run_phase(uvm_phase phase);
4     cordic_item it;
5     forever begin
6         fifo.get(it);           // blocks until the monitor writes
7         check(it);             // may take time, may call other tasks
8     end
9 endtask

```

Coverage collectors get a `uvm_subscriber`. Scoreboards get a `uvm_tlm_analysis_fifo`. Use a bare `uvm_analysis_imp` only when you need several differently-named inputs and none of them blocks.

Why does the FIFO matter? Because `write()` is a function and cannot consume time. Any checking that needs to wait, for a matching transaction from another interface, for a register read, or for a model that itself takes time, cannot be written inside `write()`. Put the item in a FIFO and do the work in a task. It also decouples the two: if the checker is slow, the monitor is not held up, because the FIFO is unbounded.

The rest of TLM

UVM also provides the full TLM-1 port set (`uvm_blocking_put_port`, `uvm_nonblocking_get_port`, `uvm_transport_port` and their exports and imps) and a TLM-2.0 subset with generic payloads and sockets, for connecting to SystemC models. You will not need them for a block-level testbench. The three constructions above cover nearly every real use.

3.11 Reporting

Stop using `$display`. UVM has four message macros, and using them is not a style preference: the simulation's pass/fail verdict is computed from them.

```

1 `uvm_info(ID, MESSAGE, VERBOSITY) // informational; filtered by verbosity
2 `uvm_warning(ID, MESSAGE)         // counted, does not fail the test
3 `uvm_error(ID, MESSAGE)           // counted, FAILS the test, run continues
4 `uvm_fatal(ID, MESSAGE)           // prints and terminates immediately

```

ID is a short string used for filtering; `get_type_name()` is a sensible default. MESSAGE is a string, so build it with `$sformatf`. Note that ``uvm_error` takes no verbosity: errors are never filtered.

△ Trap: The macros take no semicolon and no trailing comma

```

1 `uvm_info("DRV", $sformatf("n=%0d", n), UVM_LOW) // correct
2 `uvm_info("DRV", $sformatf("n=%0d", n), UVM_LOW); // stray empty statement
3 `uvm_error("SB", "mismatch", UVM_LOW)             // wrong arity: compile error

```

The stray semicolon is harmless after a statement but becomes a syntax error if the macro is the only thing inside an `if` without `begin/end`, which is precisely where you will hit it.

Get into the habit of writing the macros without a semicolon, and of putting `begin/end` around any `if` body that contains one.

3.11.1 Verbosity

Each ``uvm_info` carries a verbosity level: `UVM_NONE` (0), `UVM_LOW` (100), `UVM_MEDIUM` (200), `UVM_HIGH` (300), `UVM_FULL` (400), `UVM_DEBUG` (500). A message is printed if its level is *less than or equal to* the current threshold, which defaults to `UVM_MEDIUM`. So `UVM_NONE` always prints and `UVM_DEBUG` almost never does.

The threshold is set on the command line,

```
vsim -c +UVM_VERBOSITY=UVM_HIGH ...
```

or, more usefully, per subtree at run time:

```
1 // only the driver becomes chatty; everything else stays at MEDIUM
2 env.agt.drv.set_report_verbosity_level_hier(UVM_HIGH);
3
4 // or by message ID, anywhere in the tree
5 uvm_top.set_report_id_verbosity_hier("HANDSHAKE", UVM_FULL);
```

Recipe: Choose verbosity levels with a policy

A convention that works: `UVM_NONE` for the end-of-test summary that must always appear; `UVM_LOW` for once-per-test events (test started, configuration chosen, sequence finished); `UVM_MEDIUM` for once-per-transaction; `UVM_HIGH` for once-per-clock-cycle detail. A regression then runs at the default and produces a readable log, and a failing test can be re-run at `UVM_HIGH` for the same seed without recompiling.

3.11.2 The verdict

At the end of the run, `uvm_report_server` prints a summary:

```
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 427
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
[RNTST] 1
[cdic_driver] 500
[cdic_scoreboard] 2
[TEST_DONE] 1
```

Read the `UVM_ERROR` and `UVM_FATAL` lines: those two numbers are the verdict. A regression script greps for exactly this block. Note what this implies:

- A test that calls `$finish` directly bypasses the summary and produces no verdict. Never call `$finish` in a UVM testbench.
- A check written with `$error` instead of ``uvm_error` is not counted, so a failing test reports zero errors and the regression records a pass.

- A test in which nothing happened also reports zero errors. This is why the scoreboard in Section 3.12 raises an error when it has checked nothing: “no failures” and “no checks” must not look the same.

Two command-line switches complete the picture. The switch `+UVM_TESTNAME` names the `uvm_test` subclass that `run_test()` builds, so one compiled image runs every test in the suite. And `+UVM_MAX_QUIT_COUNT` stops the run after a given number of errors, which keeps a broken test from producing a gigabyte of log.

3.12 A complete UVM testbench for the CORDIC

Everything so far, assembled. This is a genuinely runnable testbench: type it in, compile it against `cordic_pkg.sv`, `cordic_if.sv` and `cordic_rot.sv` from Chapter 2, and it will find bugs in the CORDIC. Each listing names the class from `tb_cordic_pkg.sv` that it replaces.

Everything except the top-level module lives in one package:

```

1  `timescale 1ns/1ps
2
3  package cordic_uvm_pkg;
4
5  import uvm_pkg::*;
6  `include "uvm_macros.svh"
7  import cordic_pkg::*;
8
9  // The CORDIC processing gain and the comparison tolerance, exactly as in
10 // the hand-written testbench of the previous chapter.
11 localparam real K_GAIN = 1.6467602581210656;
12 localparam int TOL_LSB = 12;
```

Listing 3.14. Package header

3.12.1 The sequence item

Replaces `cordic_txn`. Compare it with the hand-written class of Listing 2.14: the constraints are identical, in the ordinary SystemVerilog form of Chapter 1, the `convert2string` is identical, and everything else is base-class machinery.

```

1  class cordic_item extends uvm_sequence_item;
2
3  rand data_t x0;
4  rand data_t y0;
5  rand angle_t theta;
6
7  data_t x;           // observed, filled in by the monitor
8  data_t y;
9
10 `uvm_object_utils_begin(cordic_item)
11   `uvm_field_int(x0, UVM_DEFAULT | UVM_DEC)
12   `uvm_field_int(y0, UVM_DEFAULT | UVM_DEC)
13   `uvm_field_int(theta, UVM_DEFAULT | UVM_DEC)
14   `uvm_field_int(x, UVM_DEFAULT | UVM_DEC)
15   `uvm_field_int(y, UVM_DEFAULT | UVM_DEC)
16 `uvm_object_utils_end
17
18 constraint c_theta {
19   theta inside {[ -PI_EXT : PI_EXT ]};
20 }
```

```

21  constraint c_magnitude {
22      x0 inside {[ -6000 : 6000]};
23      y0 inside {[ -6000 : 6000]};
24  }
25  constraint c_corners {
26      theta dist {
27          0                :/ 1,
28          PI_EXT           :/ 1,
29          -PI_EXT          :/ 1,
30          PI_EXT / 2       :/ 1,
31          -PI_EXT / 2      :/ 1,
32          [-PI_EXT:PI_EXT] :/ 20
33      };
34  }
35
36  function new(string name = "cordic_item");
37      super.new(name);
38  endfunction
39
40  function string convert2string();
41      return $sformatf(
42          "x0=%0d (%.4f) y0=%0d (%.4f) theta=%0d (%.4f rad) -> x=%0d y=%0d",
43          x0, q_to_real(x0, QF), y0, q_to_real(y0, QF),
44          theta, q_to_real(theta, QA), x, y);
45  endfunction
46
47  endclass : cordic_item

```

Listing 3.15. cordic_item: replaces cordic_txn

3.12.2 The sequence

Replaces `cordic_env::send_random()`. Note that it is now a class, so a directed test can extend it, and a regression can select it by name.

```

1  class cordic_random_seq extends uvm_sequence #(cordic_item);
2      `uvm_object_utils(cordic_random_seq)
3
4      rand int unsigned n_items = 200;
5      constraint c_n { n_items inside {[1 : 5000]}; }
6
7      function new(string name = "cordic_random_seq");
8          super.new(name);
9      endfunction
10
11     task body();
12         cordic_item it;
13         `uvm_info(get_type_name(),
14             $sformatf("sending %0d random items", n_items), UVM_LOW)
15         repeat (n_items) begin
16             it = cordic_item::type_id::create("it");
17             start_item(it);
18             if (!it.randomize())
19                 `uvm_fatal(get_type_name(), "item randomization failed")
20             finish_item(it);
21         end
22     endtask
23
24     endclass : cordic_random_seq
25
26     // The directed counterpart: the corner angles, with a unit-length seed so

```

```

27 // that the output is (cos theta, sin theta) directly.
28 class cordic_corner_seq extends uvm_sequence #(cordic_item);
29   `uvm_object_utils(cordic_corner_seq)
30
31   function new(string name = "cordic_corner_seq");
32     super.new(name);
33   endfunction
34
35   task body();
36     cordic_item it;
37     angle_t angles[$] = '{0, PI_EXT, -PI_EXT, PI_EXT/2, -PI_EXT/2,
38                          PI_EXT/4, -PI_EXT/4};
39     foreach (angles[i]) begin
40       it = cordic_item::type_id::create("it");
41       start_item(it);
42       it.x0 = data_t'(X0_UNIT);
43       it.y0 = '0;
44       it.theta = angles[i];
45       finish_item(it);
46     end
47   endtask
48
49 endclass : cordic_corner_seq

```

Listing 3.16. cordic_random_seq and a directed sequence

3.12.3 The driver

Replaces cordic_drv. The send() and reset() bodies are line-for-line the ones from Chapter 2; what changed is where the transactions come from and who starts the thread.

```

1  class cordic_driver extends uvm_driver #(cordic_item);
2    `uvm_component_utils(cordic_driver)
3
4    virtual cordic_if vif;
5    int unsigned stall_pct = 25; // random back-pressure on rsp_ready
6
7    function new(string name, uvm_component parent);
8      super.new(name, parent);
9    endfunction
10
11    function void build_phase(uvm_phase phase);
12      super.build_phase(phase);
13      if (!uvm_config_db #(virtual cordic_if)::get(this, "", "vif", vif))
14        `uvm_fatal("NOVIF",
15                  {"no virtual interface for ", get_full_name(), ".vif"})
16      void' (uvm_config_db #(int unsigned)::get(this, "", "stall_pct",
17                                                stall_pct));
18    endfunction
19
20    task run_phase(uvm_phase phase);
21      do_reset();
22      fork
23        response_backpressure();
24      join_none
25      forever begin
26        seq_item_port.get_next_item(req);
27        drive(req);
28        seq_item_port.item_done();
29      end
30    endtask

```

```

31
32     protected task do_reset();
33         vif.rst_n          <= 1'b0;
34         vif.driv_cb.req_valid <= 1'b0;
35         vif.driv_cb.req_x    <= '0;
36         vif.driv_cb.req_y    <= '0;
37         vif.driv_cb.req_theta <= '0;
38         vif.driv_cb.rsp_ready <= 1'b0;
39         repeat (5) @(vif.driv_cb);
40         vif.rst_n <= 1'b1;
41         @(vif.driv_cb);
42     endtask
43
44     // Drive one request and hold it until the DUT accepts it. Marked
45     // `virtual' so that a factory override can replace it -- this is what
46     // the error-injecting driver shown earlier extends.
47     protected virtual task drive(cordic_item it);
48         vif.driv_cb.req_valid <= 1'b1;
49         vif.driv_cb.req_x    <= it.x0;
50         vif.driv_cb.req_y    <= it.y0;
51         vif.driv_cb.req_theta <= it.theta;
52         do @(vif.driv_cb); while (vif.driv_cb.req_ready !== 1'b1);
53         vif.driv_cb.req_valid <= 1'b0;
54         `uvm_info(get_type_name(),
55                 {"drove ", it.convert2string()}, UVM_HIGH)
56     endtask
57
58     protected task response_backpressure();
59         forever begin
60             if ($urandom_range(99) < stall_pct) begin
61                 vif.driv_cb.rsp_ready <= 1'b0;
62                 repeat ($urandom_range(3) + 1) @(vif.driv_cb);
63             end
64             vif.driv_cb.rsp_ready <= 1'b1;
65             @(vif.driv_cb);
66         end
67     endtask
68
69 endclass : cordic_driver

```

Listing 3.17. cordic_driver: replaces cordic_drv

3.12.4 The monitor

Replaces cordic_mon. The two mailboxes have become one analysis port.

```

1     class cordic_monitor extends uvm_monitor;
2         `uvm_component_utils(cordic_monitor)
3
4         virtual cordic_if          vif;
5         uvm_analysis_port #(cordic_item) ap;
6
7         protected cordic_item pending[$];    // a queue: accepted, not answered
8
9         function new(string name, uvm_component parent);
10             super.new(name, parent);
11             ap = new("ap", this);
12         endfunction
13
14         function void build_phase(uvm_phase phase);
15             super.build_phase(phase);

```

```

16     if (!uvm_config_db #(virtual cordic_if)::get(this, "", "vif", vif))
17         `uvm_fatal("NOVIF",
18             {"no virtual interface for ", get_full_name(), ".vif"})
19     endfunction
20
21     task run_phase(uvm_phase phase);
22         cordic_item it;
23         forever begin
24             @(vif.mon_cb);
25             if (vif.rst_n !== 1'b1) begin
26                 pending.delete();
27                 continue;
28             end
29
30             // Request accepted: remember what went in.
31             if (vif.mon_cb.req_valid === 1'b1 &&
32                 vif.mon_cb.req_ready === 1'b1) begin
33                 it = cordic_item::type_id::create("it");
34                 it.x0 = vif.mon_cb.req_x;
35                 it.y0 = vif.mon_cb.req_y;
36                 it.theta = vif.mon_cb.req_theta;
37                 pending.push_back(it);
38             end
39
40             // Response accepted: complete the oldest outstanding request and
41             // broadcast it. Every subscriber sees the same handle, so nobody
42             // downstream may modify it.
43             if (vif.mon_cb.rsp_valid === 1'b1 &&
44                 vif.mon_cb.rsp_ready === 1'b1) begin
45                 if (pending.size() == 0) begin
46                     `uvm_error(get_type_name(),
47                         "response with no outstanding request")
48                 end else begin
49                     it = pending.pop_front();
50                     it.x = vif.mon_cb.rsp_x;
51                     it.y = vif.mon_cb.rsp_y;
52                     ap.write(it);
53                 end
54             end
55         end
56     endtask
57
58     // Runs after the run phase: anything still pending is a lost response.
59     function void check_phase(uvm_phase phase);
60         super.check_phase(phase);
61         if (pending.size() != 0)
62             `uvm_error(get_type_name(),
63                 $sformatf("%0d requests never produced a response",
64                     pending.size()))
65     endfunction
66
67 endclass : cordic_monitor

```

Listing 3.18. cordic_monitor: replaces cordic_mon

The check_phase is new. In Chapter 2 a lost response simply meant the drain loop timed out and \$fatal fired with an unhelpful message. Here the check is explicit and it names the number of lost transactions.

3.12.5 The scoreboard

Replaces `cordic_sb`. The reference model is unchanged: it is written against the specification, in real arithmetic, and knows nothing about micro-rotations.

```

1  class cordic_scoreboard extends uvm_scoreboard;
2      `uvm_component_utils(cordic_scoreboard)
3
4      uvm_tlm_analysis_fifo #(cordic_item) fifo;
5
6      int unsigned n_checked = 0;
7      int unsigned n_failed  = 0;
8      real        worst_err = 0.0;
9
10     function new(string name, uvm_component parent);
11         super.new(name, parent);
12         fifo = new("fifo", this);
13     endfunction
14
15     // The golden model: the specification, not the implementation.
16     function void predict(input cordic_item it,
17                           output real x_ref, output real y_ref);
18         real th = q_to_real(it.theta, QA);
19         real xr = q_to_real(it.x0,    QF);
20         real yr = q_to_real(it.y0,    QF);
21         x_ref = K_GAIN * (xr * $cos(th) - yr * $sin(th));
22         y_ref = K_GAIN * (xr * $sin(th) + yr * $cos(th));
23     endfunction
24
25     task run_phase(uvm_phase phase);
26         cordic_item it;
27         real x_ref, y_ref, ex, ey;
28         forever begin
29             fifo.get(it);
30             predict(it, x_ref, y_ref);
31
32             ex = (q_to_real(it.x, QF) - x_ref) * (2.0 ** QF); // error in LSB
33             ey = (q_to_real(it.y, QF) - y_ref) * (2.0 ** QF);
34             if (ex < 0.0) ex = -ex;
35             if (ey < 0.0) ey = -ey;
36             if (ex > worst_err) worst_err = ex;
37             if (ey > worst_err) worst_err = ey;
38
39             n_checked++;
40             if (ex > real'(TOL_LSB) || ey > real'(TOL_LSB)) begin
41                 n_failed++;
42                 `uvm_error(get_type_name(),
43                     $sformatf("MISMATCH %s want x=%.1f y=%.1f err %.2f / %.2f LSB",
44                         it.convert2string(),
45                         x_ref * (2.0 ** QF), y_ref * (2.0 ** QF), ex, ey))
46             end
47         end
48     endtask
49
50     function void check_phase(uvm_phase phase);
51         super.check_phase(phase);
52         // "no failures" and "no checks" must not look the same.
53         if (n_checked == 0)
54             `uvm_error(get_type_name(), "no transactions were checked")
55         if (!fifo.is_empty())
56             `uvm_error(get_type_name(), "transactions left unchecked in the fifo")
57     endfunction

```

```

58
59     function void report_phase(uvm_phase phase);
60         `uvm_info(get_type_name(),
61             $sformatf("%0d checked, %0d failed, worst error %.2f LSB (tol %0d)",
62                 n_checked, n_failed, worst_err, TOL_LSB), UVM_NONE)
63     endfunction
64
65 endclass : cordic_scoreboard

```

Listing 3.19. cordic_scoreboard: replaces cordic_sb

3.12.6 The coverage collector

Replaces cordic_cov. The covergroup is character-for-character the one from Chapter 2; what disappeared is the mailbox and the forever loop, because write() is called for us.

```

1  class cordic_coverage extends uvm_subscriber #(cordic_item);
2      `uvm_component_utils(cordic_coverage)
3
4      angle_t sampled_theta;
5      data_t  sampled_x0;
6
7      covergroup cg_stimulus;
8          option.per_instance = 1;
9
10         cp_quadrant : coverpoint sampled_theta {
11             bins q3_neg = {[ -PI_EXT      : -PI_EXT/2 - 1]};
12             bins q4_neg = {[ -PI_EXT/2    : -1]};
13             bins zero   = {0};
14             bins q1_pos = {[ 1           : PI_EXT/2 - 1]};
15             bins q2_pos = {[ PI_EXT/2    : PI_EXT]};
16         }
17         cp_sign_x : coverpoint sampled_x0 {
18             bins neg = {[ -32768 : -1]};
19             bins zero = {0};
20             bins pos = {[ 1 : 32767]};
21         }
22         x_quadrant_sign : cross cp_quadrant, cp_sign_x;
23     endgroup
24
25     function new(string name, uvm_component parent);
26         super.new(name, parent);
27         cg_stimulus = new(); // covergroups are not factory objects
28     endfunction
29
30     // uvm_subscriber declares this pure virtual; the argument must be
31     // named `t'. It is a function: it cannot consume time.
32     function void write(cordic_item t);
33         sampled_theta = t.theta;
34         sampled_x0    = t.x0;
35         cg_stimulus.sample();
36     endfunction
37
38     function void report_phase(uvm_phase phase);
39         `uvm_info(get_type_name(),
40             $sformatf("functional coverage %.1f %%",
41                 cg_stimulus.get_coverage()), UVM_NONE)
42     endfunction
43
44 endclass : cordic_coverage

```

Listing 3.20. cordic_coverage: replaces cordic_cov

3.12.7 The agent and the environment

Replaces `cordic_env`, split into the reusable half (the agent) and the DUT-specific half (the environment).

```

1  class cordic_agent extends uvm_agent;
2      `uvm_component_utils(cordic_agent)
3
4      cordic_driver      drv;
5      uvm_sequencer #(cordic_item)  sqr;
6      cordic_monitor      mon;
7      uvm_analysis_port #(cordic_item) ap;
8
9      function new(string name, uvm_component parent);
10         super.new(name, parent);
11     endfunction
12
13     function void build_phase(uvm_phase phase);
14         super.build_phase(phase);      // reads is_active from the config db
15         ap = new("ap", this);
16         mon = cordic_monitor::type_id::create("mon", this);
17         if (get_is_active() == UVM_ACTIVE) begin
18             sqr = uvm_sequencer #(cordic_item)::type_id::create("sqr", this);
19             drv = cordic_driver::type_id::create("drv", this);
20         end
21     endfunction
22
23     function void connect_phase(uvm_phase phase);
24         mon.ap.connect(ap);      // republish through the agent's port
25         if (get_is_active() == UVM_ACTIVE)
26             drv.seq_item_port.connect(sqr.seq_item_export);
27     endfunction
28
29 endclass : cordic_agent
30
31
32 class cordic_env extends uvm_env;
33     `uvm_component_utils(cordic_env)
34
35     cordic_agent      agt;
36     cordic_scoreboard sb;
37     cordic_coverage   cov;
38
39     function new(string name, uvm_component parent);
40         super.new(name, parent);
41     endfunction
42
43     function void build_phase(uvm_phase phase);
44         super.build_phase(phase);
45         agt = cordic_agent::type_id::create("agt", this);
46         sb = cordic_scoreboard::type_id::create("sb", this);
47         cov = cordic_coverage::type_id::create("cov", this);
48     endfunction
49
50     // One producer, two consumers. Adding a third costs one line here and
51     // nothing at all in the monitor.
52     function void connect_phase(uvm_phase phase);
53         agt.ap.connect(sb.fifo.analysis_export);
54         agt.ap.connect(cov.analysis_export);
55     endfunction
56

```

```
57 | endclass : cordic_env
```

Listing 3.21. cordic_agent and cordic_env

3.12.8 The tests

New: in Chapter 2 the test was the top-level module. Here it is a class, so the same compiled image runs all of them.

```
1 | class cordic_base_test extends uvm_test;
2 |   `uvm_component_utils(cordic_base_test)
3 |
4 |   cordic_env env;
5 |
6 |   function new(string name, uvm_component parent);
7 |     super.new(name, parent);
8 |   endfunction
9 |
10 |  function void build_phase(uvm_phase phase);
11 |    super.build_phase(phase);
12 |    env = cordic_env::type_id::create("env", this);
13 |  endfunction
14 |
15 |  function void end_of_elaboration_phase(uvm_phase phase);
16 |    super.end_of_elaboration_phase(phase);
17 |    uvm_top.print_topology();    // the tree, once, at time zero
18 |  endfunction
19 |
20 |  // Common to every derived test: give the DUT pipeline time to empty
21 |  // after the last request, before the run phase is killed.
22 |  protected virtual function void set_drain(uvm_phase phase);
23 |    phase.phase_done.set_drain_time(this, 500ns);
24 |  endfunction
25 |
26 | endclass : cordic_base_test
27 |
28 |
29 | class cordic_random_test extends cordic_base_test;
30 |   `uvm_component_utils(cordic_random_test)
31 |
32 |   function new(string name, uvm_component parent);
33 |     super.new(name, parent);
34 |   endfunction
35 |
36 |   task run_phase(uvm_phase phase);
37 |     cordic_random_seq seq;
38 |     phase.raise_objection(this, "random stimulus");
39 |     set_drain(phase);
40 |     seq = cordic_random_seq::type_id::create("seq");
41 |     if (!seq.randomize() with { n_items == 500; })
42 |       `uvm_fatal(get_type_name(), "sequence randomization failed")
43 |     seq.start(env.agt.sqr);
44 |     phase.drop_objection(this, "random stimulus");
45 |   endtask
46 |
47 | endclass : cordic_random_test
48 |
49 |
50 | class cordic_corner_test extends cordic_base_test;
51 |   `uvm_component_utils(cordic_corner_test)
52 |
```

```

53     function new(string name, uvm_component parent);
54         super.new(name, parent);
55     endfunction
56
57     task run_phase(uvm_phase phase);
58         cordic_corner_seq dseq;
59         cordic_random_seq rseq;
60         phase.raise_objection(this, "corners then random");
61         set_drain(phase);
62         dseq = cordic_corner_seq::type_id::create("dseq");
63         dseq.start(env.agt.sqr);
64         rseq = cordic_random_seq::type_id::create("rseq");
65         void'(rseq.randomize() with { n_items == 100; });
66         rseq.start(env.agt.sqr);
67         phase.drop_objection(this, "corners then random");
68     endtask
69
70 endclass : cordic_corner_test
71
72 endpackage : cordic_uvm_pkg

```

Listing 3.22. A base test and two derived tests

3.12.9 The top-level module

The only non-class file. It holds the clock, the interface, the DUT, the one configuration-database write and the call to `run_test()`.

```

1  `timescale 1ns/1ps
2
3  module tb_cordic_uvm;
4
5      import uvm_pkg::*;
6      `include "uvm_macros.svh"
7      import cordic_pkg::*;
8      import cordic_uvm_pkg::*;
9
10     // ---- clock -----
11     logic clk = 1'b0;
12     always #5ns clk = ~clk;          // 100 MHz
13
14     // ---- the pins -----
15     cordic_if vif (.clk(clk));
16
17     // ---- the DUT -----
18     cordic_rot #(.ITER(N)) dut (
19         .clk_i      (clk),
20         .rst_ni     (vif.rst_n),
21         .req_valid_i (vif.req_valid),
22         .req_ready_o (vif.req_ready),
23         .req_x_i     (vif.req_x),
24         .req_y_i     (vif.req_y),
25         .req_theta_i (vif.req_theta),
26         .rsp_valid_o (vif.rsp_valid),
27         .rsp_ready_i (vif.rsp_ready),
28         .rsp_x_o     (vif.rsp_x),
29         .rsp_y_o     (vif.rsp_y)
30     );
31
32     // ---- hand the interface to the class world, then start UVM -----
33     initial begin

```

```

34     uvm_config_db #(virtual cordic_if)::set(null, "*", "vif", vif);
35     uvm_top.set_timeout(1ms, 1);
36     run_test();                               // never returns
37 end
38
39 // Portable waveform dump, off by default: pass +DUMP to enable it. Name
40 // the scope; $dumpvars(0, tb_cordic_uvm) would dump the whole hierarchy.
41 initial if ($test$plusargs("DUMP")) begin
42     $dumpfile("tb_cordic_uvm.vcd");
43     $dumpvars(1, dut);
44 end
45
46 endmodule : tb_cordic_uvm

```

Listing 3.23. `tb_cordic_uvm`: the top-level module

What `run_test()` actually does

`run_test()` is a task in `uvm_pkg`. It reads `+UVM_TESTNAME`, asks the factory for an object of that type, gives it the name `uvm_test_top` and no parent, and then runs the phase schedule over the tree that the test's `build_phase` creates. When the last phase finishes it prints the report summary and calls `$finish` itself. It does not return, which is why nothing may follow it in the `initial` block.

If `+UVM_TESTNAME` names a class that is not registered with the factory, you get `UVM_FATAL ... [INVST] Requested test from command line +UVM_TESTNAME=... not found`. The usual cause is a missing `~uvm_component_utils`, or a package that was compiled but never imported by the top-level module, so its static registrations never ran.

3.13 The register layer

The CORDIC has no registers, so this section is a map rather than a tutorial. Any block with a control interface has a register map, and UVM provides a modelling layer for it.

uvm_reg_field one field: width, offset, access policy (RW, RO, W1C, RC, and about twenty others), reset value.

uvm_reg one register: a name, an offset, and its fields. Each register carries a *mirror*, the model's belief about the hardware's current value, and a *desired* value.

uvm_reg_block a group of registers and other blocks, with one or more address maps. The block is the unit that matches an IP-XACT component.

uvm_reg_adapter the translator. It converts a generic `uvm_reg_bus_op` into your bus agent's sequence item and back. You write one adapter per bus protocol, ever.

uvm_reg_predictor listens on the bus monitor's analysis port and updates the mirror from observed traffic, so that the model stays correct even when somebody else writes the registers.

With a model in place, a sequence stops talking about addresses:

```

1  uvm_status_e status;
2  uvm_reg_data_t value;
3
4  // front door: a real bus transaction, through the adapter and the driver
5  regmodel.ctrl.write(status, 32'h0000_0003, UVM_FRONTDOOR);
6  regmodel.ctrl.read (status, value,          UVM_FRONTDOOR);
7

```

```

8 // back door: a direct hierarchical poke, zero simulation time
9 regmodel.ctrl1.write(status, 32'h0000_0003, UVM_BACKDOOR);

```

A *front-door* access produces real cycles on the bus, exercises the bus interface, and takes time. A *back-door* access reaches into the design hierarchy with a hierarchical reference (the path is supplied by `add_hdl_path`) and changes the register directly, in zero time. Use the front door to verify the register interface, and the back door to set up a state you do not want to spend a thousand cycles reaching.

What the model buys you: a set of ready-made sequences that check every reset value, every writable bit and every access policy without a line of test code (`uvm_reg_hw_reset_seq` and its relatives); a mirror that lets a scoreboard ask “what is the current mode?” without snooping the bus; and tests written against register names, which survive an address-map change.

Nobody writes a register model by hand

A medium-sized block has a few hundred registers, and the model is roughly twenty lines per register. Generating it is standard practice. The input is whatever the project treats as the specification: an IP-XACT .xml component description, a SystemRDL file, or, very commonly, a spreadsheet exported to CSV. Tools such as PeakRDL, Semifore CSRCompiler, IDesignSpec or a home-made Python script emit the UVM model, the RTL, the C header and the documentation from the same source. If you find yourself editing a `uvm_reg_block` by hand, somebody has lost the generator.

3.14 Running it

UVM is a package. Compiling it is compiling one file with one include directory, and every simulator has a shortcut for it.

3.14.1 QuestaSim

The explicit form, which shows what is happening. It is the flow of Section 2.9 with one extra compile at the front:

```

vlib work
vmap work work

# 1. the UVM library itself (once; $UVM_HOME points into the Questa install)
vlog -sv +incdir+$UVM_HOME/src $UVM_HOME/src/uvm_pkg.sv

# 2. the design and the testbench
vlog -sv +incdir+$UVM_HOME/src cordic_pkg.sv cordic_if.sv cordic_rot.sv
vlog -sv +incdir+$UVM_HOME/src cordic_uvm_pkg.sv tb_cordic_uvm.sv

# 3. optimise, keeping enough visibility for the factory and the waveform
vopt -o tb_opt +acc=rnb work.tb_cordic_uvm

# 4. run one test
vsim -c -sv_seed random work.tb_opt \
    +UVM_TESTNAME=cordic_random_test \
    +UVM_VERBOSITY=UVM_MEDIUM \
    -do uvm.do

```

The short form uses the built-in library, which is pre-compiled and faster, and lets `vsim` run `vopt` for you:

```
vlog -sv -uvm cordic_pkg.sv cordic_if.sv cordic_rot.sv \
      cordic_uvm_pkg.sv tb_cordic_uvm.sv
vsim -c -uvm -voptargs="+acc=rnb" tb_cordic_uvm \
      +UVM_TESTNAME=cordic_random_test -sv_seed random -do uvm.do
```

The do file is shorter than the one in Section 2.9, because UVM computes the verdict itself and `run_test()` calls `$finish` when the last phase is over:

```
# uvm.do -- batch execution of a UVM test
onerror {quit -code 3}

# log the DUT only. `log -r /*' would also record every UVM class member.
log -r /tb_cordic_uvm/dut/*

run -all
quit -code 0
```

Note the scope on `log -r`. Logging the whole hierarchy in a UVM testbench records the class world as well as the pins, and a `.wlf` that would have been tens of megabytes becomes hundreds. QuestaSim shows that `.wlf` in its own wave window; the `$dumpfile/$dumpvars` pair in Listing 3.23 is the portable alternative, writing a VCD that GTKWave and Surfer can open, and the same warning applies to its first argument. Section 2.9.4 covers the choice.

Use `-uvmhome <path>` if you need a specific UVM version rather than the one bundled with the tool. Mixing a hand-compiled `uvm_pkg.sv` with the tool's built-in one produces duplicate-definition errors that are tedious to untangle, so pick one and be consistent across the whole project.

`-sv_seed random` is important. Without it every run uses the same seed, and a constrained-random testbench that always generates the same stimulus is a directed testbench with extra steps. Print the seed (Questa does so in the banner) so that a failing run can be reproduced exactly with `-sv_seed <n>`.

3.14.2 The other two

```
# Cadence Xcelium
xrun -uvm -sv cordic_pkg.sv cordic_if.sv cordic_rot.sv \
      cordic_uvm_pkg.sv tb_cordic_uvm.sv \
      +UVM_TESTNAME=cordic_random_test -svseed random

# Synopsys VCS
vcs -sverilog -ntb_opts uvm -full64 cordic_pkg.sv cordic_if.sv \
      cordic_rot.sv cordic_uvm_pkg.sv tb_cordic_uvm.sv -o simv
./simv +UVM_TESTNAME=cordic_random_test +ntb_random_seed_automatic
```

3.14.3 Reading the output

A successful run ends like this:

```
UVM_INFO cordic_scoreboard.sv(78) @ 26505 ns:
  uvm_test_top.env.sb [cordic_scoreboard]
  500 checked, 0 failed, worst error 3.41 LSB (tol 12)
UVM_INFO cordic_coverage.sv(52) @ 26505 ns:
  uvm_test_top.env.cov [cordic_coverage]
  functional coverage 93.3 %
UVM_INFO ../uvm_report_server.svh(847) @ 26505 ns:
  reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---
```

```

** Report counts by severity
UVM_INFO :    4
UVM_WARNING : 0
UVM_ERROR  : 0
UVM_FATAL  : 0

```

Every message carries the file, the line, the simulation time and the full hierarchical name of the component that produced it. That last field is the one you will use: `uvm_test_top.env.sb` tells you immediately which instance complained, which in a testbench with six agents is not obvious from the message text.

Recipe: Make the regression script read the right thing

Grep for the two lines `UVM_ERROR :` and `UVM_FATAL :` in the report summary and require both counts to be zero. Do not grep for the word “error” anywhere in the log: a message containing the word “error” in its text (`worst error 3.41 LSB`, above) will fail a passing test. Do not rely on the simulator’s exit status either, because a UVM run that ends with errors still exits normally.

3.15 When not to use UVM

This section exists because everything before it makes UVM look inevitable, and it is not.

A block with three ports and one protocol does not need it. If the whole verification effort is “drive a valid/ready channel, check the output against a model”, you will spend more lines on UVM infrastructure than on verification. The testbench in Chapter 2 does that job in about 470 lines that a new team member reads in twenty minutes.

A three-week project does not need it. The break-even point for UVM is somewhere around the moment you write your third test that reuses the second test’s environment. If the project ends before that, the investment never pays back.

A course exercise does not need it. A laboratory assignment is graded on whether the design works, and a UVM testbench in which the student does not understand the phase order teaches nothing about verification. Learn UVM *after* you can write Chapter 2’s testbench from memory, not instead.

A single-person project rarely needs it. Much of UVM’s value is social: a shared vocabulary, a shared structure, an agent one engineer can hand to another. With one engineer, the shared vocabulary is your own habits.

Where UVM does pay:

- **Reuse across blocks.** Five blocks on the same bus share one bus agent, one adapter and one register-model generator.
- **Reuse across levels.** The block-level agent goes passive and the same monitor, scoreboard and coverage run at subsystem and chip level.
- **Reuse across projects.** A protocol agent written once is used for a decade.
- **A team larger than one.** Structure that is enforced by a library is structure that survives people joining and leaving.
- **Commercial VIP.** If you buy an AXI or PCIe VIP, it is UVM, and your testbench must be too.

3.15.1 The alternatives, fairly

A hand-written layered testbench. What you built in Chapter 2. Full SystemVerilog power, meaning classes, constrained random, functional coverage and assertions, with none of the library. It scales further than people expect, and the point at which it stops scaling is usually the point at which you need reuse across teams. Many production testbenches for small blocks are exactly this.

cocotb, with or without pyuvm. The testbench is Python; the simulator is driven through VPI/VHPI. Stimulus, checking and the reference model are written in a language with real libraries: `numpy` for the CORDIC's golden model, `pytest` for the regression. `pyuvm` is a faithful port of the UVM class library to Python, so the six boxes and the phases keep their names. It drives QuestaSim through the FLI and VPI, and it also drives Verilator and Icarus Verilog, so a cocotb testbench can run without a licence. the *Simulation Cookbook* builds one.

OSVVM, for VHDL. A VHDL library providing randomisation, functional coverage, a scoreboard and a transaction-level interface, with no need to leave VHDL at all. It is mature, well documented, and runs under QuestaSim and under NVC. If your organisation is VHDL and intends to stay VHDL, OSVVM is the serious answer and deserves a look before a rewrite.

UVVM, for VHDL. Also VHDL, also open source, structured around VVCs, VHDL Verification Components, which occupy the same conceptual place as UVM agents, with a central sequencer issuing commands to them. UVVM emphasises a directed, readable command language over constrained-random generation, which suits control-path blocks well.

Choose the methodology that matches the size of the problem and the size of the team, not the one on the recruitment advertisement. Then learn UVM anyway, because the recruitment advertisement is real.

3.16 Checklist

- UVM is a SystemVerilog class library, not a language extension. Every rust identifier in this chapter's listings is something you could have written yourself, and in Chapter 2 you did.
- The six boxes are the same as in Chapter 2: transaction, stimulus, driver, monitor, scoreboard, coverage. UVM adds a base class for each, plus the agent, the test class, the factory, the configuration database and phasing.
- `uvm_component` = structure, built once, has a parent and a hierarchical name. `uvm_object` = data, created and destroyed constantly, has a name but no place.
- Always call `super.build_phase(phase)` first. Omitting it breaks configuration and the agent's active/passive switch, silently.
- Build is top-down (the parent creates the children); connect is bottom-up (both ends must exist). `run_phase` is the only phase in which time passes.
- Create components and sequence items with `type_id::create()`, never with `new`. `new` bypasses the factory and every override becomes a no-op with no warning.
- Set factory overrides *before* the component is created, which means before `super.build_phase()` in the test.
- Get the virtual interface from the module into the classes with a `uvm_config_db #(virtual cordic_if)::set` call placed before `run_test()`, and always test the return value of the

matching `get` with a ``uvm_fatal`.

- Raise an objection in the test before starting the sequence and drop it after. No objection, no test. Set a drain time long enough for the DUT pipeline to empty, and a global timeout so that a stuck objection fails instead of hanging.
- The driver loop is `get_next_item(req)`, `drive`, `item_done()`. Missing `item_done()` hangs after exactly one transaction.
- Randomise the item between `start_item` and `finish_item`, not before.
- Analysis ports broadcast and never block. Coverage collectors get a `uvm_subscriber`; scoreboards get a `uvm_tlm_analysis_fifo`, because checking may take time and `write()` may not.
- Report with ``uvm_info/`uvm_warning/`uvm_error/`uvm_fatal`, never with `$display` or `$error`. The verdict is the error count in the report summary, and `$error` is not counted.
- Make the scoreboard raise an error when it has checked nothing. “Zero failures” and “zero checks” print the same verdict otherwise.
- Never call `$finish` yourself: it skips the report summary.
- Select the test with `+UVM_TESTNAME`, the log level with `+UVM_VERBOSITY`, and always run with a random seed (`-sv_seed random`); record the seed so a failure can be reproduced.
- Debug switches worth memorising: `+UVM OBJECTION_TRACE` for a test that will not end, `+UVM_CONFIG_DB_TRACE` for a value that does not arrive, `uvm_top.print_topology()` for a tree that is not what you think it is.
- Field automation macros are convenient and slow. Learn with them, then write `do_copy`, `do_compare` and `convert2string` by hand when it matters. Never put a `real` in a ``uvm_field_int`.
- Do not reach for UVM on a small block, a short project, or a one-person effort. Reach for it when something will be reused: across blocks, across levels of hierarchy, across projects, or across a team.

Further Reading

Standards

- IEEE Std 1800-2017, *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*. The language reference manual. Chapter 9 (processes), Chapter 10 (assignment statements) and Chapter 4 (scheduling semantics) are the three a VHDL designer should read first.
- IEEE Std 1076-2008, *IEEE Standard VHDL Language Reference Manual*, and IEEE Std 1076-2019 for the newer features discussed in the *Design Cookbook*.
- IEEE Std 1800.2-2020, *Universal Verification Methodology Language Reference Manual*.
- IEEE Std 1364-2005, the last standalone Verilog standard, useful only when reading legacy code.

Books

- C. Spear and G. Tumbush, *SystemVerilog for Verification*, 3rd ed., Springer. The standard text on the class-based testbench, and the natural sequel to Chapter 2.
- S. Sutherland, S. Davidmann and P. Flake, *SystemVerilog for Design*, 2nd ed., Springer. The design-side counterpart, written by people who were in the room when the language was standardised.
- D. Thomas, *The Verilog Hardware Description Language*, and P. Ashenden, *The Designer's Guide to VHDL*. The two classics, for when you want to know why something is the way it is.
- P. Ashenden and J. Lewis, *VHDL-2008: Just the New Stuff*. Short, and it covers most of what the *Design Cookbook* says VHDL can do.
- M. Keating, *The Simple Art of SoC Design*. Not about either language, and worth reading anyway.

Papers and application notes

- C. Cummings, *Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!*, SNUG 2000. The clearest published explanation of the blocking/non-blocking rules and the races they exist to prevent. Everything else by the same author on `always_comb`, `unique/priority` and clock-domain crossing is worth the time.
- J. E. Volder, *The CORDIC Trigonometric Computing Technique*, IRE Transactions on Electronic Computers, 1959. The original paper behind the *Example Cookbook*.
- J. S. Walther, *A Unified Algorithm for Elementary Functions*, Spring Joint Computer Conference, 1971. The generalisation.
- R. Andraka, *A Survey of CORDIC Algorithms for FPGA Based Computers*, FPGA 1998. The practical implementation reference, including the gain, the convergence range and the quadrant folding used in the *Example Cookbook*.

Tools and online resources

- *Verilator* manual, <https://verilator.org/guide/>. Read the chapters on the language subset and on the C++ API before writing a C++ testbench.
- *cocotb* documentation, <https://docs.cocotb.org/>.
- *GTKWave* (<https://gtkwave.sourceforge.net>) and *Surfer* (<https://surfer-project.org>), the two waveform viewers that read VCD and FST.
- *OSVVM* (<https://osvvm.org>) and *UVVM* (<https://uvvm.org>), the two VHDL verification frameworks referred to throughout the *Design Cookbook* and Chapter 3.
- The *lowRISC* and *OpenTitan* SystemVerilog style guide, <https://github.com/lowRISC/style-guides>. The best free statement of what disciplined SystemVerilog looks like, and the source of the naming conventions used in this book.
- *Verible*, <https://github.com/chipsalliance/verible>, for formatting and linting.

Index

- [\\$display](#), 27
- [\\$dumpfile](#), 58
- [\\$error](#), 35
- [\\$fatal](#), 35
- [\\$finish](#), 26
- [\\$value\\$plusargs](#), 57
- array
 - [methods](#), 3
- [assertion](#), 15
 - [concurrent](#), 53
 - [immediate](#), 53
 - [sampling](#), 15
- [associative array](#), 1
- [automatic lifetime](#), 9, 52
- [AVM](#), 64
- [bind](#), 55
- [class](#), 3
 - [inheritance](#), 5
- [clocking block](#), 29
- [cocotb](#), 101
- [configuration \(VHDL\)](#), 74
- [constraint](#), 10
- [cover property](#), 56
- [coverage](#)
 - [functional](#), 12
- [coverage closure](#), 59
- [covergroup](#), 12, 45
- [drain](#), 53
- [driver](#), 41
- [dynamic array](#), 1
- [environment](#), 47
- [final block](#), 34
- [fork](#)
 - [join](#), 6
 - [join_none](#), 51
- [functional coverage](#), 45
- [generator](#), 40
- [golden model](#), 31
- [GTKWave](#), 58
- [IEEE 1800.2](#), 64
- [initial block](#), 25
- [mailbox](#), 6, 50
- [monitor](#), 43
- [OSVVM](#), 101
- [OVM](#), 64
- [plusarg](#), 57
- [polymorphism](#), 5
- [program block](#), 18
- [pyuvvm](#), 101
- [QuestaSim](#), 56
 - [UVM](#), 98
- [queue](#), 1
- [race condition](#), 27
- [randc](#), 10
- [randomisation](#), 10
- [reference model](#), 31
- [scoreboard](#), 44
- [self-checking testbench](#), 31
- [semaphore](#), 6
- [static lifetime](#), 9, 52
- [Surfer](#), 58
- [SVA](#), 15
- [test](#), 48
- [testbench](#)
 - [structure](#), 21
- [timescale](#), 24
- [timeunit](#), 24
- [tolerance](#), 33
- [transaction](#), 38
- [UVM](#)
 - [agent](#), 82
 - [analysis port](#), 83
 - [back-door access](#), 97
 - [build_phase](#), 70
 - [drain time](#), 71
 - [factory](#), 72
 - [factory override](#), 73
 - [field automation macros](#), 78, 79
 - [get_full_name](#), 67
 - [history](#), 63

- objection, [71](#)
- pass/fail verdict, [86](#)
- phases, [68](#)
- register layer, [97](#)
- run-time sub-phases, [70](#)
- seq_item_port, [81](#)
- sequence, [77](#)
- sequencer, [77](#)
- TLM, [83](#)
- type_id::create, [75](#)
- uvm_analysis_imp, [84](#)
- uvm_component, [67](#)
- uvm_config_db, [75](#)
- uvm_object, [67](#)
- uvm_subscriber, [84](#)
- UVM_TESTNAME, [87](#)
- uvm_tlm_analysis_fifo, [84](#)
- verbosity, [86](#)
- virtual sequence, [80](#)
- when not to use, [100](#)
- UVVM, [101](#)
- VCD, [58](#)
- VIP, [83](#)
- virtual interface, [41](#), [49](#), [75](#)
- VMM, [64](#)
- watchdog, [35](#)
- waveform, [58](#)